



Embedded Vision Intelligent Laboratory

微算機系統 MICROPROCESSOR SYSTEMS

Instructor : Yen-Lin Chen(陳彥霖)

Dept. Computer Science and Information Engineering

National Taipei University of Technology

ylchen@ntut.edu.tw



關於處理器指令處理的趨勢

早期

- 單一循序處理：指令在完成後才會接下一個指令，所以較為受限
- 基本流水線技術：分為多階段，可重疊執行

現代

- 超純量與亂序處理：
 - 超純量處理器：在一個時脈週期啟動多條指令。
 - 亂序執行：動態排程，繞過阻塞指令
- 分支預測與推測執行：
 - 分支預測：在分支指令執行結束之前猜測哪一路分支將會被執行
 - 推測執行：在處理器系統資源過剩的情況下‘並列’處理其他任務



關於處理器指令處理的關聯性

1. 硬體、軟體協同設計

- 編譯器最佳化與處理器架構的協同：編譯器產生適合的指令訊列；處理器提供足夠的指令資訊
- 即時編譯與動態最佳化：適應不同處理器特性

2. 記憶體架構與指令處理

- 記憶體階層：快取設計與預取機制會影響執行效率減少等待時間
- NUMA：一種多處理器的電腦設計的主記憶體架構，對不同記憶體區域有不同存取延遲

3. 電源管理與指令處理

- 動態頻率與電壓調整：根據應用程序的計算複雜度來動態調整處理器的電壓與頻率，使裝置可以延長使用時間
- 選擇性核心啟用：非關聯路徑可在低功耗核心執行



目錄

- IC設計工具技術趨勢與探索
- 基本設計步驟
- 狀態指定問題
- Mealy狀態模型
- 以CAD工具設計有限狀態機器
- 串列加法器範例
- 狀態最小化
- 作為仲裁器電路的FSM
- 同步順序電路的分析
- 演算法狀態機器圖
- 順序電路的正式模型
- 總結評論



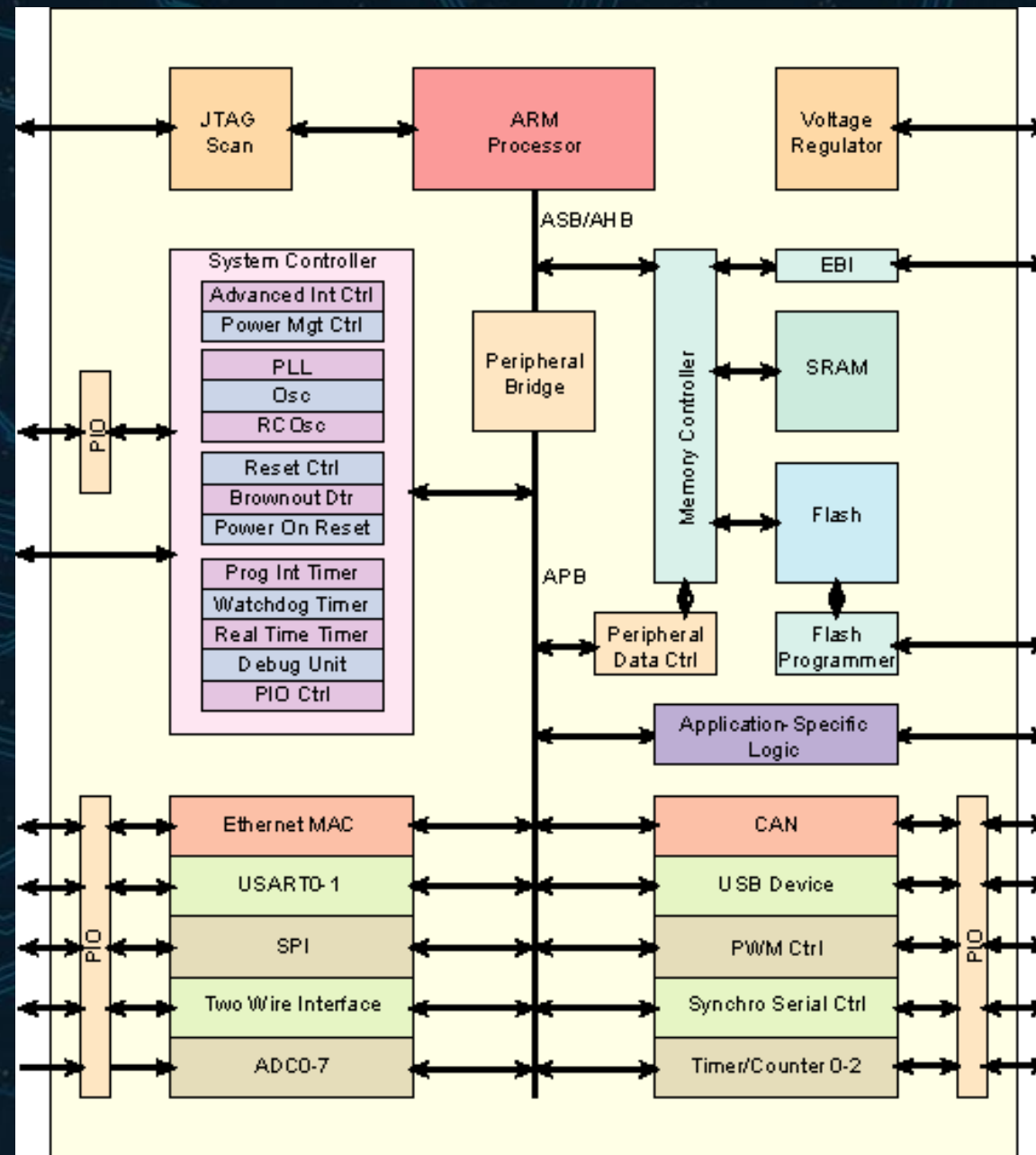
IC設計工具技術趨勢與探索

- 目前的IC設計產業正處在一場大革命的前夕，對更低成本、功耗的無止境追求和越來越短的產品上市壓力，迫使IC供應商提供採用0.13微米或以下的千萬閘級系統單晶片**SoC (system-on-a-chip)**，而這些SoC的高複雜性設計必須仰賴**EDA供應商提供全新的設計工具和方法**，以實現類比前後端、混合信號和數位電路的完全整合。



SoC基本架構介紹

- 典型的單晶片系統具有以下部分：
- 至少一個微控制器或微處理器、數位訊號處理器，但是也可以有多個中央控制核心
- 記憶體則可以是唯讀記憶體、隨機存取記憶體、EEPROM和快閃記憶體中的一種或多種
- 用於提供定時器訊號的振盪器和鎖相環電路
- 由計數器和計時器、電源電路組成的外部裝置
- 不同標準的連線介面，如通用序列匯流排、火線、乙太網路、通用非同步收發和序列周邊介面等
- 用於在數位訊號和類比訊號之間轉換的類比數位轉換器和數位類比轉換器
- 電壓調理電路以及穩壓器

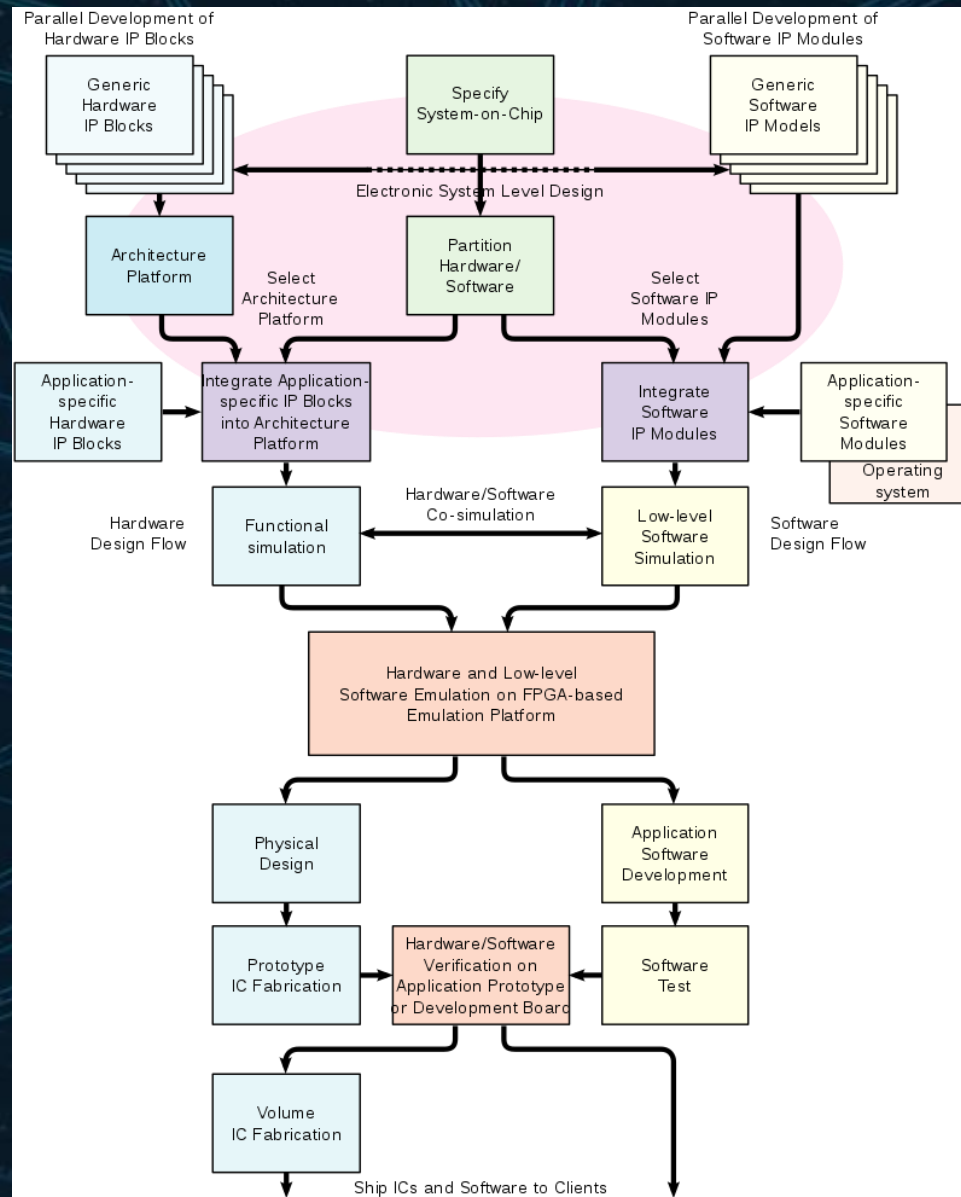


基於微控制器的單晶片系統(SOC)



SoC設計流程

- 一個完整單晶片系統由**硬體和軟體兩部分組成**，其中軟體用於控制硬體部分的**控制器、微處理器或數位訊號處理器核心以及外部裝置和介面**。
- 單晶片系統的設計流程主要是其硬體和軟體的設計。
- 由於單晶片系統的整合度已經達到數百萬門，工程師必須儘可能採取**可重用的設計思路**。
- 工程師通常會使用精心設計的**模擬器(Emulator)或者在通用的現場可程式化邏輯閘陣列(FPGA)上執行程式**，來測試之前進行的系統級、行為級（或用另一個術語暫存器傳輸級，即RTL）的設計代碼，這一步的目的是在設計專案在進行最後的硬體生產（投片）之前，其軟、硬體的功能、效能得到最後的確認，並改正所有功能、時序、功耗上的錯誤。

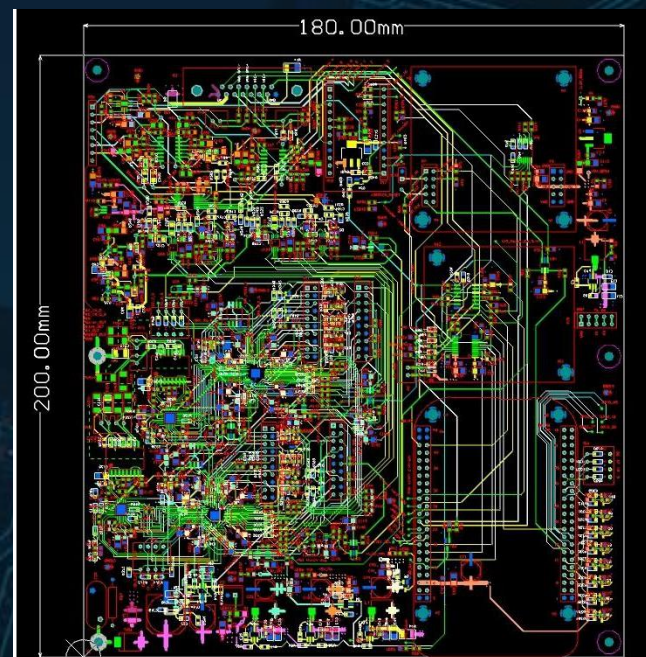


單晶片系統的設計流程示意圖



電子設計自動化工具 EDA

- 在電子設計自動化出現之前，設計人員必須手工完成積體電路的設計、布線等工作，這是因為當時所謂積體電路的複雜程度遠不及現在。





電子設計自動化工具 EDA

- 隨著積體電路規模的擴大、半導體技術的發展，EDA的重要性急劇增加。這些工具的使用者包括半導體元件製造中心的硬體技術人員，他們的工作是操作半導體元件製造設備並管理整個工作車間。電子設計自動化工具還被用來將設計的功能導入到類似現場可程式化邏輯閘陣列的半定製可程式邏輯裝置，或者生產全定製的特殊應用積體電路。

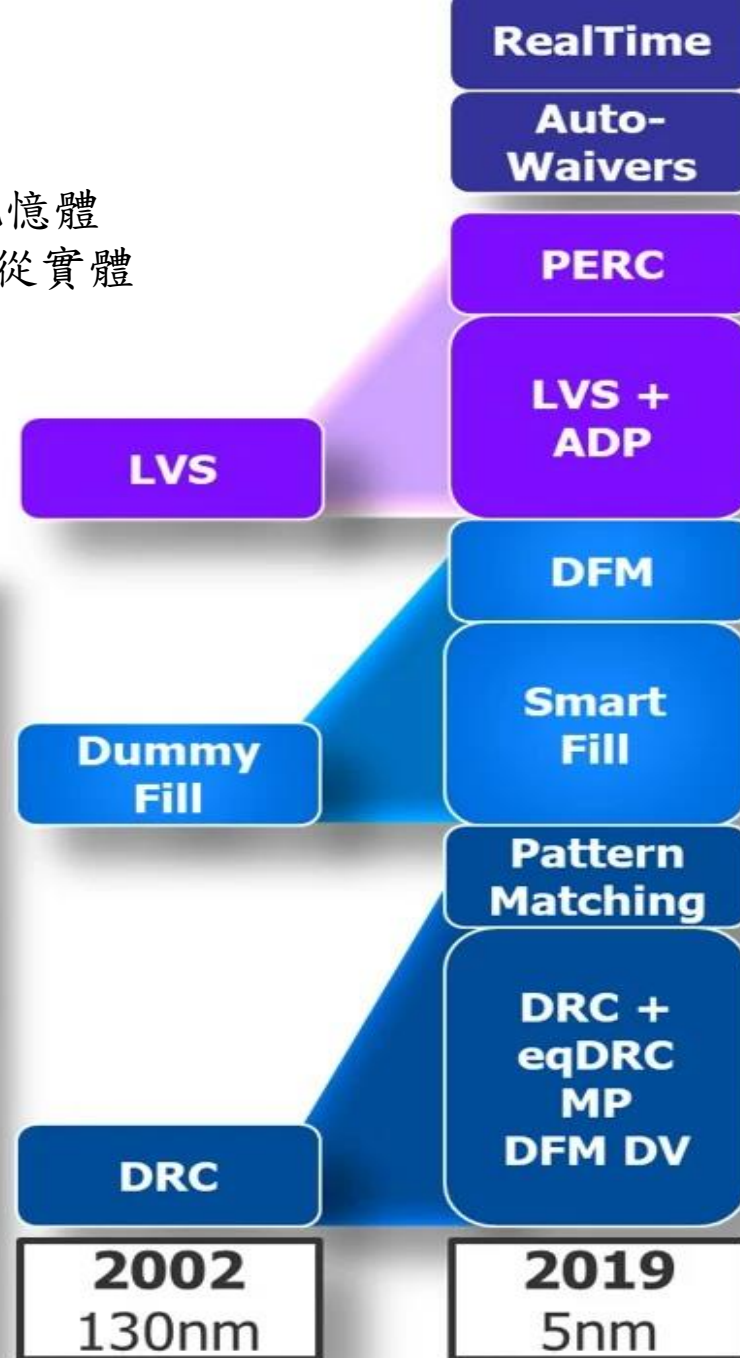
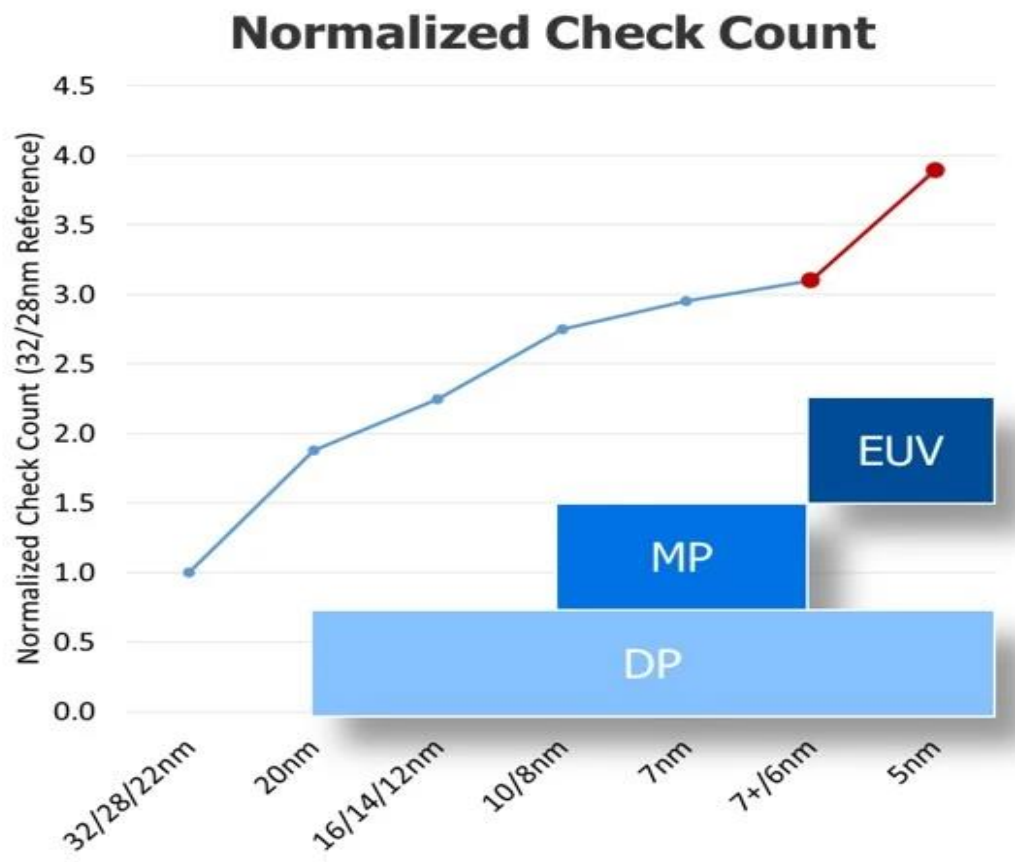


EDA現況

- 現今數位電路非常**模組化**，產線最前端將設計流程標準化，把設計流程區分為許多「**細胞**」（**cells**），而暫不考慮技術，接著細胞則以特定的積體電路技術實現邏輯或其他電子功能。製造商通常會提供**組件庫**（**libraries of components**），以及**符合標準類比工具的類比模型給生產流程**。類比EDA工具較不模組化，因為它需要更多的功能，零件間需要更多的互動，而零件一般說較不理想。



設計規則和規則的運算數量持續增加，執行時間和記憶體的需求也隨之增加；新增設計規則檢查(DRC)，導致從實體驗證到投片的過程中需要更多的DRC反覆驗證運算。
(資料來源：Siemens EDA)





EDA現況

- 如今的積體電路，從系統架構開始，落實到功能的定義和實現，最終實現整個晶片的版圖設計與驗證，是一項複雜的系統工程，結合了人類智慧的最高成果。以2022年最新的NVIDIA RTX 4090來說，採用台積電4奈米製程，其中結合有760億個電晶體，若沒有EDA輔助，設計這樣複雜的電路並保證良率是無法想像的。可見EDA這套工具，賦能了積體電路設計與製造的創新，當之無愧的站在了產業鏈的頂端。



EDA未來發展趨勢

- 目前，EDA在國際市場上已經發展成為了相對成熟的產業，而每年增長率只有10%左右。但有業內人士分析認為，這並不代表日後發展的機會在變小，在未來EDA的發展趨勢有：

- AI賦能
- EDA上雲
- 異質架構整合



介紹-同步順序電路(Synchronous Sequential Circuits)

- 本章將介紹一般電路的輸出和電路過去的行為有關，也和輸入的現在值有關，這稱為**順序電路 (sequential circuit)**。
- 大部分的情況都是用**時脈訊號**來控制順序電路的運作；這種電路稱為**同步順序電路 (synchronous sequential circuit)**，另一種沒有使用時脈訊號的稱為**非同步順序電路 (asynchronous sequential circuit)**。
- 同步電路比較容易設計，且用於大部分的實際應用。
- 可以由時脈正緣（0到1的轉換）或時脈負緣（1到0的轉換）所觸發。我們稱造成狀態改變的時脈邊緣為**有效時脈邊緣 (active clock edge)**。



- 圖1為邏輯電路產生的順序電路的輸出，為正反器現在狀態和主要輸入的函數。為了區別兩者，通常稱輸出只和電路狀態有關的順序電路為Moore類型，而輸出和狀態與主要輸入都有關的是Mealy類型。
- 順序電路也稱為有限狀態機器 (finite state machine, FSM)，這是常見於技術文獻的較正式名稱。這名稱是因為這些電路的功能性行為可以有限個數的狀態表示。我們在本章會常常用有限狀態機器 (finite state machine) 來稱呼順序電路。

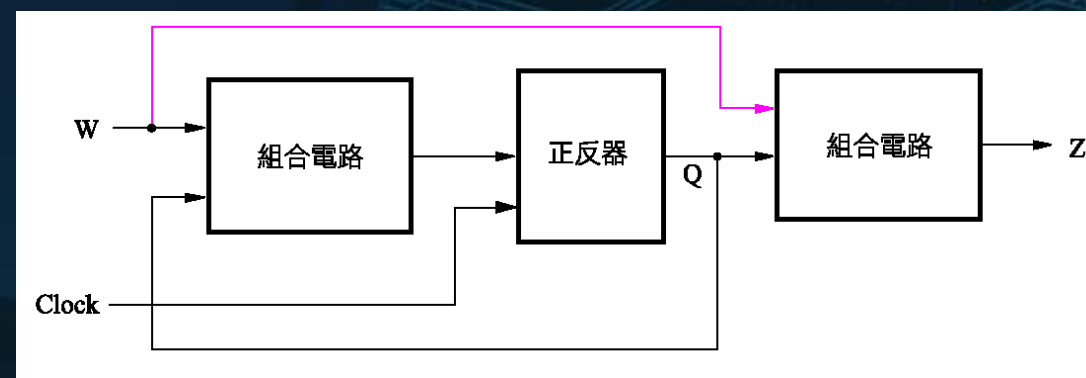
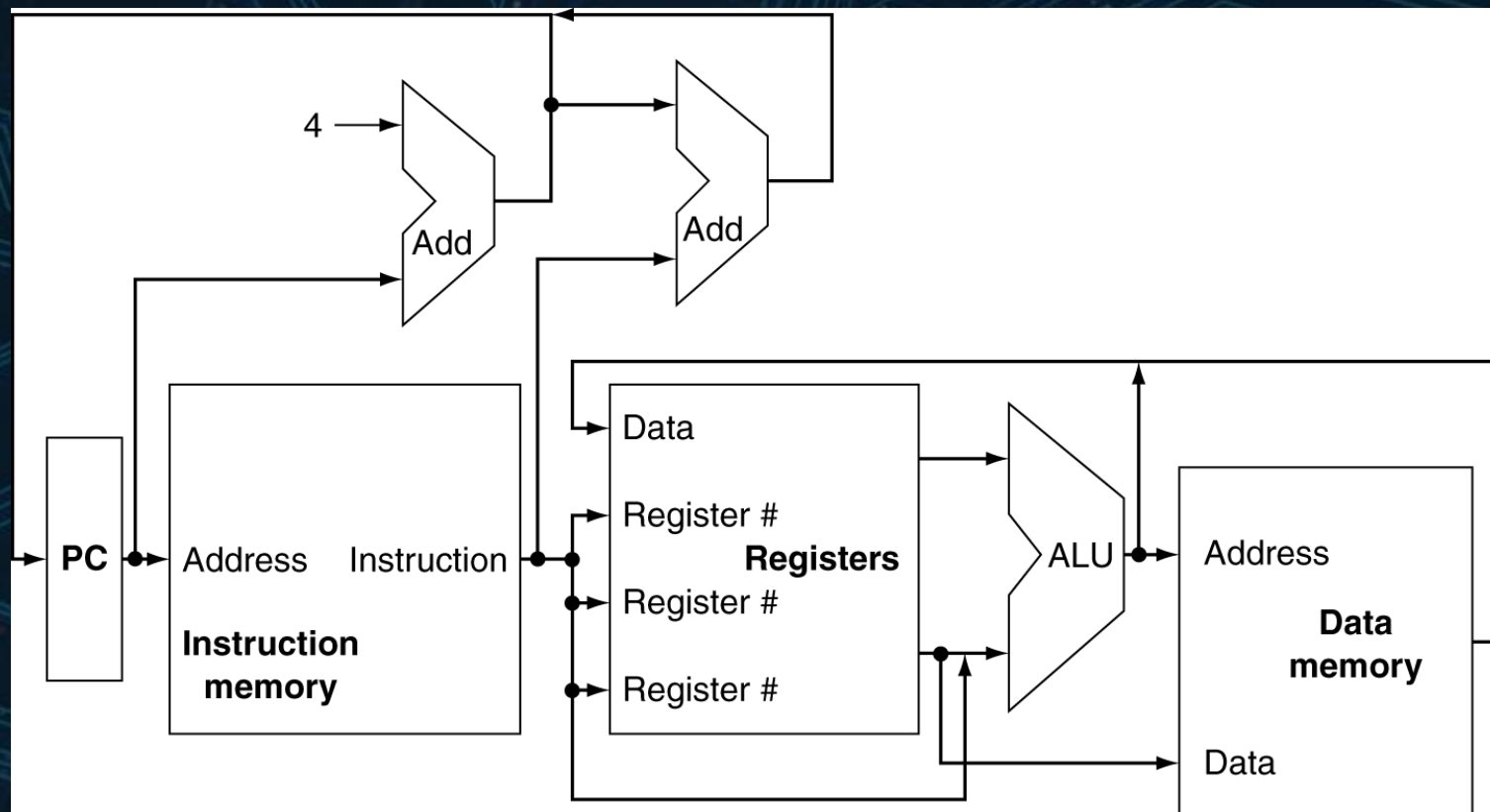


圖1 順序電路的一般架構



CPU Overview





一、基本設計步驟 Basic Design Steps

- 我們將以簡單的範例來說明設計順序電路的技巧。假設要設計符合下列規格的電路：
 - 電路有一個輸入 w 和一個輸出 z 。
 - 電路所有改變都發生在時脈訊號的正緣。
 - 若相鄰兩時脈循環間輸入的 w 值為1，則輸出 z 值為1，否則 z 值為0。
- 根據此規格，輸出 z 值不只和現在的 w 值有關。為了說明，考慮11個時脈循環其間 w 和 z 值的序列，如圖2所示。假設 w 為任意值， z 值則對應到我們的規格。



時脈循環:	t ₀	t ₁	t ₂	t ₃	t ₄	t ₅	t ₆	t ₇	t ₈	t ₉	t ₁₀
w:	0	1	0	1	1	0	1	1	1	0	1
z:	0	0	0	0	0	1	0	0	1	1	0

圖2 輸入和輸出訊號的序列



二、狀態圖 State Diagram

- 一個開始的好方法是選擇某個特定狀態作為**起始 (starting) 狀態**；此即電源第一次打開或reset訊號有效時電路應該進入的狀態。
- 觀念上最簡單的方式是以**狀態圖(state diagram)**作圖形化表示，以**節點（圓圈）**表示電路的狀態，以**方向弧形**表示狀態間的轉換。圖3的狀態圖定義的行為對應到我們的規格。

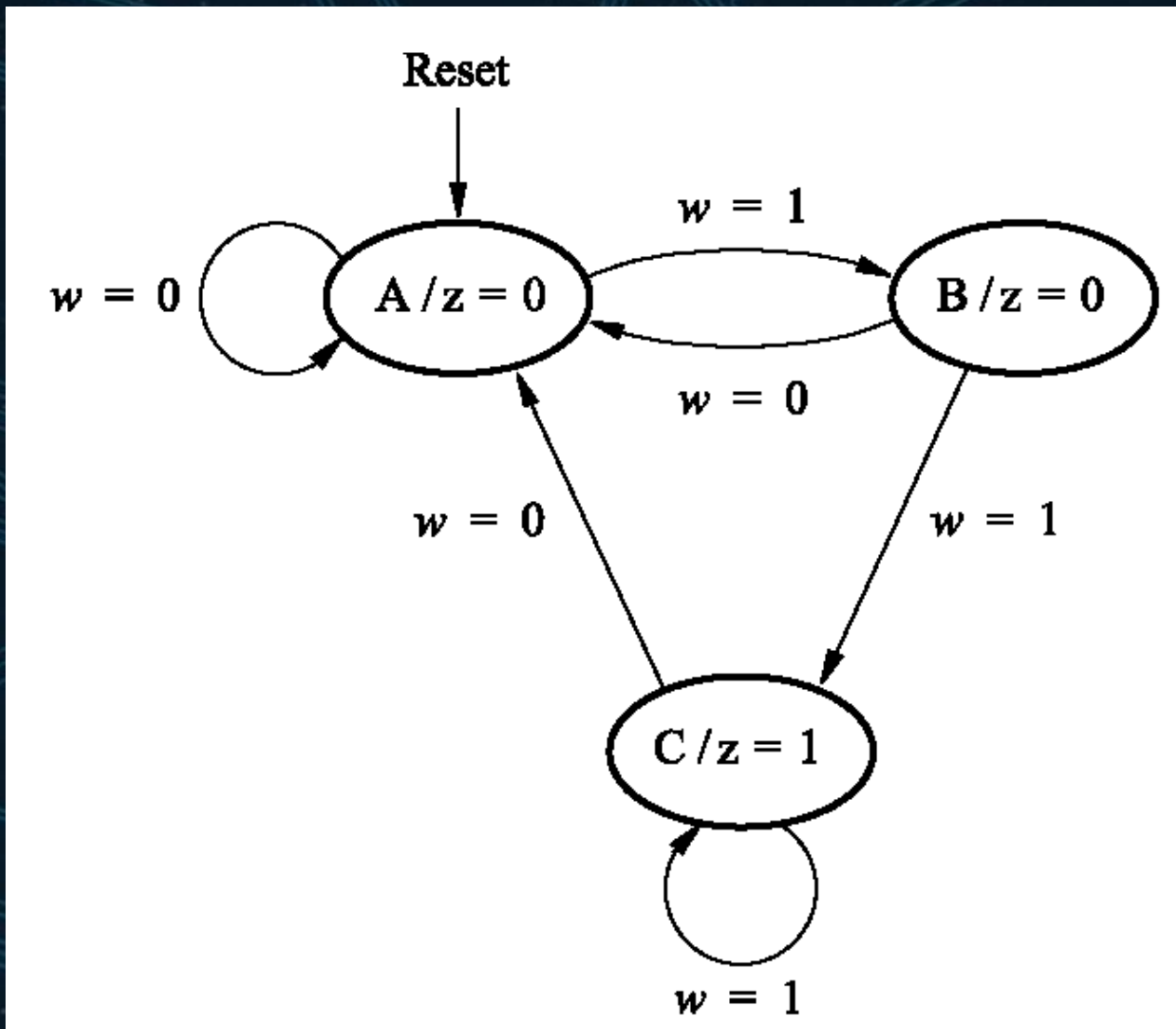


圖3 簡單順序電路的狀態圖。



三、狀態表 State Table

- 圖4為我們順序電路的狀態表 (state table)。此表顯示了每個現在狀態 (present state) 到下一個狀態 (next state) 的所有轉換，對不同的輸入訊號。



四、狀態指定 State Assignment

- 圖4的狀態表以字母A、B和C定義三個狀態。實作於邏輯電路時，每個狀態都以狀態變數 (state variable) 的某個特定估值（值的組合）表示。每個狀態變數以正反器形式實作。因為必須實作三個狀態，使用兩個狀態變數就已足夠。
- 每個有效時脈邊緣會使得正反器改變其狀態到那時的Y1和Y2值。因此，Y1和Y2稱為下個狀態的變數 (next-state variable)。
- 圖6的表格通常稱為狀態指定表 (state-assigned table)。



現在 狀態	下個狀態		輸出 z
	$w = 0$	$w = 1$	
A	A	B	0
B	A	C	0
C	A	C	1

圖4 圖3順序電路的狀態表。

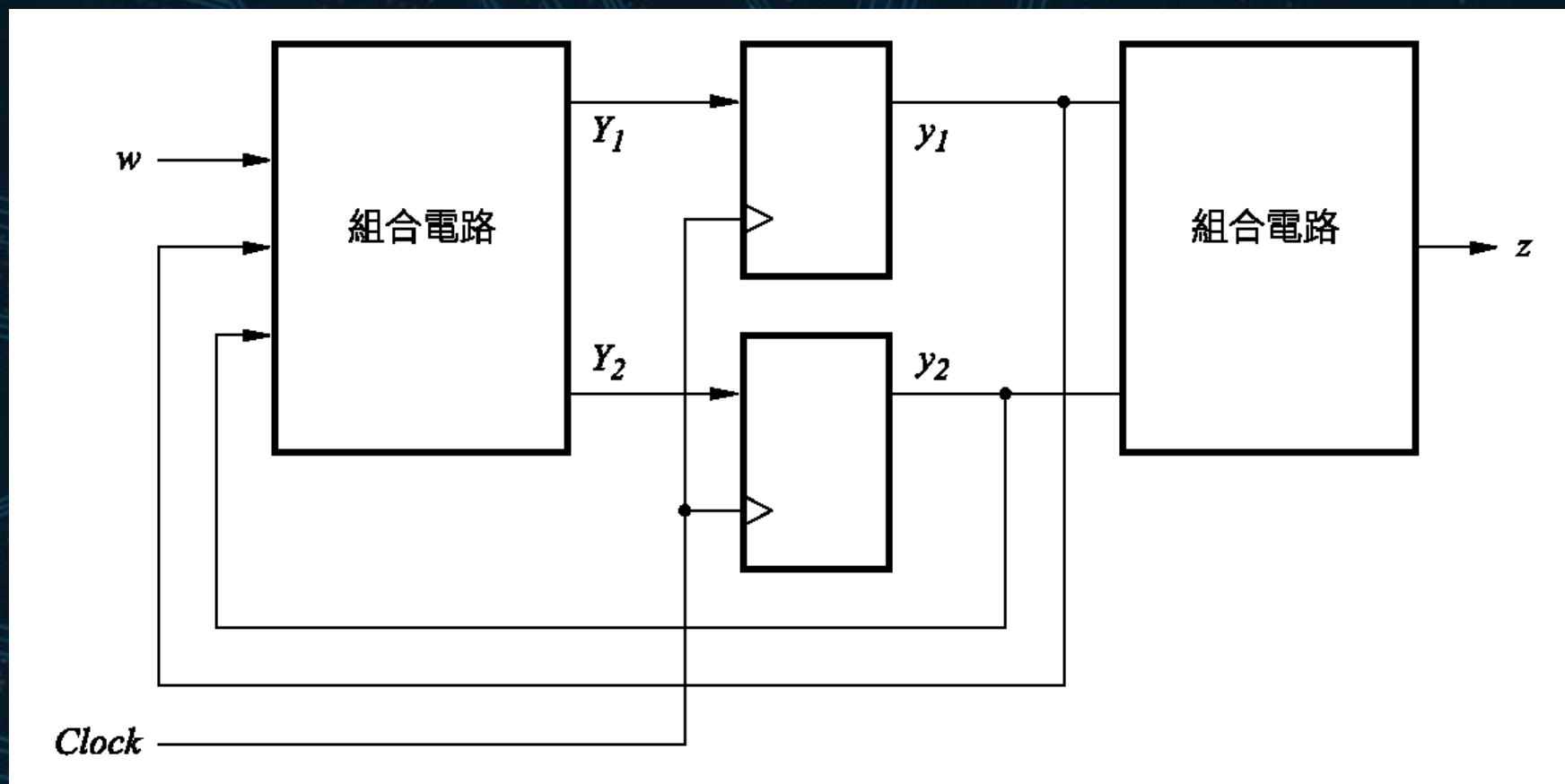


圖5 一般性順序電路，具有輸入 w 、輸出 z 和兩個狀態正反器。



	現在 狀態 y_2y_1	下個狀態		輸出 z
		$w = 0$	$w = 1$	
		Y_2Y_1	Y_2Y_1	
A	00	00	01	0
B	01	00	10	0
C	10	00	10	1
	11	dd	dd	d

圖6 圖4順序電路的狀態指定表。



五、正反器的選擇和下一個狀態與輸出表示式的推導

Choice Of Flip-flops And Derivation Of Next-state And Output Expressions

- 要求的邏輯表示式推導如圖7所示。
- 產生的不理會項於卡諾圖以d表示。以不理會項簡化表示式，得到

$$\begin{aligned}Y_1 &= w\bar{y}_1\bar{y}_2 \\ Y_2 &= w(y_1 + y_2) \\ z &= y_2\end{aligned}$$

		y_2y_1			
		00	01	11	10
w	0	0	0	d	0
	1	1	0	d	0

		y_2y_1			
		00	01	11	10
w	0	0	0	d	0
	1	0	1	d	1

y_2	y_1	
	0	1
0	0	0
1	1	d

忽略不理會項

$$Y_1 = w\bar{y}_1\bar{y}_2$$

$$Y_2 = wy_1\bar{y}_2 + w\bar{y}_1y_2$$

$$z = \bar{y}_1y_2$$

使用不理會項

$$Y_1 = w\bar{y}_1\bar{y}_2$$

$$\begin{aligned} Y_2 &= wy_1 + wy_2 \\ &= w(y_1 + y_2) \end{aligned}$$

$$z = y_2$$

圖7 圖6順序電路邏輯表示式的推導。

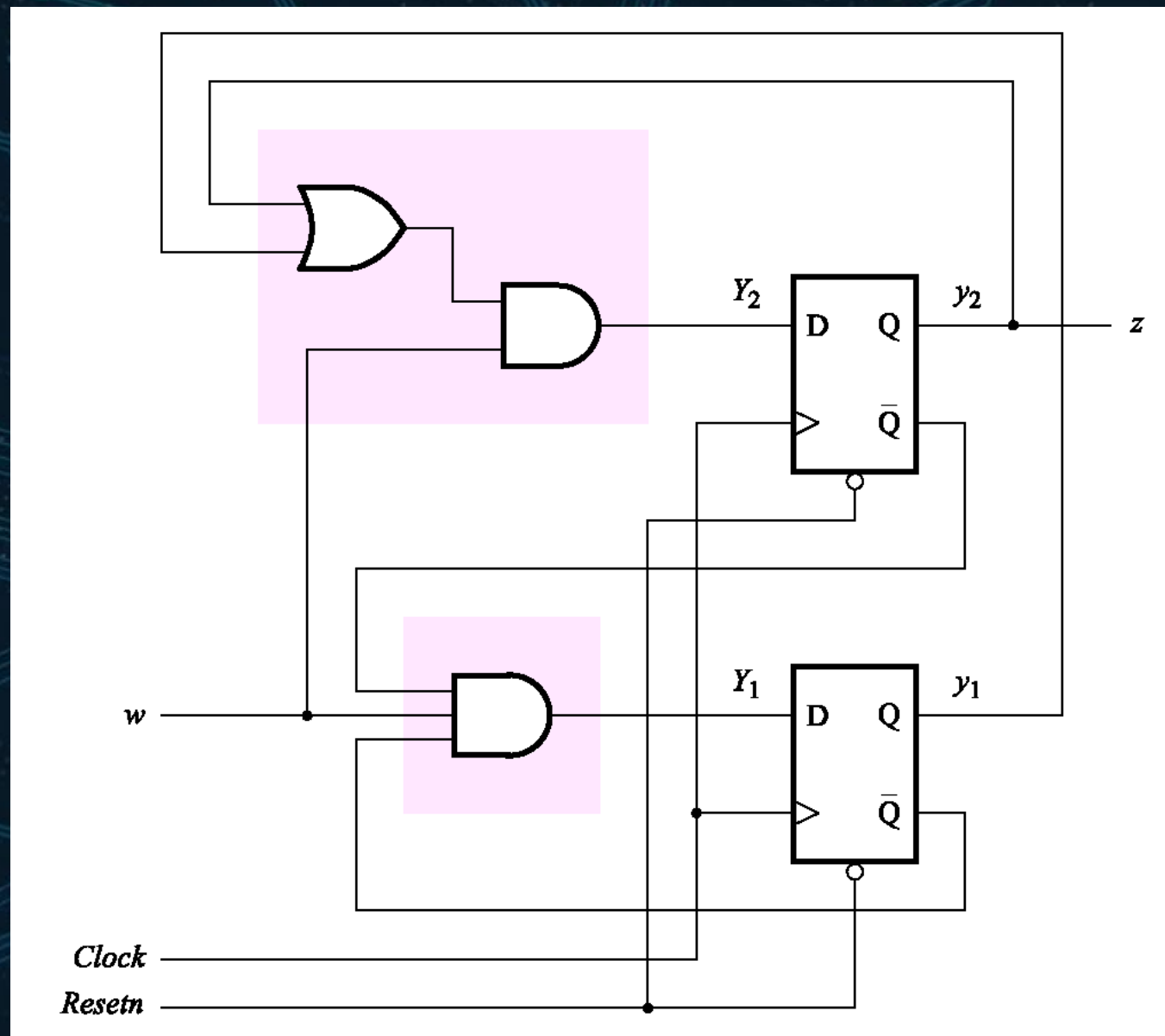


圖8 圖7順序電路的最終實作。



六、時序圖 Timeing Diagram

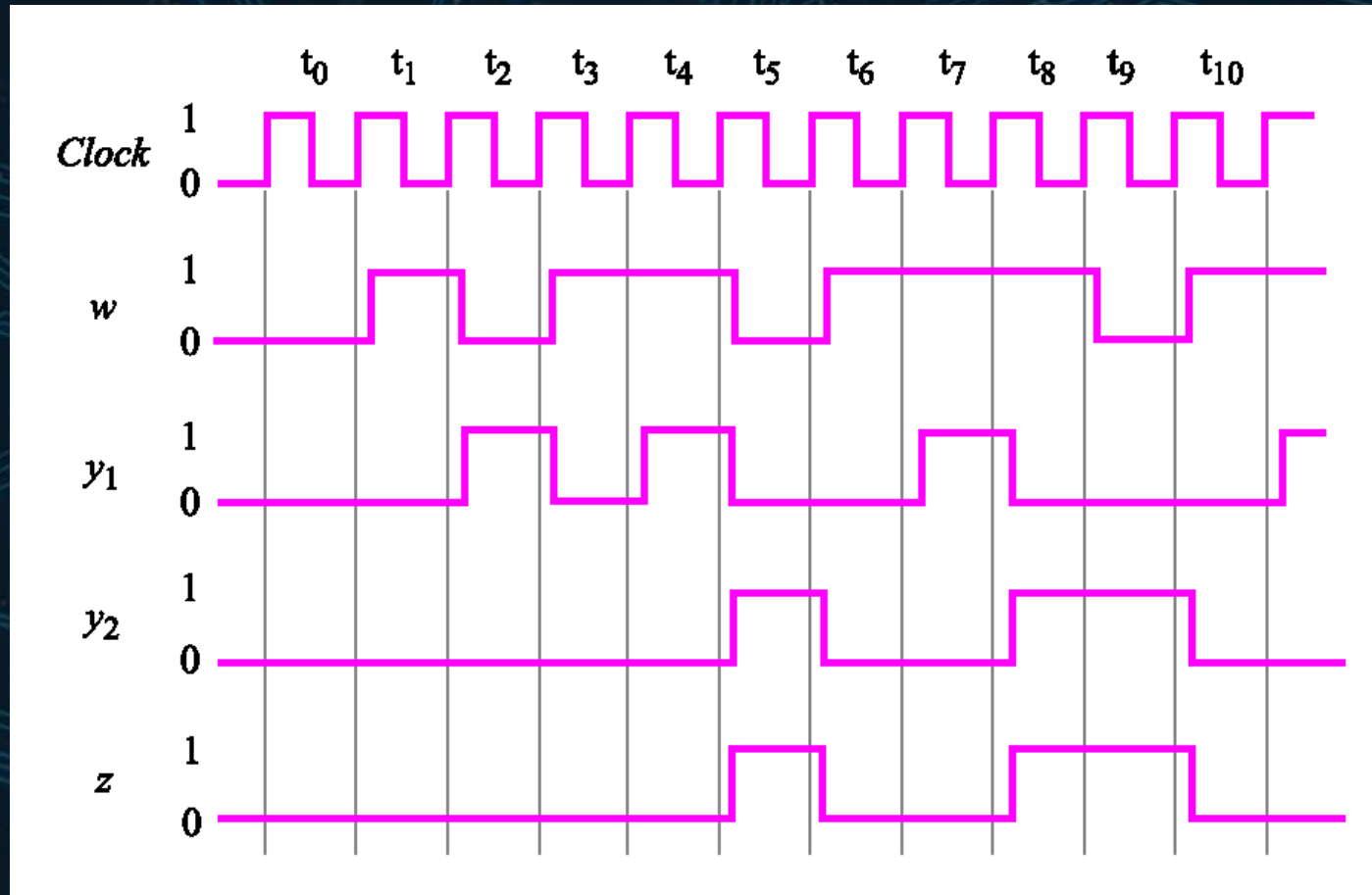


圖9 圖8電路的時序圖。



七、設計步驟總結 Summary Of Design Steps

- 我們將設計同步順序電路的步驟總結如下：
 - 得到要求電路的規格。
 - 推導機器的狀態，首先選擇起始狀態。
 - 給定電路規格，考慮電路輸入的所有估值，建立這些輸入所需的新狀態。
 - 為了追蹤經過的狀態，建立狀態圖。
 - 完成後，狀態圖會顯示機器中所有狀態，以及從一個狀態移動到另一個狀態的條件。
 - 根據狀態圖建立狀態表。或者直接從步驟2建立狀態表，不必先建立狀態圖。



八、設計步驟總結(續) Summary Of Design Steps

- 在我們的順序電路範例只有三個狀態，因此建立狀態表不包含多餘的狀態相當簡單。然而，實際上電路的狀態個數很多。這種情況首先推導狀態表不一定會產生最佳結果。幾乎可以確定會包含不必要的狀態這可以由最小化狀態個數的程序來修正。
- 決定用來表示所有狀態和執行狀態指定的狀態變數個數。對給定的順序電路，有許多不同的可能狀態指定。有些指定可能比其他的更好。



九、設計步驟總結(續) Summary Of Design Steps

- 選擇使用於電路的正反器類型。推導下個狀態邏輯表示式以控制所有正反器輸入，以及推導電路輸出的邏輯表示式。到目前為止我們使用**D正反器**。我們將在後面考慮其他類型正反器。
- 實作邏輯表示式的電路。



範例1

- 我們在之前章節介紹過匯流排的觀念，以及哪些連接可使得暫存器內容被轉移到另一個暫存器。
- 圖13的電路顯示如何以三態緩衝器將選定暫存器的內容放到匯流排，以及如何將匯流排上的資料載入另一個暫存器。
- 圖15控制機制，可以將R1和R2的內容互換，以位移暫存器來實現。現在將以FSM的方法設計要求的控制機制。
 - 暫存器R1和R2的內容可以互換，以暫存器R3作為暫時儲存位置，如下：
 - 首先將R2的內容載入R3，使用控制訊號 $R2out = 1$ 和 $R3in = 1$ 。
 - 然後將R1的內容轉移到R2，使用 $R1out = 1$ 和 $R2in = 1$ 。
 - 最後，將R3的內容（即先前R2的內容）轉移到R1，使用 $R3out = 1$ 和 $R1in = 1$ 。
- 因為這些步驟完成了要求的互換，設定訊號 $Done = 1$ 表示工作完成。假設執行互換是對應於輸入訊號w的脈波，期間為一個時脈循環。圖6.10為涉及控制電路的外部訊號。

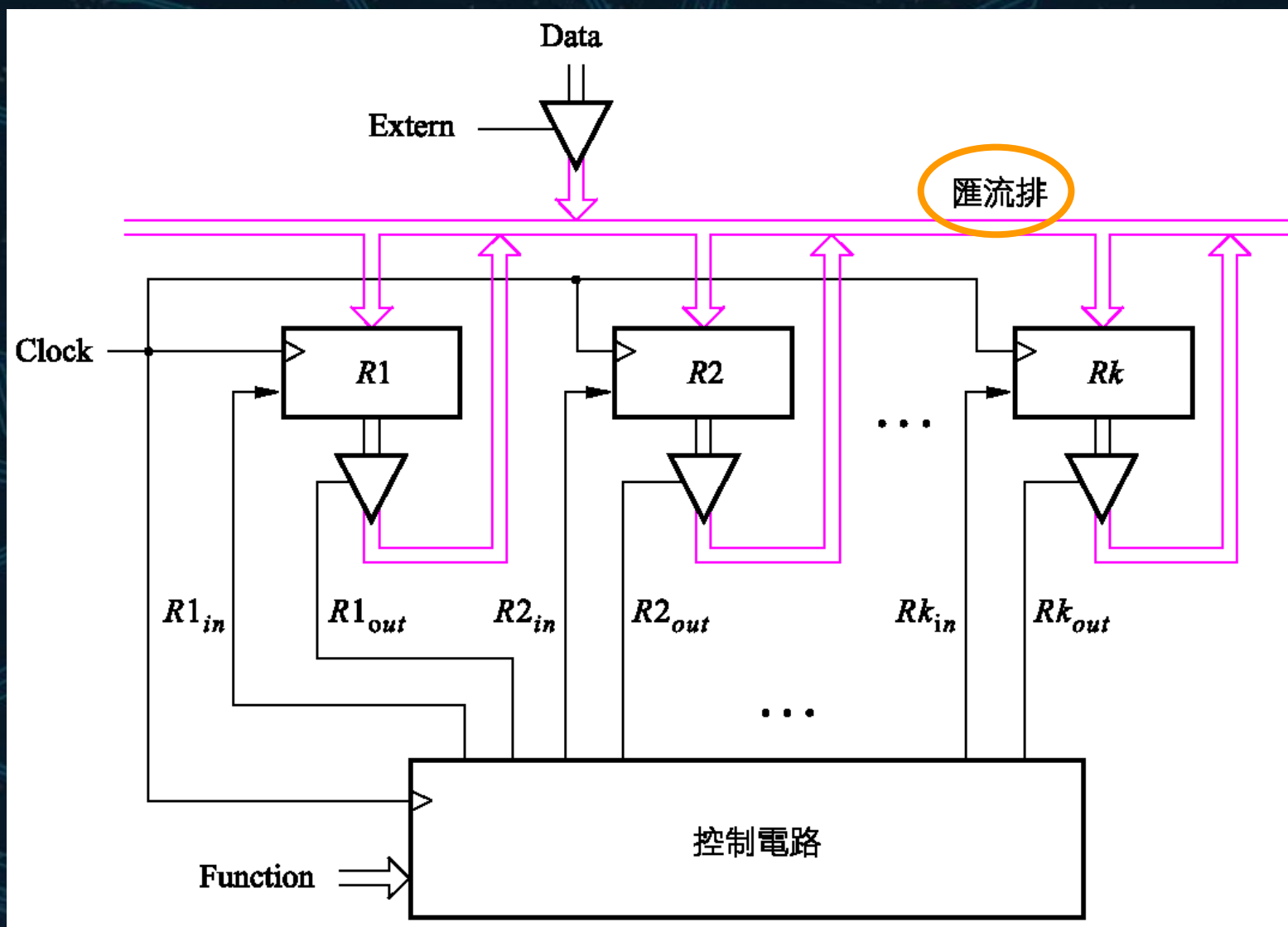


圖10 有 k 個暫存器的數位系統。

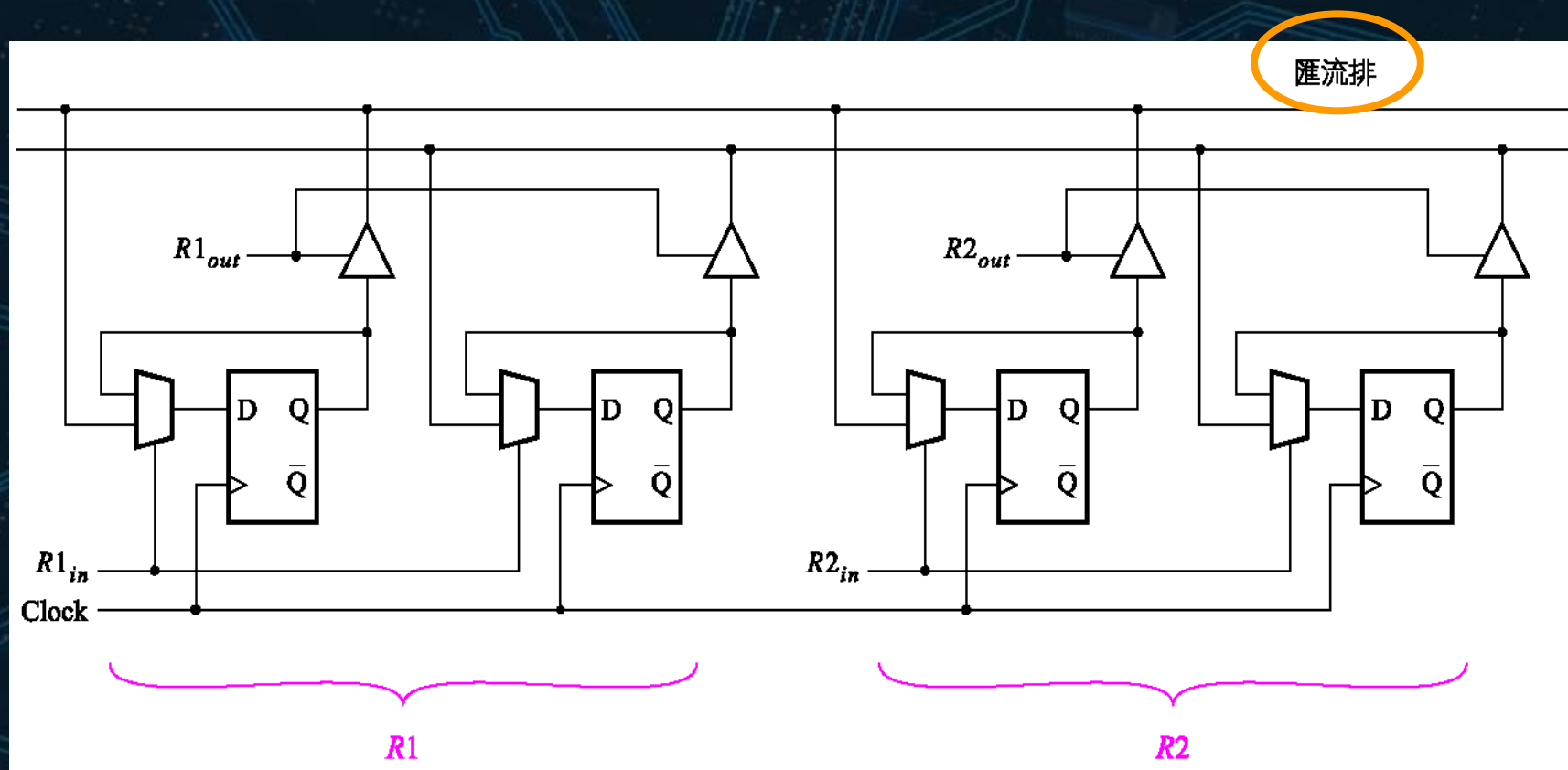


圖11 將暫存器連接到匯流排的細節。



以位移暫存器控制 Using A Shift Register For Control

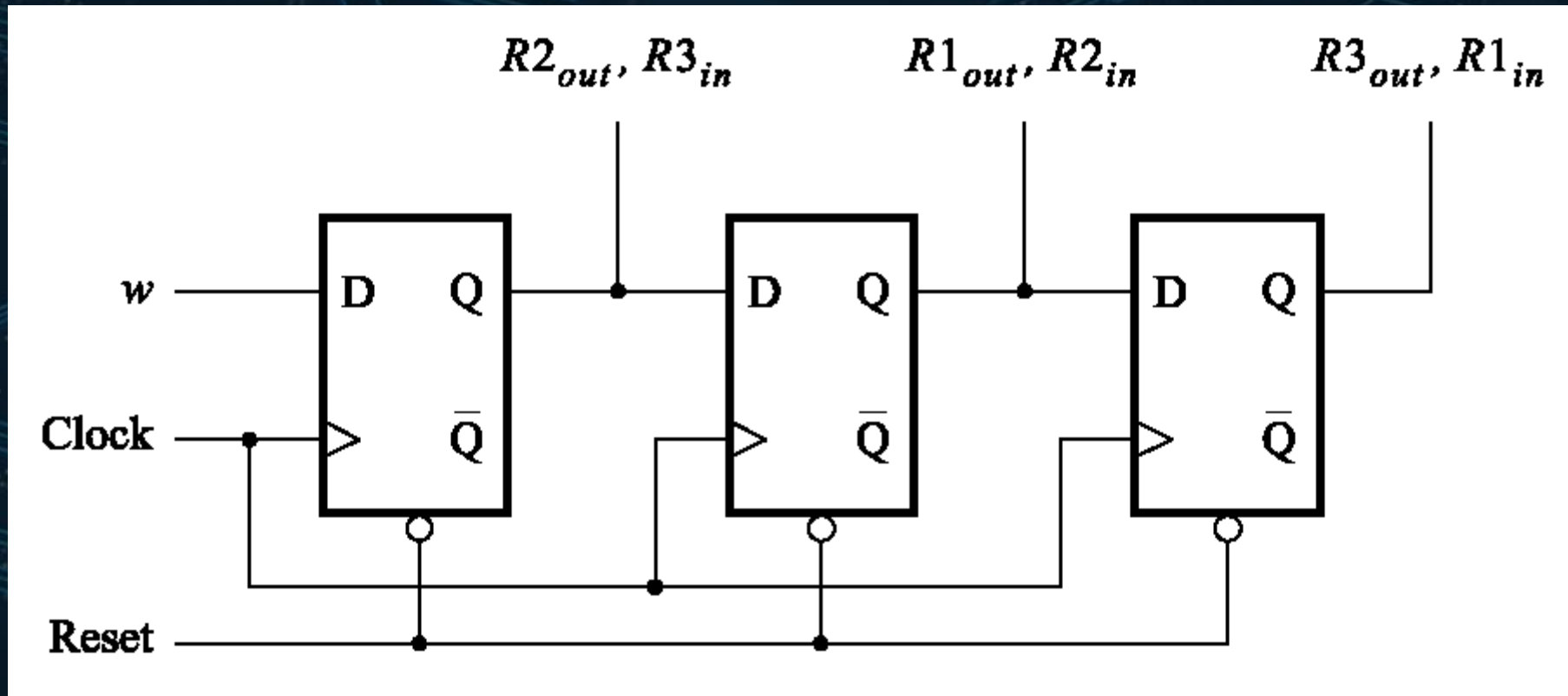


圖11 位移暫存器控制電路。

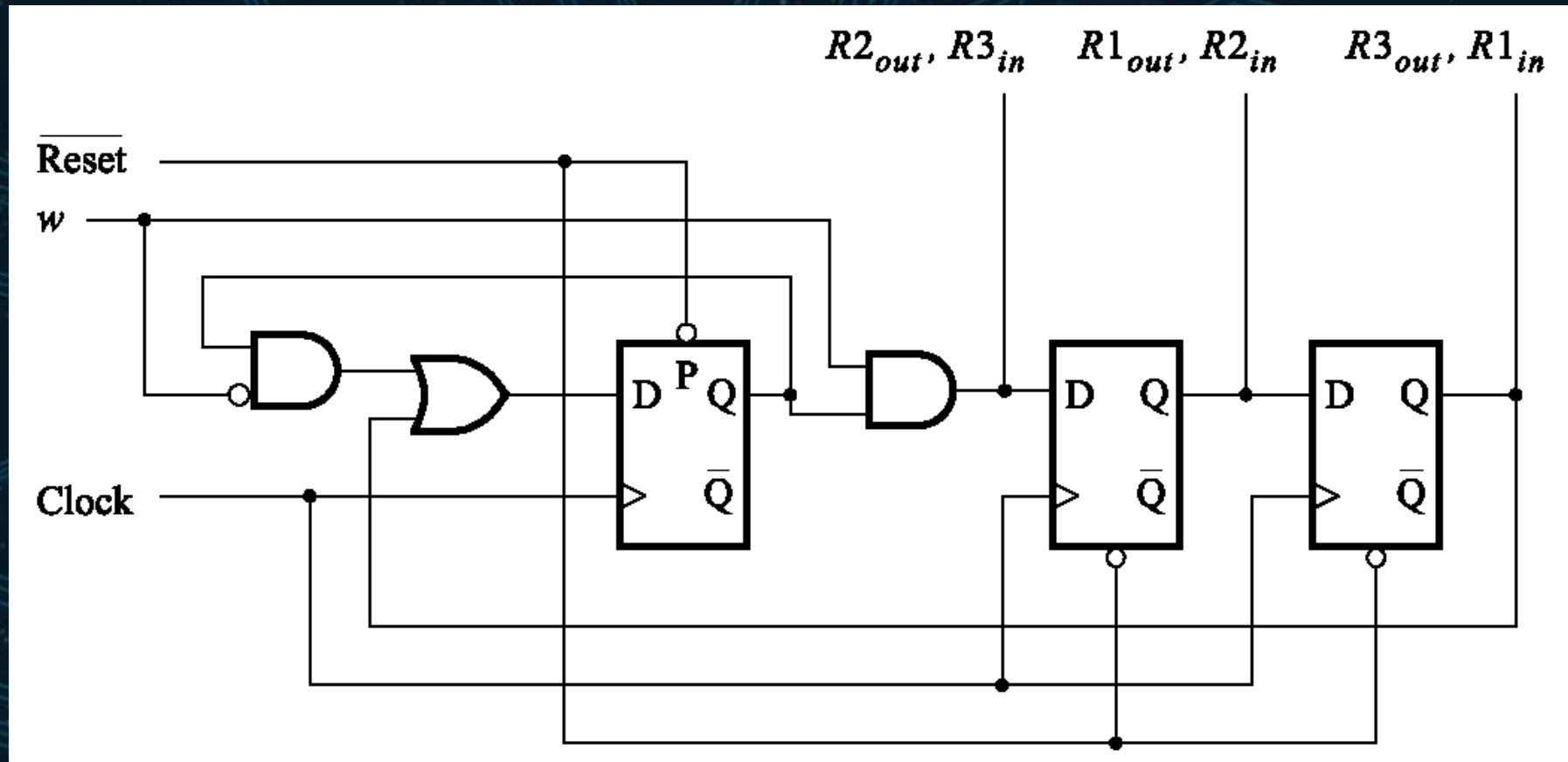


圖12 修改後的控制電路。

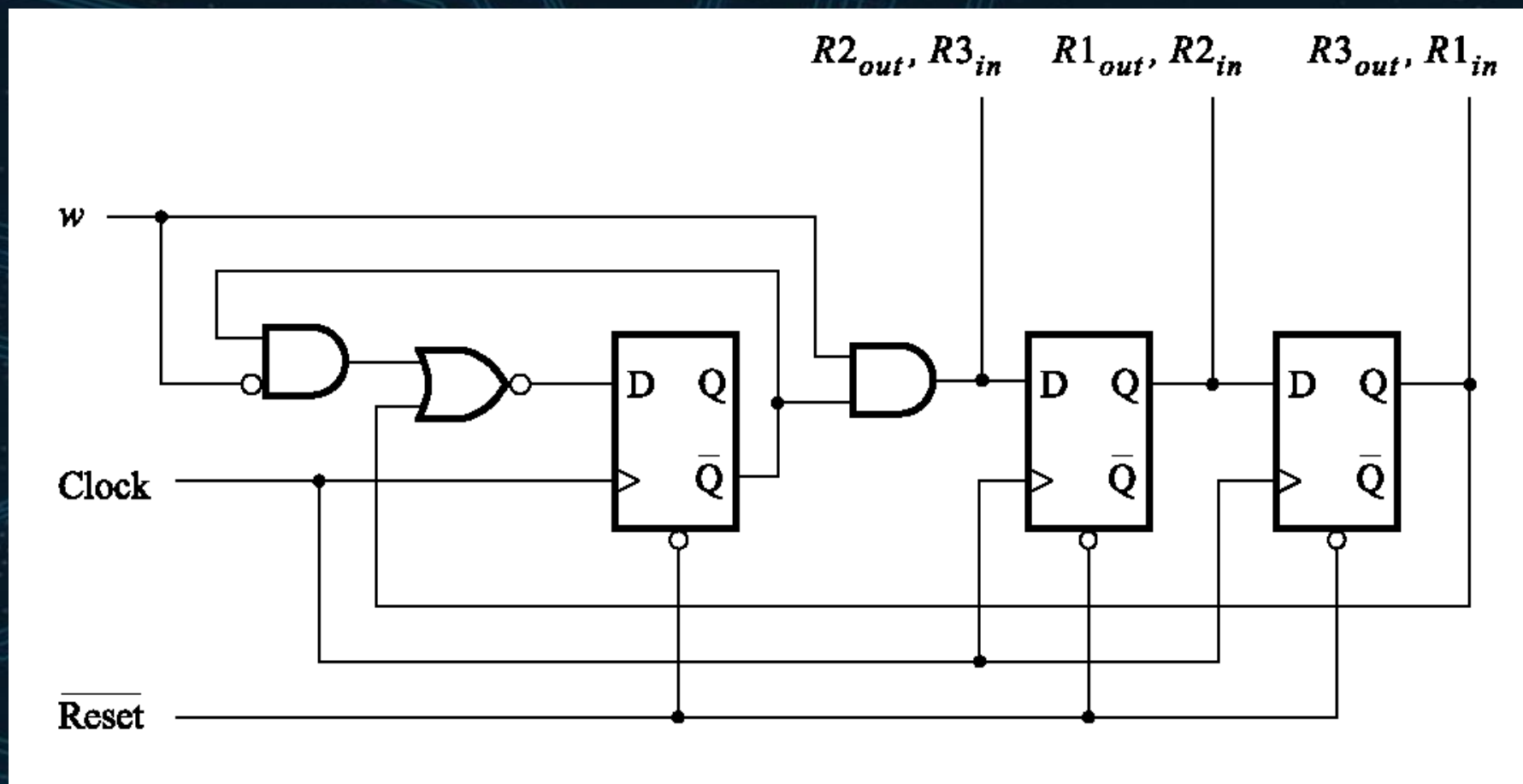


圖13 圖12電路的修改版本。

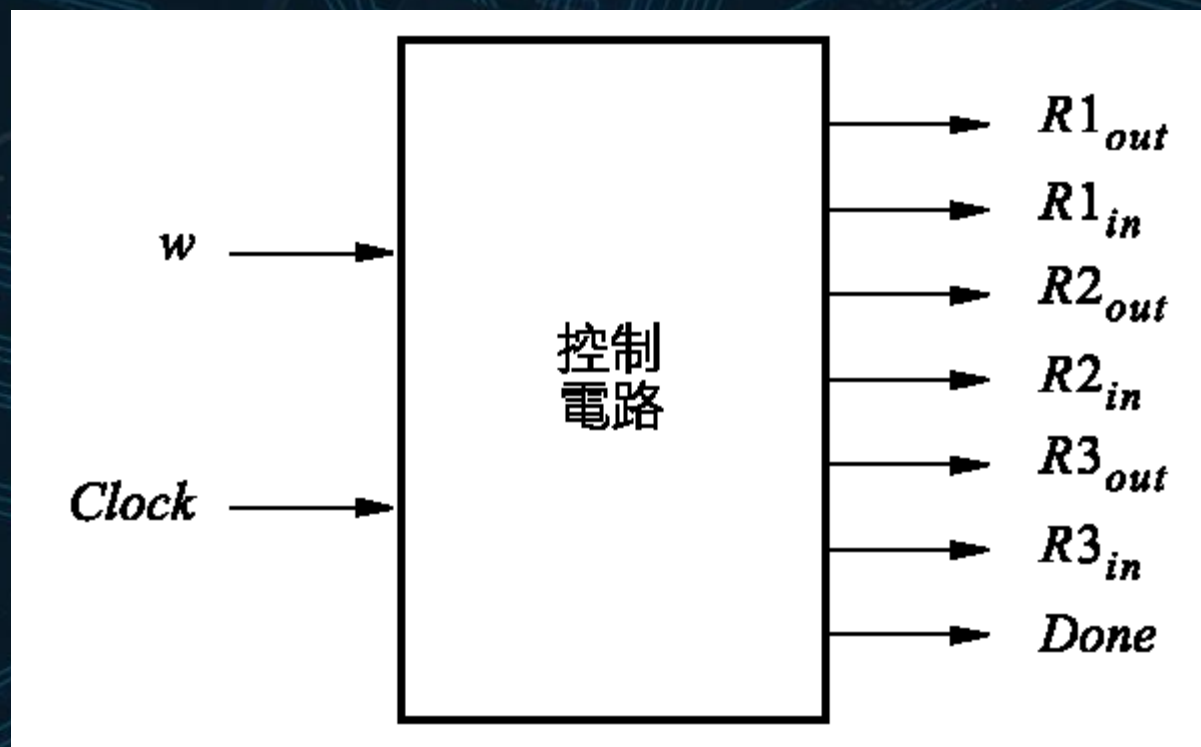


圖14 範例1所需的訊號。



VHDL程式碼

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all ;

ENTITY regn IS
    GENERIC ( N : INTEGER := 8 );
    PORT ( R      : IN      STD_LOGIC_VECTOR(N-1 DOWNT0 0);
          Rin, Clock : IN      STD_LOGIC ;
          Q      : OUT     STD_LOGIC_VECTOR(N-1 DOWNT0 0) );
END regn ;

ARCHITECTURE Behavior OF regn IS
BEGIN
    PROCESS
    BEGIN
        WAIT UNTIL Clock'EVENT AND Clock = '1' ;
        IF Rin = '1' THEN
            Q <= R ;
        END IF ;
    END PROCESS ;
END Behavior ;
```

圖15 圖11類型n位元暫存器的程式碼。



VHDL程式碼

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all ;

ENTITY trin IS
    GENERIC ( N : INTEGER := 8 );
    PORT (
        X      : IN      STD_LOGIC_VECTOR(N-1 DOWNT0 0);
        E      : IN      STD_LOGIC ;
        F      : OUT     STD_LOGIC_VECTOR(N-1 DOWNT0 0) );
END trin ;

ARCHITECTURE Behavior OF trin IS
BEGIN
    F <= (OTHERS => 'Z') WHEN E = '0' ELSE X ;
END Behavior ;
```

圖16 n 位元三態模組的程式碼。



- 圖17為產生要求序列的輸出控制訊號的順序電路狀態圖。只有當輸出訊號為1時才顯示。在其他情況下輸出訊號為0。
 - 在起始狀態A沒有顯示任何轉換，所有的輸出訊號皆為0。電路保持在此狀態直到互換的要求到達，也就是 w 變成1。
 - 在狀態B，將R2的內容轉移到R3所需的訊號被設定。下一個有效時脈邊緣將這些內容放到R3。
 - 這使得電路改變到狀態C，不管 w 是0或1。在此狀態將R1轉移到R2所需的訊號被設定。
 - 轉移發生在下一個有效時脈邊緣，電路變成狀態D，與 w 值無關。最後的轉移，從R3到R1，於離開狀態D的時脈邊緣執行，也使得電路回到狀態A。

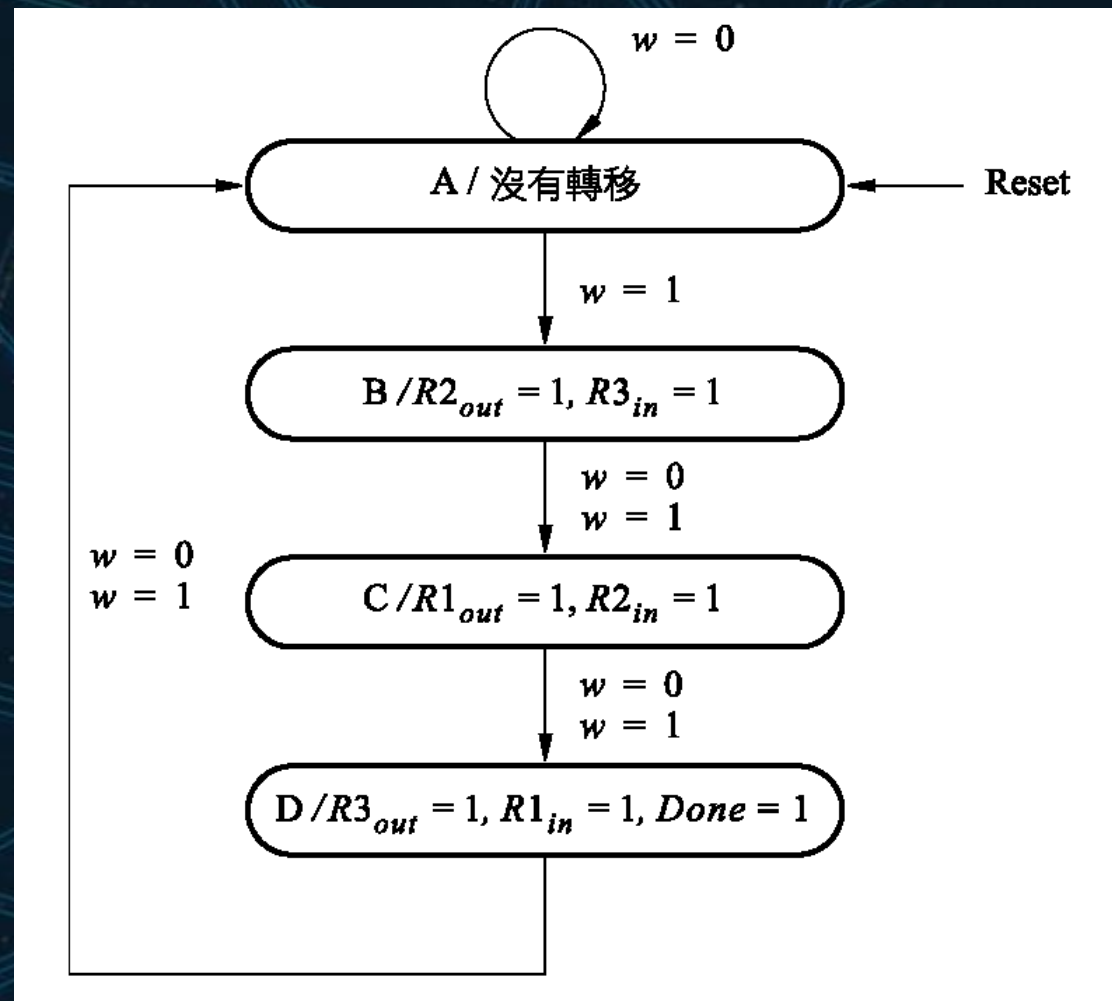


圖17 範例1的狀態圖。



圖18顯示相同的資訊於狀態表。因為有四個狀態，必須用兩個狀態變數 y_1 和 y_2 。直觀的狀態指定方法是將狀態A、B、C和D分別指定估值 $y_2y_1 = 00$ 、 01 、 10 和 11 ，產生圖19的狀態指定表。使用此指定和D正反器，得到下個狀態表示式如圖20所示。即為

$$Y_1 = w\bar{y}_1 + \bar{y}_1y_2$$
$$Y_2 = y_1\bar{y}_2 + \bar{y}_1y_2$$

現在 狀態	下個狀態		輸出						
	$w = 0$	$w = 1$	$R1_{out}$	$R1_{in}$	$R2_{out}$	$R2_{in}$	$R3_{out}$	$R3_{in}$	$Done$
A	A	B	0	0	0	0	0	0	0
B	C	C	0	0	1	0	0	1	0
C	D	D	1	0	0	1	0	0	0
D	A	A	0	1	0	0	1	0	1

圖18 範例1的狀態表。



現在 狀態	下個狀態		輸出							
	$w = 0$	$w = 1$								
	y_2y_1	Y_2Y_1	Y_2Y_1	$R1_{out}$	$R1_{in}$	$R2_{out}$	$R2_{in}$	$R3_{out}$	$R3_{in}$	$Done$
A	0 0	0 0	0 1	0	0	0	0	0	0	0
B	0 1	1 0	1 0	0	0	1	0	0	1	0
C	1 0	1 1	1 1	1	0	0	1	0	0	0
D	1 1	0 0	0 0	0	1	0	0	1	0	1

圖19 圖18順序電路的狀態指定表。



y_2y_1		00	01	11	10
w	0				1
	1	1			1

$$Y_1 = w\bar{y}_1 + \bar{y}_1y_2$$

y_2y_1		00	01	11	10
w	0		1		1
	1		1		1

$$Y_2 = y_1\bar{y}_2 + \bar{y}_1y_2$$

圖20 圖19順序電路的下個狀態表示式推導。



- 輸出控制訊號為

$$R2_{out} = R3_{in} = y_1 \bar{y}_2$$

$$R1_{out} = R2_{in} = \bar{y}_1 y_2$$

$$R1_{in} = R3_{out} = Done = y_1 y_2$$

- 這些表示式產生圖21的電路。此電路比位移暫存器更複雜，但只需要兩個正反器，而不是三個。

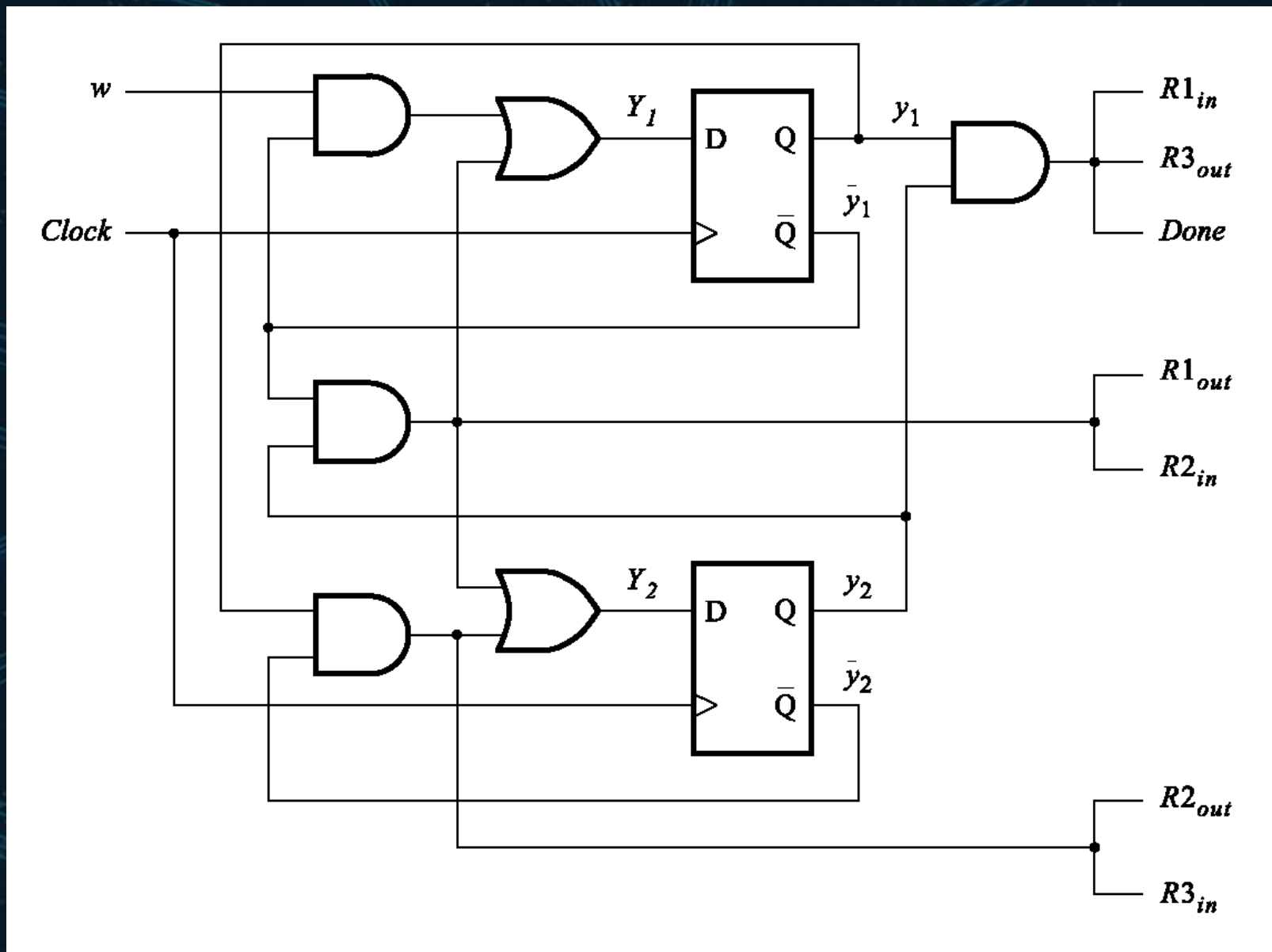


圖21 圖19順序電路的最終實作。



VHDL程式碼

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all ;

ENTITY regn IS
    GENERIC ( N : INTEGER := 8 );
    PORT (
        R          : IN          STD_LOGIC_VECTOR(N-1 DOWNT0 0) ;
        Rin, Clock : IN          STD_LOGIC ;
        Q          : OUT STD_LOGIC_VECTOR(N-1 DOWNT0 0) ) ;
END regn ;

ARCHITECTURE Behavior OF regn IS
BEGIN
    PROCESS
    BEGIN
        WAIT UNTIL Clock'EVENT AND Clock = '1' ;
        IF Rin = '1' THEN
            Q <= R ;
        END IF ;
    END PROCESS ;
END Behavior ;
```

圖62 圖57類型n位元暫存器的程式碼。



```
LIBRARY ieee ;  
USE ieee.std_logic_1164.all ;  
  
ENTITY trin IS  
    GENERIC ( N : INTEGER := 8 ) ;  
    PORT ( X   : IN      STD_LOGIC_VECTOR(N-1 DOWNT0 0) ;  
          E   : IN      STD_LOGIC ;  
          F   : OUT     STD_LOGIC_VECTOR(N-1 DOWNT0 0) ) ;  
END trin ;  
  
ARCHITECTURE Behavior OF trin IS  
BEGIN  
    F <= (OTHERS => 'Z') WHEN E = '0' ELSE X ;  
END Behavior ;
```

圖63 n 位元三態模組的程式碼。



```
LIBRARY ieee ;
USE ieee.std_logic_1164.all ;

ENTITY shiftr IS -- left-to-right shift register with async reset
    GENERIC ( K : INTEGER := 4 ) ;
    PORT (   Resetn, Clock, w      : IN          STD_LOGIC ;
            Q                      : BUFFER     STD_LOGIC_VECTOR(1 TO K) ) ;
END shiftr ;

ARCHITECTURE Behavior OF shiftr IS
BEGIN
    PROCESS ( Resetn, Clock )
    BEGIN
        IF Resetn = '0' THEN
            Q <= (OTHERS => '0') ;
        ELSIF Clock'EVENT AND Clock = '1' THEN
            Genbits: FOR i IN K DOWNTO 2 LOOP
                Q(i) <= Q(i-1) ;
            END LOOP ;
            Q(1) <= w ;
        END IF ;
    END PROCESS ;
END Behavior ;
```

圖64 圖58 位移暫存器的程式碼。



```
LIBRARY ieee ;
USE ieee.std_logic_1164.all ;

PACKAGE components IS
    COMPONENT regn -- register
        GENERIC ( N : INTEGER := 8 ) ;
        PORT (      R          : IN      STD_LOGIC_VECTOR(N-1 DOWNT0 0) ;
                  Rin, Clock   : IN      STD_LOGIC ;
                  Q            : OUT     STD_LOGIC_VECTOR(N-1 DOWNT0 0) ) ;
    END COMPONENT ;

    COMPONENT shiftr -- left-to-right shift register with async reset
        GENERIC ( K : INTEGER := 4 ) ;
        PORT      (  Resetn, Clock, w      : IN      STD_LOGIC ;
                    Q                      : BUFFER STD_LOGIC_VECTOR(1 TO K) ) ;
    END component ;

    COMPONENT trin -- tri-state buffers
        GENERIC ( N : INTEGER := 8 ) ;
        PORT (      X : IN  STD_LOGIC_VECTOR(N-1 DOWNT0 0) ;
                  E  : IN  STD_LOGIC ;
                  F   : OUT STD_LOGIC_VECTOR(N-1 DOWNT0 0) ) ;
    END COMPONENT ;
END components ;
```

圖 65 包裝和元件宣告



```
LIBRARY ieee ;
USE ieee.std_logic_1164.all ;
USE work.components.all ;

ENTITY swap IS
    PORT ( Data      : IN  STD_LOGIC_VECTOR(7 DOWNTO 0) ;
           Resetn, w  : IN  STD_LOGIC ;
           Clock, Extern : IN  STD_LOGIC ;
           RinExt      : IN  STD_LOGIC_VECTOR(1 TO 3) ;
           BusWires : INOUT STD_LOGIC_VECTOR(7 DOWNTO 0) ) ;
END swap ;

ARCHITECTURE Behavior OF swap IS
    SIGNAL Rin, Rout, Q : STD_LOGIC_VECTOR(1 TO 3) ;
    SIGNAL R1, R2, R3 : STD_LOGIC_VECTOR(7 DOWNTO 0) ;
BEGIN
    control: shiftr GENERIC MAP ( K => 3 )
        PORT MAP ( Resetn, Clock, w, Q ) ;
    Rin(1) <= RinExt(1) OR Q(3) ;
    Rin(2) <= RinExt(2) OR Q(2) ;
    Rin(3) <= RinExt(3) OR Q(1) ;
    Rout(1) <= Q(2) ; Rout(2) <= Q(1) ; Rout(3) <= Q(3) ;

    tri_ext: trin PORT MAP ( Data, Extern, BusWires ) ;
    reg1: regn PORT MAP ( BusWires, Rin(1), Clock, R1 ) ;
    reg2: regn PORT MAP ( BusWires, Rin(2), Clock, R2 ) ;
    reg3: regn PORT MAP ( BusWires, Rin(3), Clock, R3 ) ;
    tri1: trin PORT MAP ( R1, Rout(1), BusWires ) ;
    tri2: trin PORT MAP ( R2, Rout(2), BusWires ) ;
    tri3: trin PORT MAP ( R3, Rout(3), BusWires ) ;
END Behavior ;
```

圖 66
類似圖 56 的數位系統



以多工器實作匯流排

Using Multiplexers To Implement A Bus

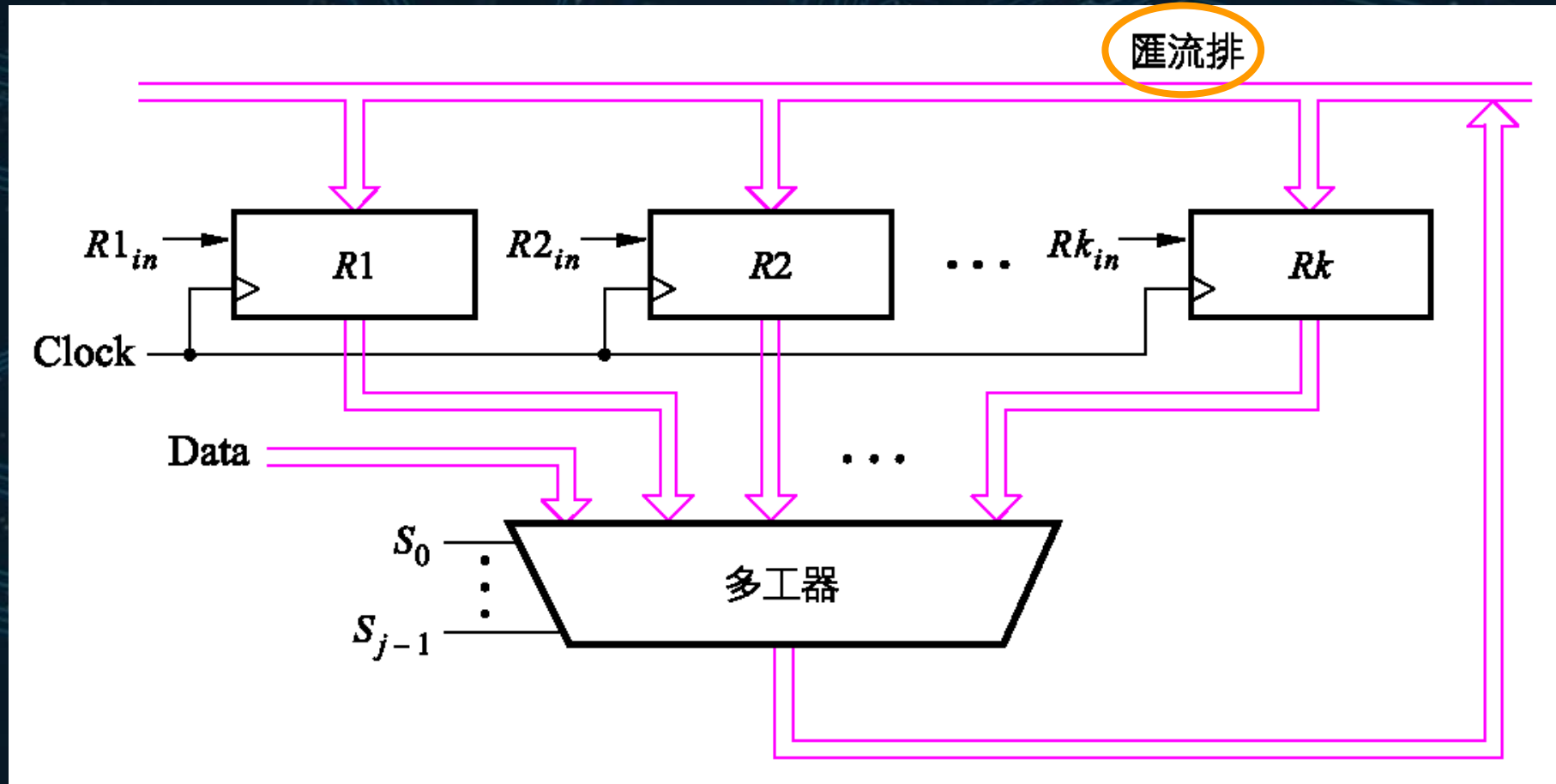


圖22 以多工器實作匯流排。



使用多工器的VHDL程式碼

VHDL Code Using Multiplexers

```
USE ieee.std_logic_1164.all ;
USE work.components.all ;

ENTITY swapmux IS
    PORT (    Data           : IN          STD_LOGIC_VECTOR(7 DOWNT0 0) ;
            Resetn, w       : IN          STD_LOGIC ;
            Clock           : IN          STD_LOGIC ;
            RinExt          : IN          STD_LOGIC_VECTOR(1 TO 3) ;
            BusWires        : BUFFER      STD_LOGIC_VECTOR(7 DOWNT0 0) ) ;
END swapmux ;

ARCHITECTURE Behavior OF swapmux IS
    SIGNAL Rin, Q : STD_LOGIC_VECTOR(1 TO 3) ;
    SIGNAL S : STD_LOGIC_VECTOR(1 DOWNT0 0) ;
    SIGNAL R1, R2, R3 : STD_LOGIC_VECTOR(7 DOWNT0 0) ;
BEGIN
    control: shiftr GENERIC MAP ( K => 3 )
        PORT MAP ( Resetn, Clock, w, Q ) ;
```

圖23 以多工器實作匯流排。



```
Rin(1) <= RinExt(1) OR Q(3) ;
Rin(2) <= RinExt(2) OR Q(2) ;
Rin(3) <= RinExt(3) OR Q(1) ;
reg1: regn PORT MAP ( BusWires, Rin(1), Clock, R1 ) ;
reg2: regn PORT MAP ( BusWires, Rin(2), Clock, R2 ) ;
reg3: regn PORT MAP ( BusWires, Rin(3), Clock, R3 ) ;

encoder:
WITH Q SELECT
    S <= "00" WHEN "000",
        "10" WHEN "100",
        "01" WHEN "010",
        "11" WHEN OTHERS ;
muxes: --eight 4-to-1 multiplexers
WITH S SELECT
    BusWires <= Data WHEN "00",
        R1 WHEN "01",
        R2 WHEN "10",
        R3 WHEN OTHERS ;

END Behavior ;
```

圖24 以多工器實作匯流排。



ARCHITECTURE Behavior OF swapmux IS

SIGNAL Rin, Q : STD_LOGIC_VECTOR(1 TO 3) ;

SIGNAL R1, R2, R3 : STD_LOGIC_VECTOR(7 DOWNT0 0) ;

BEGIN

control: **shiftr** GENERIC MAP (K => 3)

PORT MAP (Resetn, Clock, w, Q) ;

Rin(1) <= RinExt(1) OR Q(3) ;

Rin(2) <= RinExt(2) OR Q(2) ;

Rin(3) <= RinExt(3) OR Q(1) ;

reg1: **regn PORT MAP** (BusWires, Rin(1), Clock, R1) ;

reg2: regn PORT MAP (BusWires, Rin(2), Clock, R2) ;

reg3: regn PORT MAP (BusWires, Rin(3), Clock, R3) ;

muxes:

WITH Q SELECT

BusWires <=	Data	WHEN "000",
	R2	WHEN "100",
	R1	WHEN "010",
	R3	WHEN OTHERS ;

END Behavior ;

圖 25 圖 24 中 architecture 的簡化版本

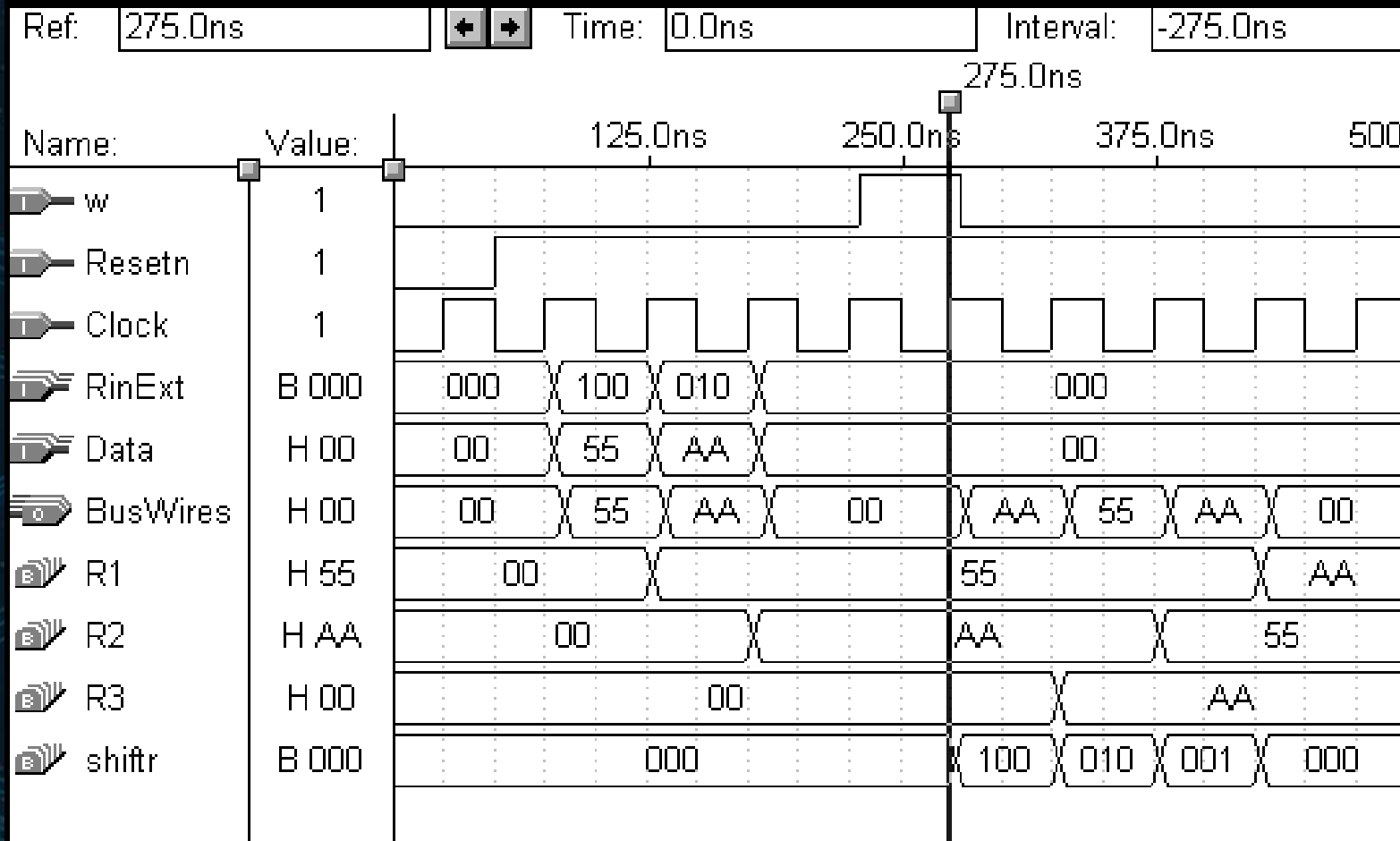


圖 26 圖23中的VHDL程式碼的時序模擬



十、狀態指定問題 State-assignment Problem

- 圖45為一種可能性。此例中狀態A、B和C分別以估值 $y_2y_1 = 00$ 、 01 和 11 表示。剩下來的估值 $y_2y_1 = 10$ 是不需要的，將其視為不理會條件。若我們再次選擇以D正反器實作電路，下個狀態和輸出表示式為

$$\begin{aligned}Y_1 &= D_1 = w \\Y_2 &= D_2 = wy_1 \\z &= y_2\end{aligned}$$

- 這些式子定義圖46的電路。



時脈循環:	t ₀	t ₁	t ₂	t ₃	t ₄	t ₅	t ₆	t ₇	t ₈	t ₉	t ₁₀
w:	0	1	0	1	1	0	1	1	1	0	1
z:	0	0	0	0	0	1	0	0	1	1	0

圖44 輸入和輸出訊號的序列



	現在 狀態 y_2y_1	下個狀態		輸出 z
		$w = 0$	$w = 1$	
		Y_2Y_1	Y_2Y_1	
A	00	00	01	0
B	01	00	11	0
C	11	00	11	1
	10	dd	dd	d

圖45 順序電路的改進狀態指定。

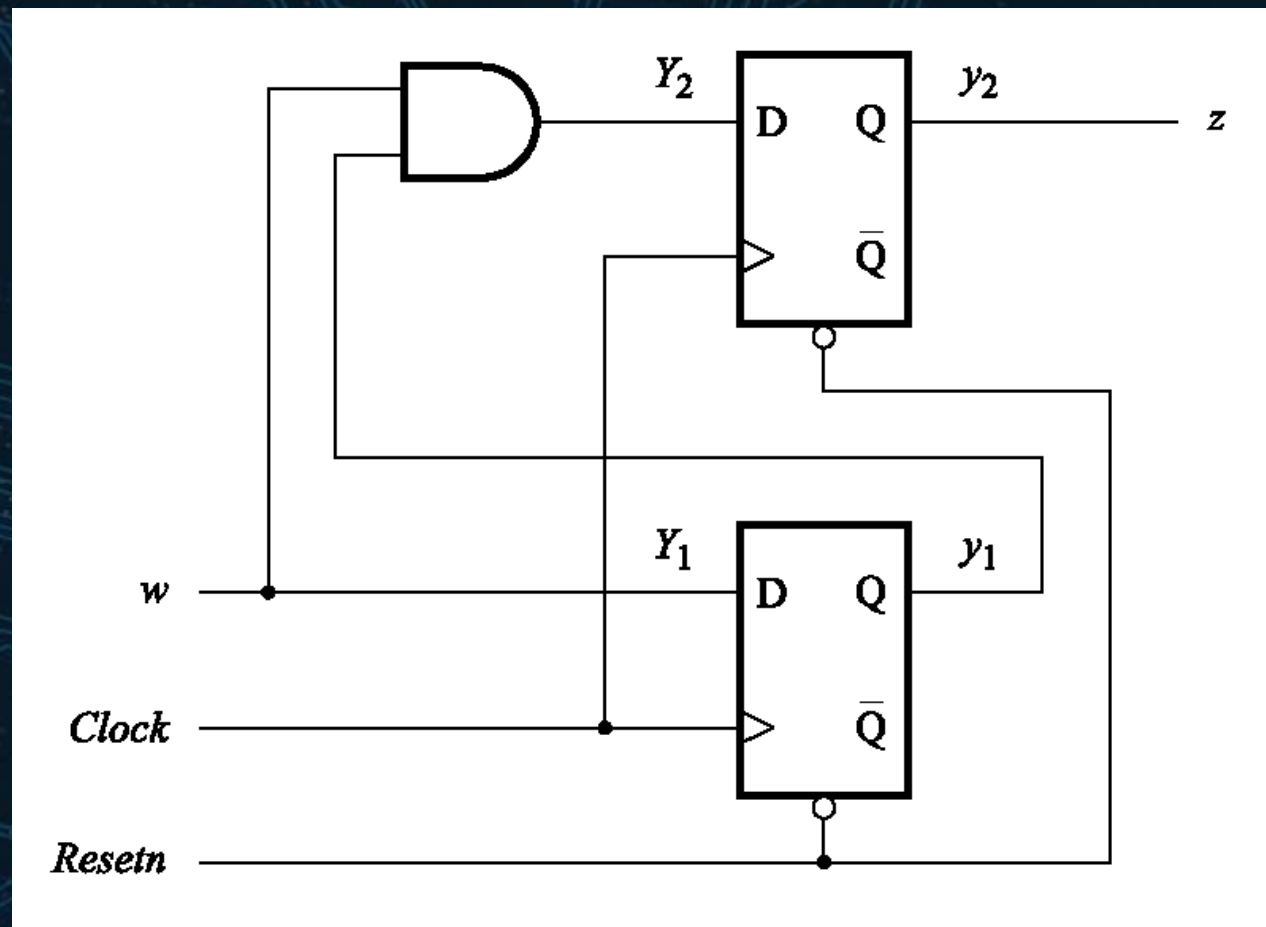


圖46 圖45改進後狀態指定的最終電路。



範例2

- 我們在圖19對圖18的順序電路使用直觀的狀態指定。考慮交換指定狀態C和D的估值的影響，如圖24所示。因此下個狀態表示式為
$$\begin{aligned} Y_1 &= w\bar{y}_2 + y_1\bar{y}_2 \\ Y_2 &= y_1 \end{aligned}$$

- 如圖25所得到的。輸出表示式為
$$\begin{aligned} R1_{out} &= R2_{in} = y_1y_2 \\ R1_{in} &= R3_{out} = Done = \bar{y}_1y_2 \\ R1_{out} &= R3_{in} = y_1\bar{y}_2 \end{aligned}$$

- 這些式子產生比圖21略微簡單的電路。

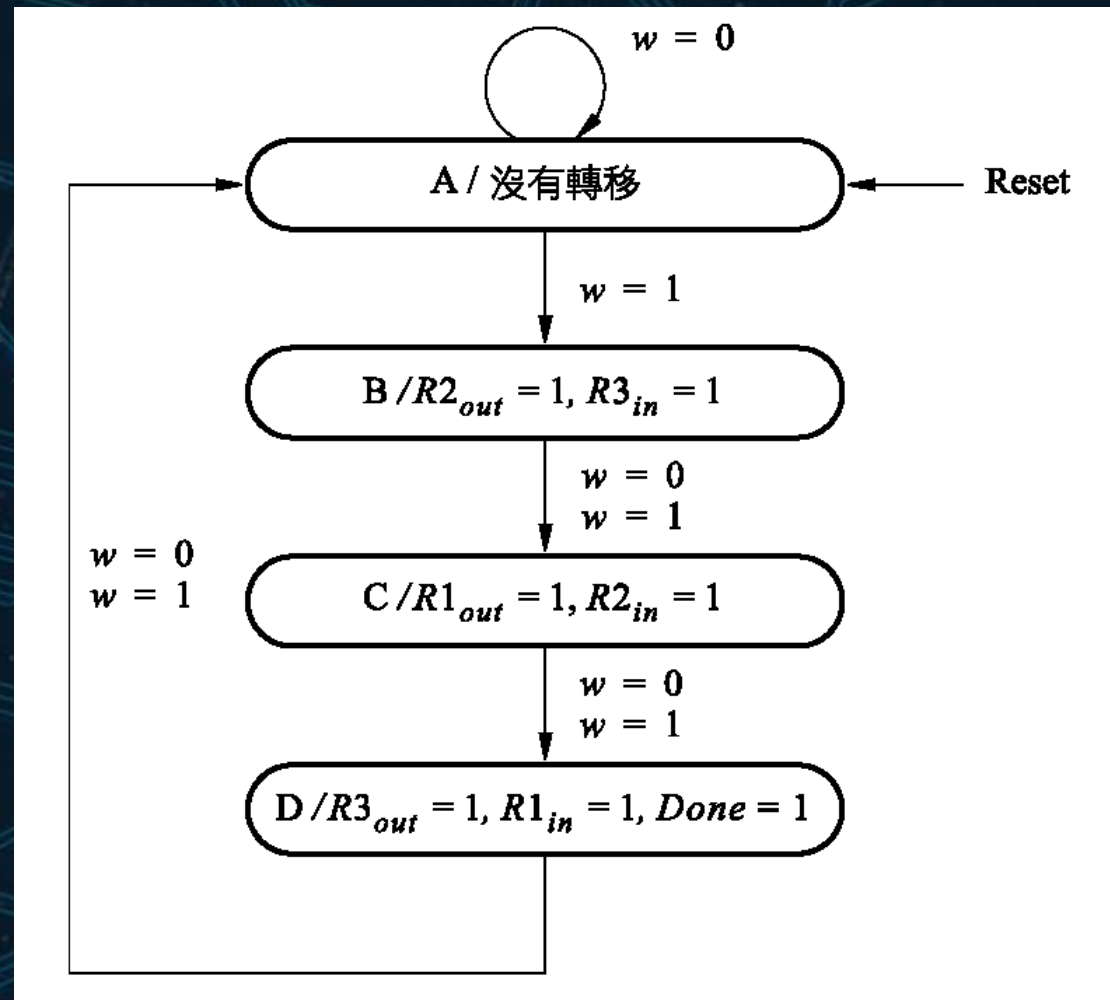


圖47 範例1的狀態圖。



	現在 狀態	下個狀態		輸出						
		$w = 0$	$w = 1$							
		y_2y_1	Y_2Y_1	$R1_{out}$	$R1_{in}$	$R2_{out}$	$R2_{in}$	$R3_{out}$	$R3_{in}$	$Done$
A	0 0	0 0	0 1	0	0	0	0	0	0	0
B	0 1	1 1	1 1	0	0	1	0	0	1	0
C	1 1	1 0	1 0	1	0	0	1	0	0	0
D	1 0	0 0	0 0	0	1	0	0	1	0	1

圖48 圖18順序電路的改進狀態指定。



y_2y_1		00	01	11	10
w	0		1		
	1	1	1		

$$Y_1 = w\bar{y}_2 + y_1\bar{y}_2$$

y_2y_1		00	01	11	10
w	0		1	1	
	1		1	1	

$$Y_2 = y_1$$

圖49 圖48順序電路下個狀態表示式的推導。



十一、單一熱門編碼 One-hot Encoding

- 值為1的變數稱為『熱門』。此方法稱為單一熱門編碼 (one-hot encoding)。
- 圖50顯示如何將單一熱門狀態指定應用在圖4的順序電路。
- 狀態變數剩下的五個估值沒有用到。在推導下個狀態和輸出表示式時可以視為不理會項。使用此指定，結果表示式為

$$\begin{aligned}Y_1 &= \bar{w} \\Y_2 &= wy_1 \\Y_3 &= w\bar{y}_1 \\z &= y_3\end{aligned}$$

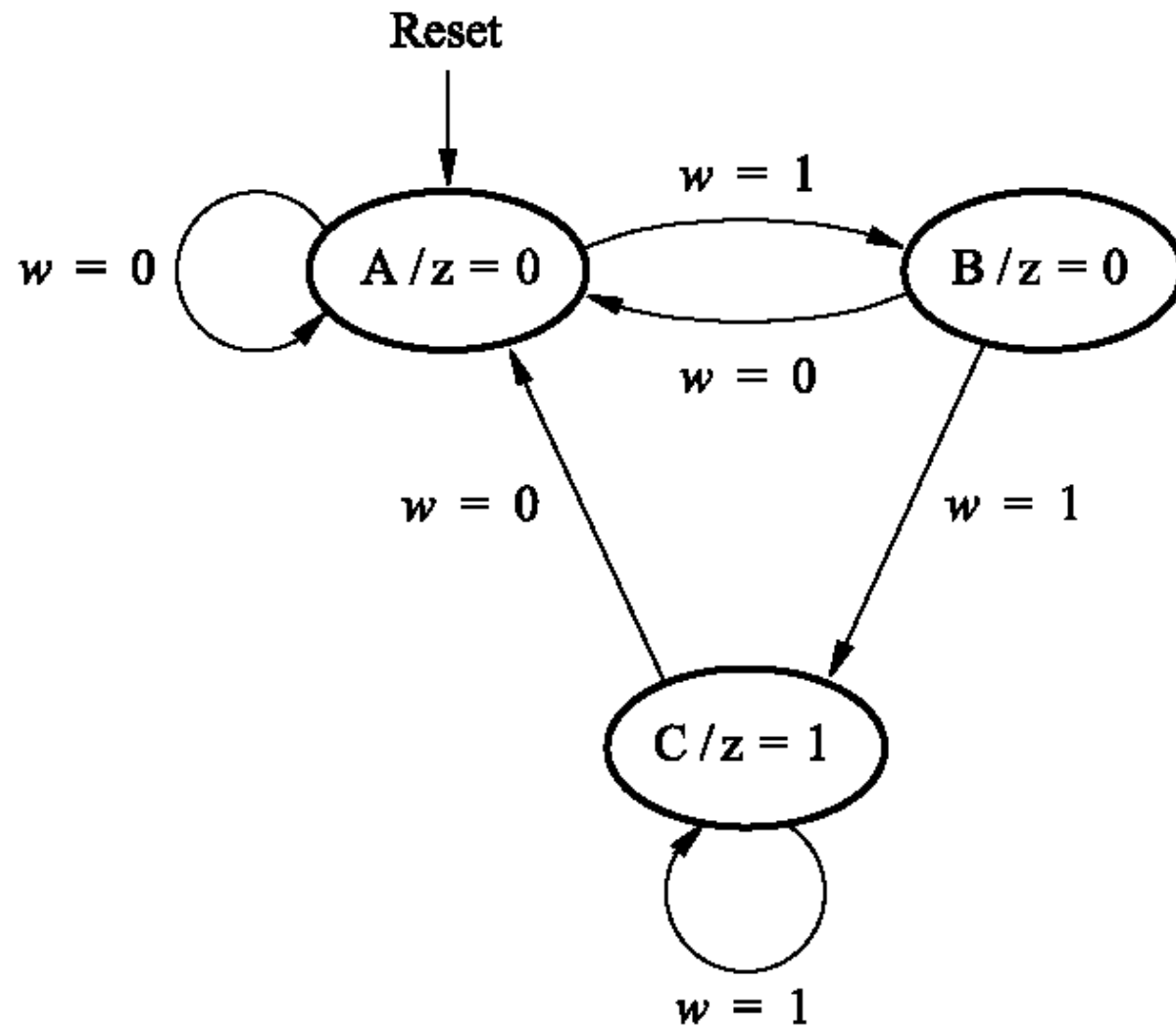


圖3 簡單順序電路的狀態圖。



	現在 狀態 $y_3y_2y_1$	下個狀態		輸出 z
		$w = 0$	$w = 1$	
		$Y_3Y_2Y_1$	$Y_3Y_2Y_1$	
A	0 0 1	0 0 1	0 1 0	0
B	0 1 0	0 0 1	1 0 0	0
C	1 0 0	0 0 1	1 0 0	1

圖50 圖4順序電路的單一熱門指定。



範例3

- 單一熱門狀態指定可以用在圖18的順序電路，如圖51所示。需要四個狀態變數，而狀態A、B、C和D分別編碼為 $y_4y_3y_2y_1 = 0001$ 、 0010 、 0100 和 1000 。將剩下的12個狀態變數估值視為不理會項，下個狀態表示式為

$$\begin{aligned}Y_1 &= \bar{w}y_1 + y_4 \\Y_2 &= wy_1 \\Y_3 &= y_2 \\Y_4 &= y_3\end{aligned}$$

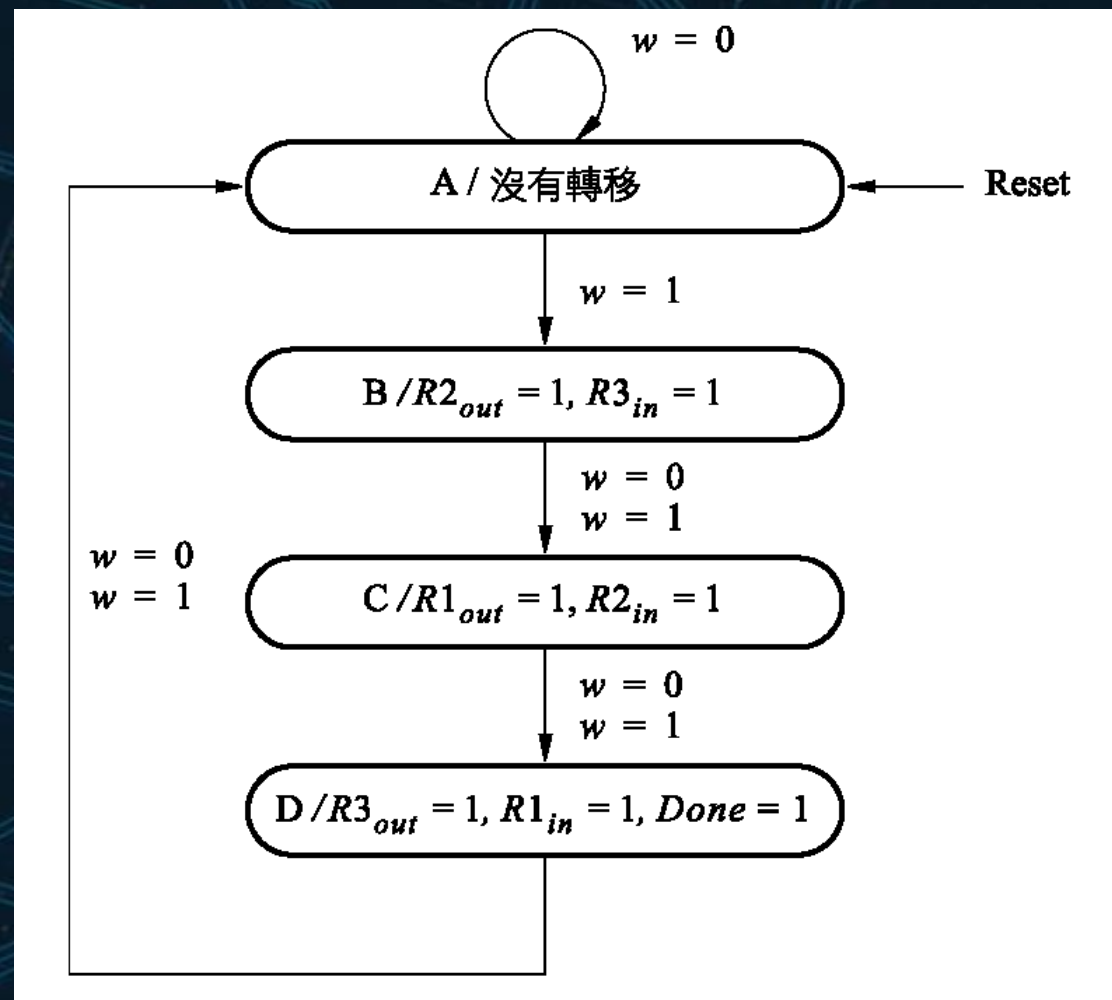


圖17 範例1的狀態圖。



- 注意，我們可以由圖17狀態圖看出這些表示式。若FSM在狀態A且 $w = 0$ ，或FSM在狀態D，則正反器 y_1 應該被設成1，因此 $Y_1 = \bar{w}y_1 + y_4$ 。若現在狀態為A且 $w = 1$ ，則正反器 y_2 應該被設成1，因此 $Y_2 = wy_1$ 。若FSM現在在狀態B或C，則正反器 y_3 和 y_4 應該分別被設成1，因此 $Y_3 = y_2$ 且 $Y_4 = y_3$ 。
- 輸出表示式即為正反器輸出，使得

$$R1_{out} = R2_{in} = y_3$$

$$R1_{in} = R3_{out} = Done = y_4$$

$$R2_{out} = R3_{in} = y_2$$

- 這些表示式都比範例2所得到的還簡單，但是需要四個正反器，而不是兩個。



	現在 狀態	下個狀態		輸出						
		w = 0	w = 1							
		$y_4y_3y_2y_1$	$Y_4Y_3Y_2Y_1$ $Y_4Y_3Y_2Y_1$							
A	0 0 0 1	0 0 0 1	0 0 1 0	0	0	0	0	0	0	0
B	0 0 1 0	0 1 0 0	0 1 0 0	0	0	1	0	0	1	0
C	0 1 0 0	1 0 0 0	1 0 0 0	1	0	0	1	0	0	0
D	1 0 0 0	0 0 0 1	0 0 0 1	0	1	0	0	1	0	1

圖51 圖18順序電路的單一熱門狀態指定。



十二、Mealy狀態模型 Mealy State Model

- 假設以D正反器實現y，則所需的下個狀態與輸出表示式為
$$\begin{aligned} Y &= D = w \\ z &= wy \end{aligned}$$
- 結果電路與時序圖如圖6.26所示。時序圖對應到圖44的輸入輸出序列。



時脈循環:	t ₀	t ₁	t ₂	t ₃	t ₄	t ₅	t ₆	t ₇	t ₈	t ₉	t ₁₀
w:	0	1	0	1	1	0	1	1	1	0	1
z:	0	0	0	0	1	0	0	1	1	0	0

圖44 輸入和輸出訊號序列。

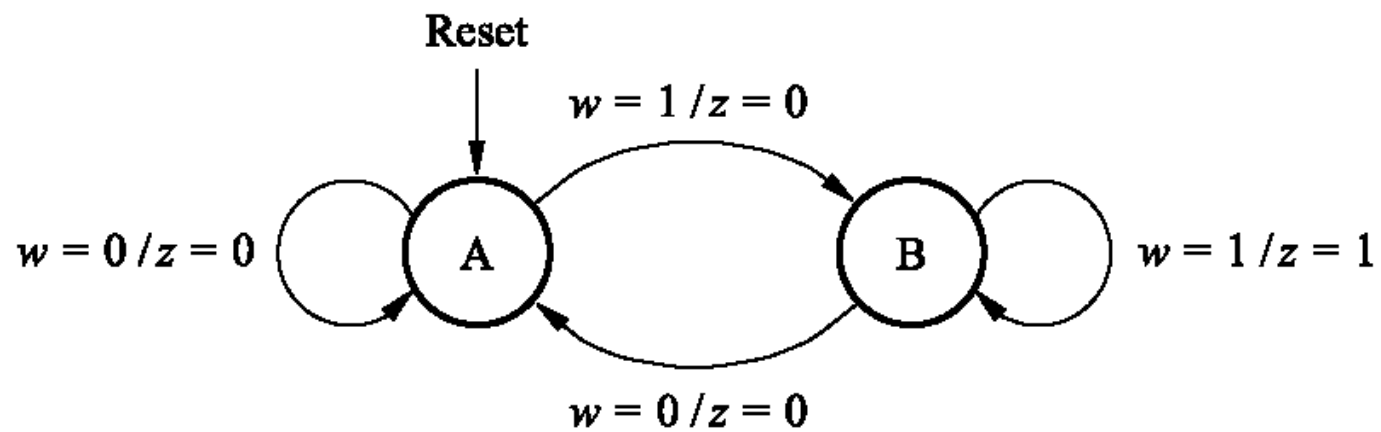


圖52 實現圖44工作的FSM的狀態圖。



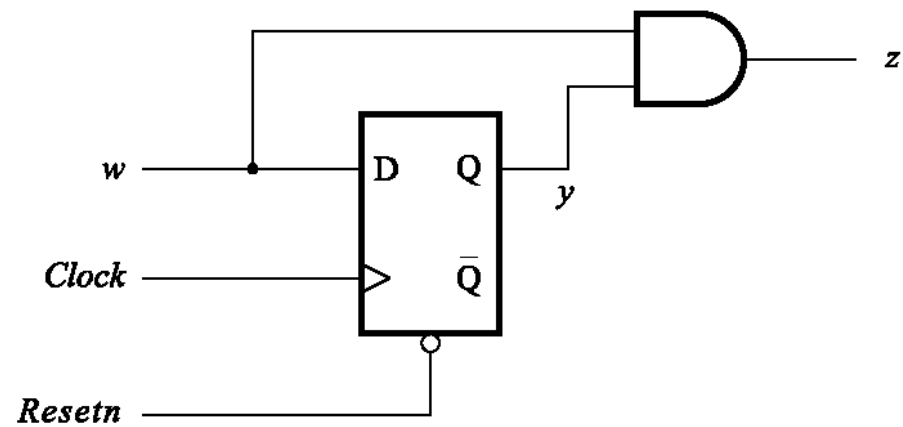
現在 狀態	下個狀態		輸出 z	
	$w = 0$	$w = 1$	$w = 0$	$w = 1$
A	A	B	0	0
B	A	B	0	1

圖53 圖52中FSM的狀態表。

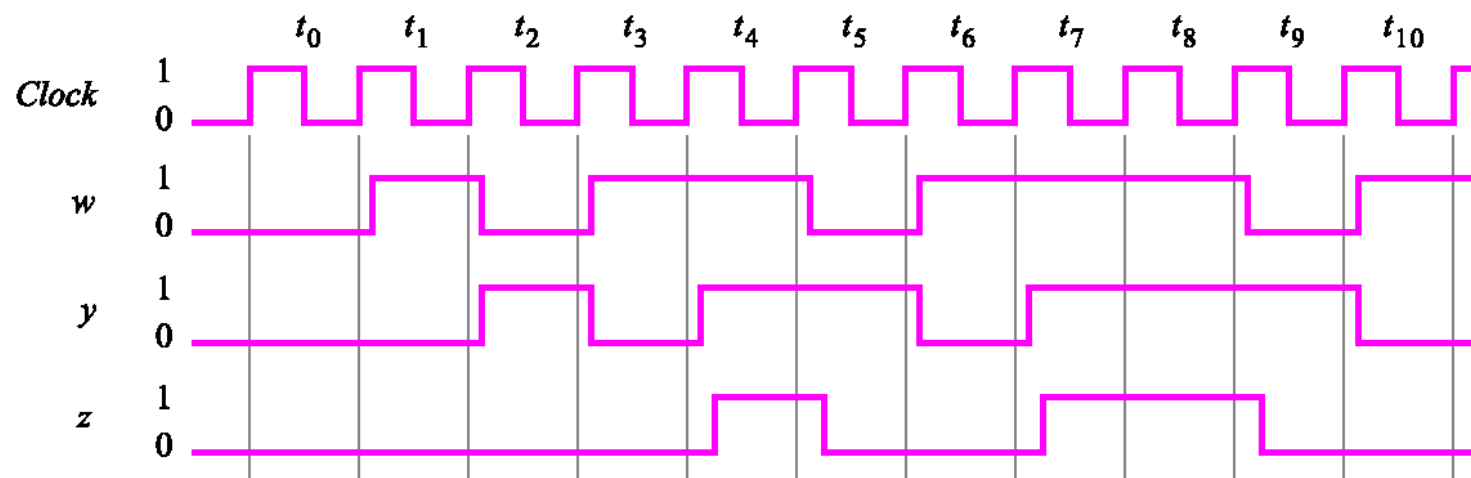


	現在 狀態	下個狀態		輸出	
		$w = 0$	$w = 1$	$w = 0$	$w = 1$
	y	Y	Y	z	z
A	0	0	1	0	0
B	1	0	1	0	1

圖54 圖53中FSM的狀態指定表。

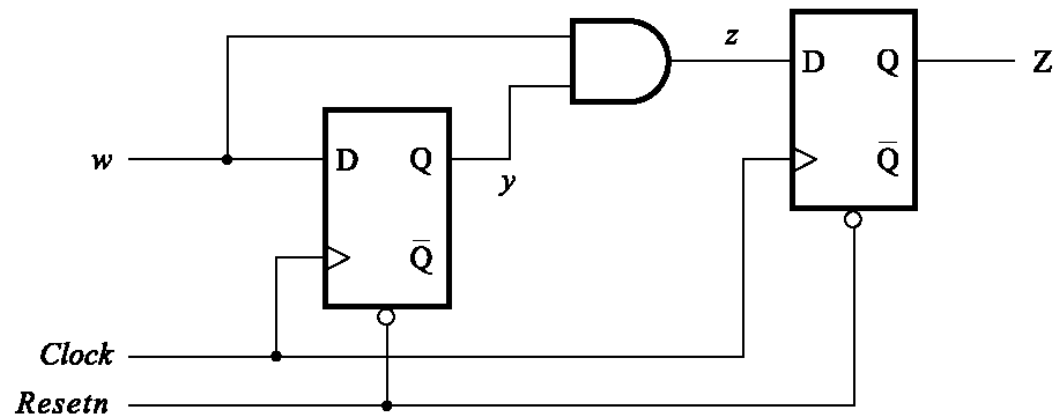


(a) 電路

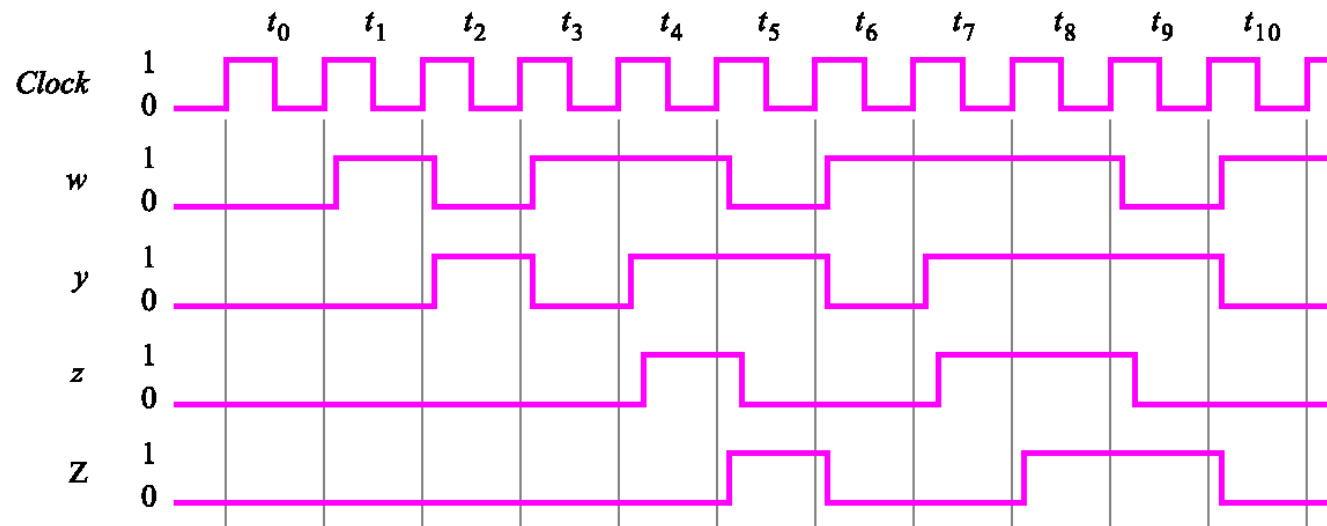


(b) 時序圖

圖55 圖54中FSM的實作。



(a) 電路



(b) 時序圖

圖56 實作圖44規格的電路。



範例4

- 我們在範例1考慮交換兩個暫存器內容的控制電路，以Moore類型FSM實作。同樣的工作也可以由Mealy類型FSM實作，如圖57所示。
- 狀態A同樣作為重置狀態，但是一旦 w 從0變成1，輸出控制訊號 $R2_{out}$ 和 $R3_{in}$ 被設定。
- 當電路離開狀態A到狀態B時，這些訊號保持被設定直到下一個時脈循環的開始。
- 在狀態B時，對 $w = 0$ 和 $w = 1$ 時輸出都被設定。
- 最後，在狀態C設定 $R3_{out}$ 和 $R1_{in}$ 可完成交換。



範例4

- 控制電路的Mealy類型實現需要三個狀態。這不一定會產生較簡單的電路，因為仍需要兩個正反器來實作狀態變數。
- 和Moore類型實現相比，最重要的不同是輸出訊號的時序。實作圖57中FSM的電路產生輸出控制訊號，比範例1和2的電路要快一個時脈循環。

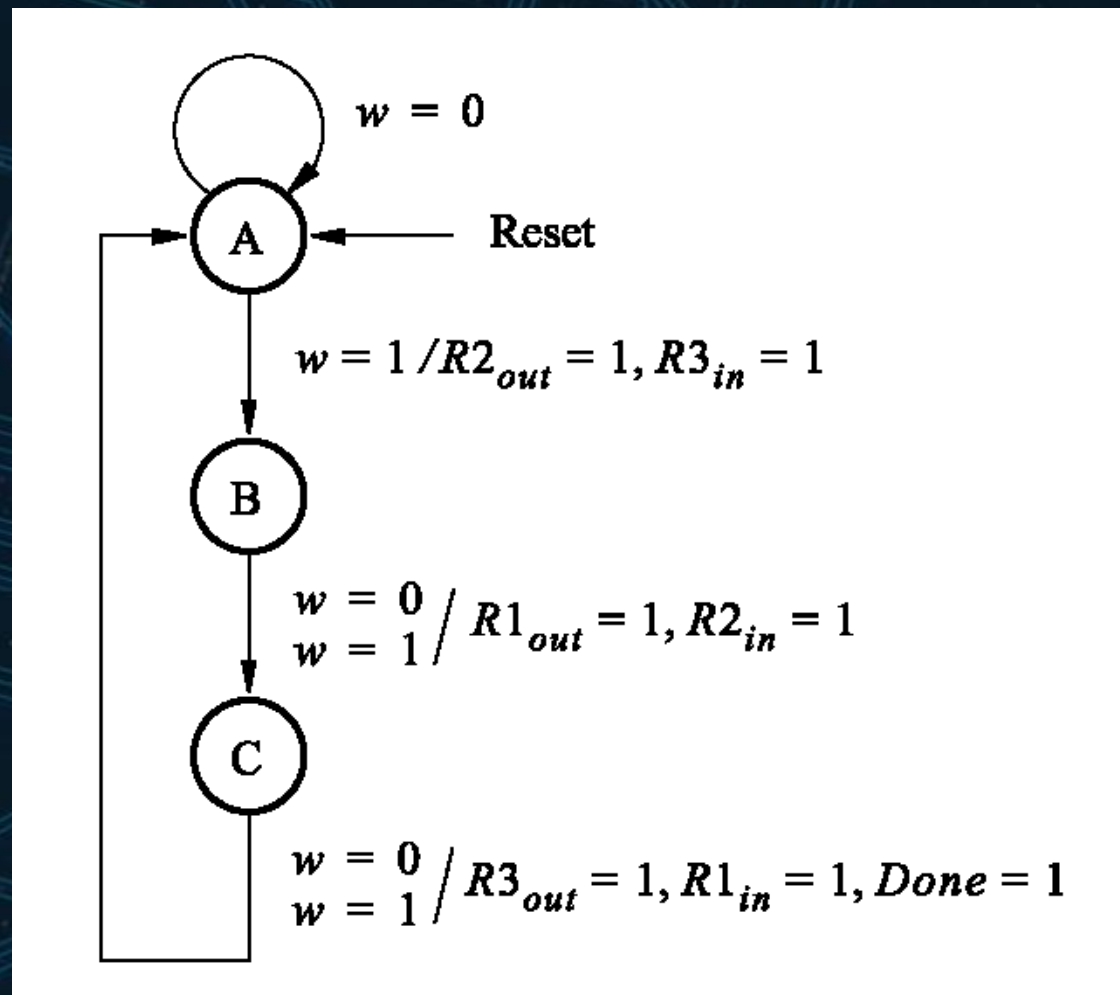


圖57 範例4的Mealy狀態圖。

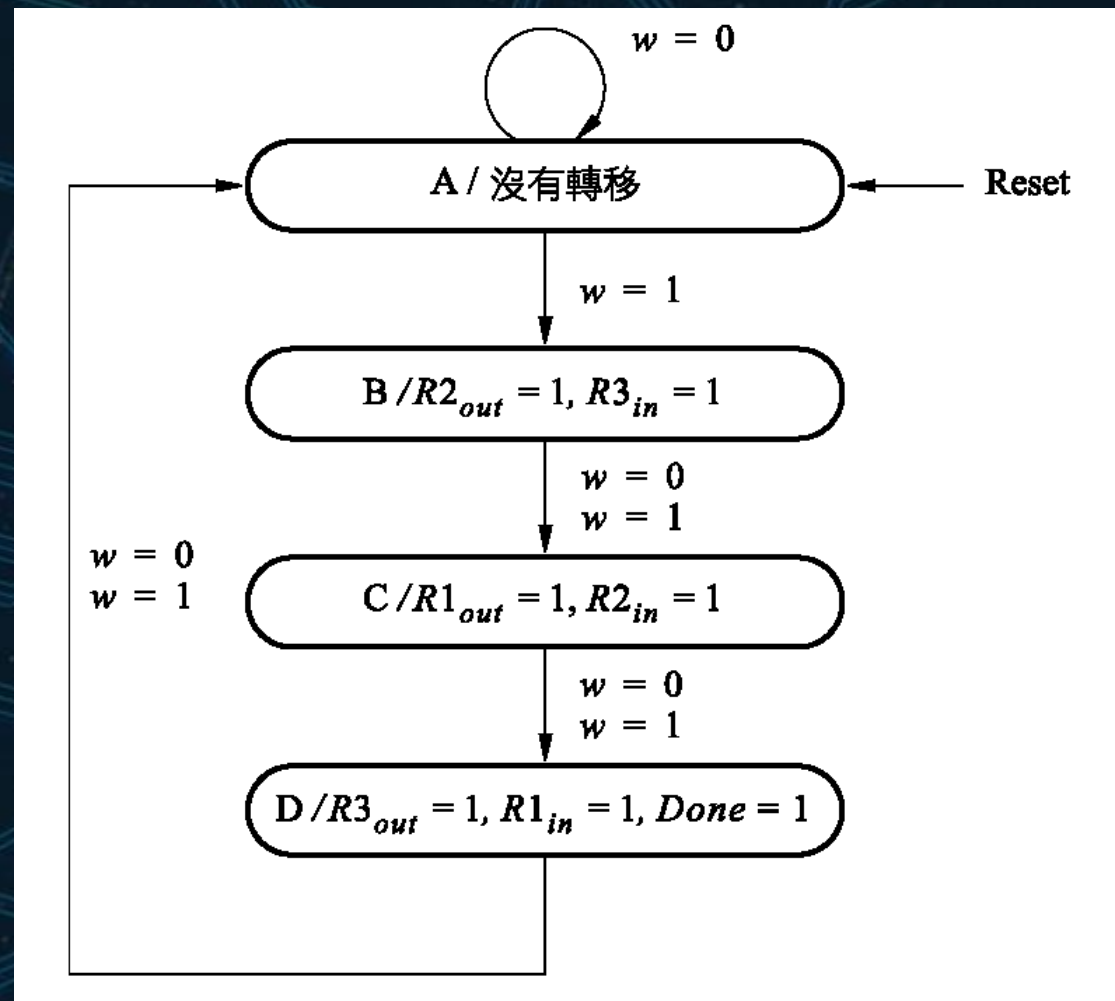


圖58 範例1的Moore狀態圖。



- 注意，使用圖57的FSM，**交換暫存器R1和R2的內容**，整個程序需要**三個時脈循環**，起於狀態A也結束於狀態A。使用範例1的**Moore類型FSM**，交換程序需要**四個時脈循環**，在回到狀態A之前。
- 假設我們想要以單一熱門編碼實作此FSM。需要三個正反器狀態A、B和C分別指定為估值 $y_3y_2y_1 = 001$ 、 010 和 100 。檢視圖57的狀態圖，我們可以用看的推導出下個狀態方程式。若FSM在狀態A且 $w = 0$ ，或FSM在狀態C，則正反器 y_1 的輸入值為1，因此 $Y_1 = \bar{w}y_1 + y_3$ 。若FSM在狀態A且 $w = 1$ ，則正反器 y_2 的輸入值為1，因此 $Y_2 = wy_1$ 。若現在狀態為B，則正反器 y_3 的輸入值為1，因此 $Y_3 = y_2$ 。



十三、以CAD工具設計有限狀態機器 Design Of Finite State Machines Using CAD Tools

- 使CAD工具設計FSM的基本方法如下：設計者以先前介紹的手動技巧，從狀態圖求出包含正反器和邏輯閘的電路。畫出電路圖或撰寫結構性硬體描述語言 (hardware description language, HDL) 程式碼，將此電路輸入CAD系統。



十三、以CAD工具設計有限狀態機器

Design Of Finite State Machines Using CAD Tools

- 從狀態圖手動合成是相當繁複的。因為CAD工具就是要消除這類工作，以CAD工具設計FSM有另一種更吸引人的方法。比較好的方式是直接將狀態圖輸入CAD系統，且自動化執行整個合成程序。
- CAD工具以兩種主要方式支援此方法。一個方法是允許設計者以類似圖示擷取工具的圖形工具畫出狀態圖。使用者以圓圈代表狀態，以弧形代表狀態轉換和應該產生的輸出。另一個更普遍的方法是用HDL撰寫代表狀態圖的程式碼。



十四、Moore類型FSM的VHDL程式碼

```
USE ieee.std_logic_1164.all ;

ENTITY simple IS
    PORT ( Clock, Resetn, w      : IN  STD_LOGIC ;
          z                      : OUT  STD_LOGIC ) ;
END simple ;

ARCHITECTURE Behavior OF simple IS
    TYPE State_type IS (A, B, C) ;
    SIGNAL y : State_type ;
BEGIN
    PROCESS ( Resetn, Clock )
    BEGIN
        IF Resetn = '0' THEN
            y <= A ;
        ELSIF Clock'EVENT AND Clock = '1' THEN
            con't ...
        END IF;
    END PROCESS;
END Behavior;
```

圖59 圖3的FSM的VHDL程式碼。



```
CASE y IS
    WHEN A =>
        IF w = '0' THEN
            y <= A;
        ELSE
            y <= B;
        END IF;
    WHEN B =>
        IF w = '0' THEN
            y <= A;
        ELSE
            y <= C;
        END IF;
    WHEN C =>
        IF w = '0' THEN
            y <= A;
        ELSE
            y <= C;
        END IF;
    END CASE;
END IF;
END PROCESS;
z <= '1' WHEN y = C ELSE '0';
END Behavior;
```

圖60 圖3的FSM的VHDL程式碼

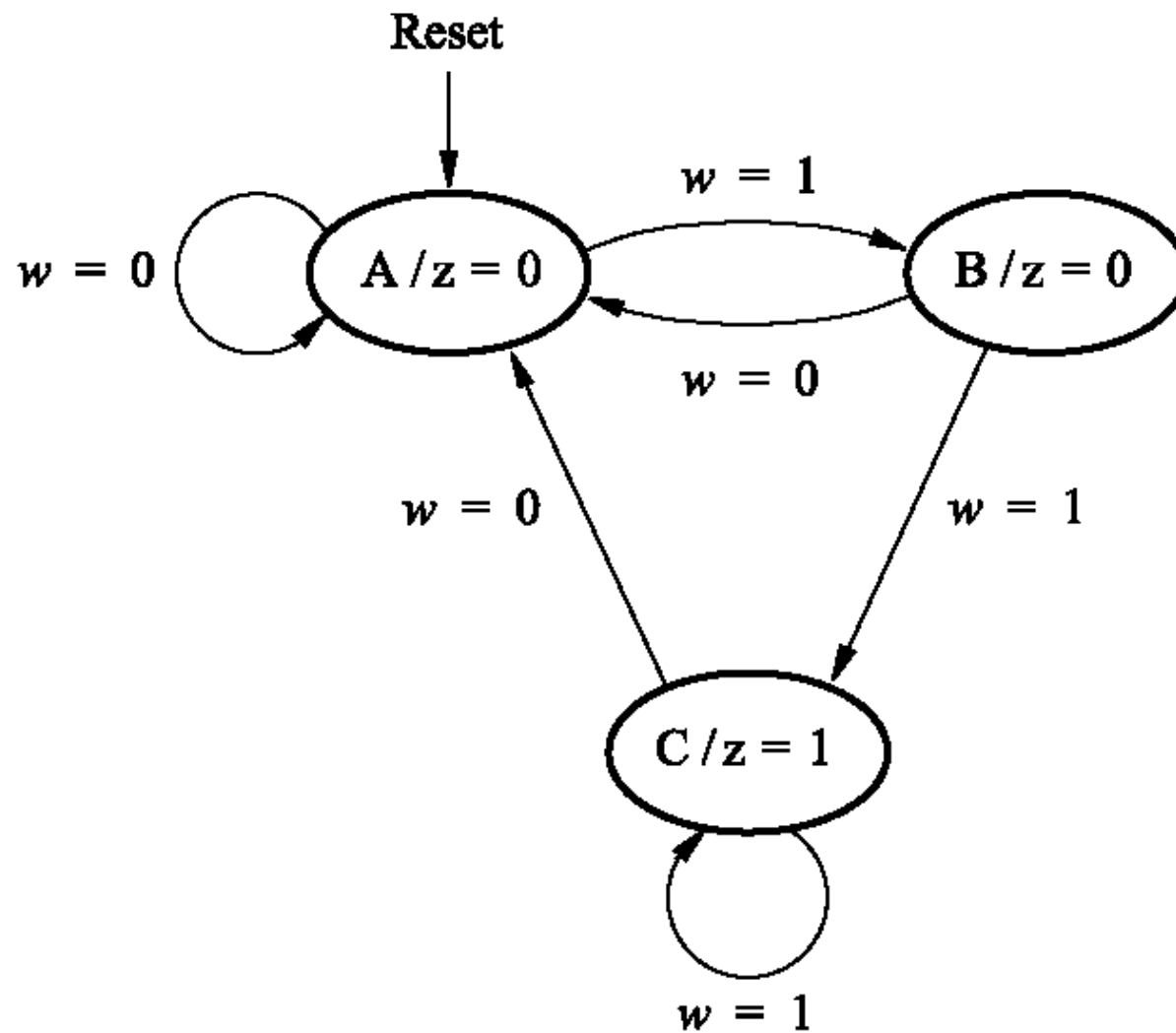


圖3 簡單順序電路的狀態圖。



十五、VHDL程式碼的合成 Synthesis Of VHDL Code

- 我們以圖59及圖60的程式碼為例，以合成工具將其合成，實作於CPLD。此合成產生兩個正反器，輸入為 Y_1 和 Y_2 ，輸出為 y_1 和 y_2 。合成工具產生的下個狀態表示式為

$$Y_1 = w\bar{y}_1\bar{y}_2$$

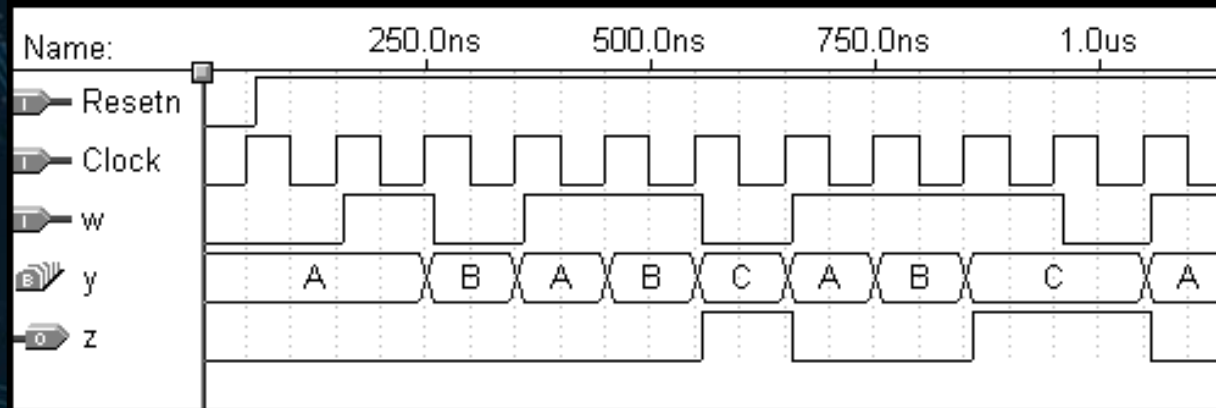
$$Y_2 = wy_1 + wy_2$$

- 輸出表示式為 $z = \bar{y}_1y_2$
- 這些式子對應到圖7的情況，沒有用到的狀態樣式 $y_1y_2 = 11$ 在 Y_1 、 Y_2 和 z 的卡諾圖中被視為不理會項。

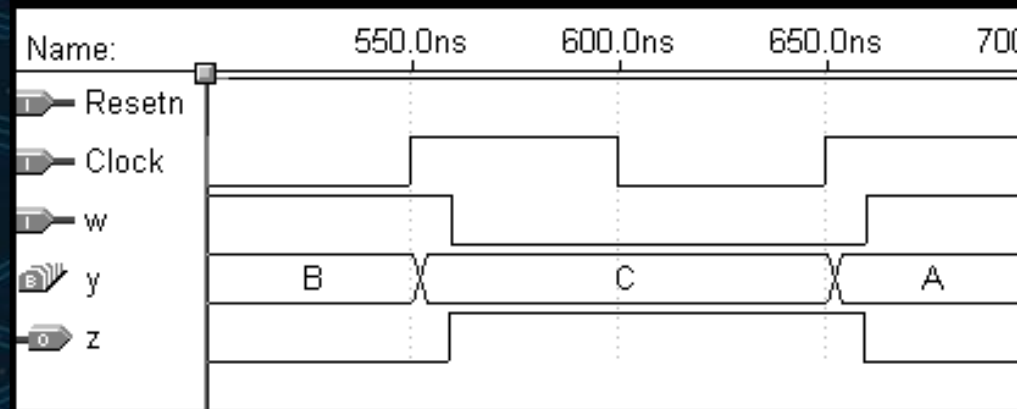


- 從此專用接腳，有個**全域接線 (global wire)** 將時脈訊號分佈到晶片中每一個正反器。全域接線將時脈訊號分佈到正反器，使得每個正反器時脈訊號的到達時間差異，稱為**時脈偏斜 (clock skew)**，可以最小化。





(a) 時序模擬結果



(b) 將模擬結果放大，顯示時序的細節

圖62 圖61電路的模擬結果。



十六、VHDL程式碼的其他風格 Alternative Styles Of VHDL Code

```
ARCHITECTURE Behavior OF simple IS
    TYPE State_type IS (A B C);
    SIGNAL y_present, y_next : State_type;
BEGIN
    PROCESS ( w, y_present )
    BEGIN
        CASE y_present IS
            WHEN A =>
                IF w = '0' THEN
                    y_next <= A;
                ELSE
                    y_next <= B;
                END IF;
            WHEN B =>
                IF w = '0' THEN
                    y_next <= A;
                ELSE
                    y_next <= C;
                END IF;
        END CASE;
    END PROCESS;
```

```
        WHEN C =>
            IF w = '0' THEN
                y_next <= A;
            ELSE
                y_next <= C;
            END IF;
        END CASE;
    END PROCESS;

    PROCESS (Clock, Resetn)
    BEGIN
        IF Resetn = '0' THEN
            y_present <= A;
        ELSIF (Clock'EVENT AND Clock = '1')
        THEN
            y_present <= y_next;
        END IF;
    END PROCESS;

    z <= '1' WHEN y_present = C ELSE '0'
END Behavior;
```

圖63 圖3的FSM程式碼的另一種VHDL程式碼



十七、VHDL程式碼的狀態指定 Specifying the State Assignment in VHDL Code

- 狀態指定可能會影響設計電路的複雜度。狀態指定程序的明顯目的是將實作成本最小化。必須最佳化的成本函數為邏輯閘和正反器的個數
- 特性 (ATTRIBUTE) 是參照VHDL程式碼中物件的一些資訊類型。所有的訊號都自動有許多相關的預先定義的(predefined)特性。
- 除了預先定義的特性之外，也可以建立使用者定義的(user-defined)特性。



```
ARCHITECTURE Behavior OF simple IS
  TYPE State_TYPE IS (A, B, C);
  ATTRIBUTE ENUM_ENCODING                : STRING;
  ATTRIBUTE ENUM_ENCODING OF State_type  : TYPE IS "00 01 11";
  SIGNAL y_present, y_next : State_type;
BEGIN

con't ...
```

圖 64 手動狀態指定的使用者定義特性



十八、以VHDL指定FSM規格 Specification Of Fsms Using VHDL

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all ;

ENTITY simple IS
    PORT ( Clock, Resetn, w      : IN  STD_LOGIC ;
          z                      : OUT STD_LOGIC ) ;
END simple ;

ARCHITECTURE Behavior OF simple IS
    SIGNAL y_present, y_next : STD_LOGIC_VECTOR(1 DOWNT0 0);
    CONSTANT A : STD_LOGIC_VECTOR(1 DOWNT0 0) := "00" ;
    CONSTANT B : STD_LOGIC_VECTOR(1 DOWNT0 0) := "01" ;
    CONSTANT C : STD_LOGIC_VECTOR(1 DOWNT0 0) := "11" ;
BEGIN
    PROCESS ( w, y_present )
    BEGIN
        CASE y_present IS
            WHEN A =>
                IF w = '0' THEN y_next <= A ;
                ELSE y_next <= B ;
                END IF ;
            ... con't
```

圖 65 使用常數作為手動狀態指定(Moore機形式)



```
        WHEN B =>
            IF w = '0' THEN y_next <= A;
            ELSE y_next <= C;
            END IF;
        WHEN C =>
            IF w = '0' THEN y_next <= A;
            ELSE y_next <= C;
            END IF;
        WHEN OTHERS =>
            y_next <= A;
    END CASE;
END PROCESS;

PROCESS ( Clock, Resetn )
BEGIN
    IF Resetn = '0' THEN
        y_present <= A;
    ELSIF (Clock'EVENT AND Clock = '1') THEN
        y_present <= y_next;
    END IF;
END PROCESS;
z <= '1' WHEN y_present = C ELSE '0'
END Behavior;
```

圖 66 使用常數作為手動狀態指定(Moore機形式)

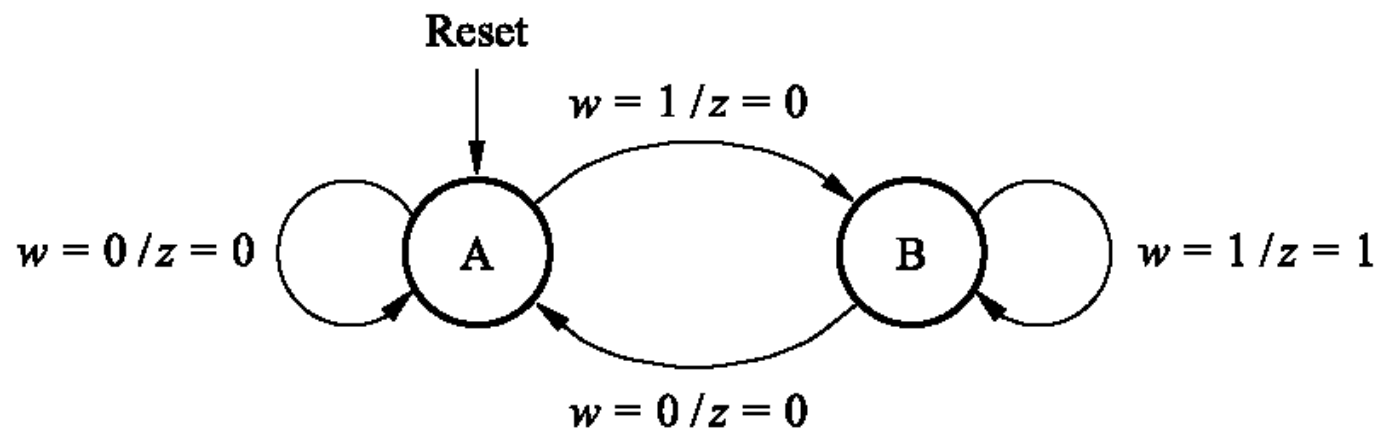


圖52 實現圖44工作的FSM的狀態圖。



```
LIBRARY ieee ;
USE ieee.std_logic_1164.all ;

ENTITY mealy IS
    PORT ( Clock, Resetn, w      : IN    STD_LOGIC ;
          z                      : OUT  STD_LOGIC ) ;
END mealy ;

ARCHITECTURE Behavior OF mealy IS
    TYPE State_type IS (A, B) ;
    SIGNAL    State_type ;
BEGIN
    PROCESS ( Resetn, Clock )
    BEGIN
        IF Resetn = '0' THEN
            y <= A ;
        ELSIF (Clock'EVENT AND Clock = '1') THEN
            CASE y IS
                WHEN A =>
                    IF w = '0' THEN y <= A ;
                    ELSE y <= B ;
                    END IF ;
            ... con't
```

圖67 圖52的Mealy機器的VHDL程式碼。



```
        WHEN B =>
            IF w = '0' THEN y <= A;
            ELSE y <= B;
            END IF;
        END CASE;
    END IF;
END PROCESS;

PROCESS ( y, w ) --Mealy機輸出訊號由目前狀態與輸入訊號決定
BEGIN
    CASE y IS
        WHEN A =>
            z <= '0';
        WHEN B =>
            z <= w;
        END CASE;
    END PROCESS;
END Behavior;
```

圖68 圖52的Mealy機器的VHDL程式碼。

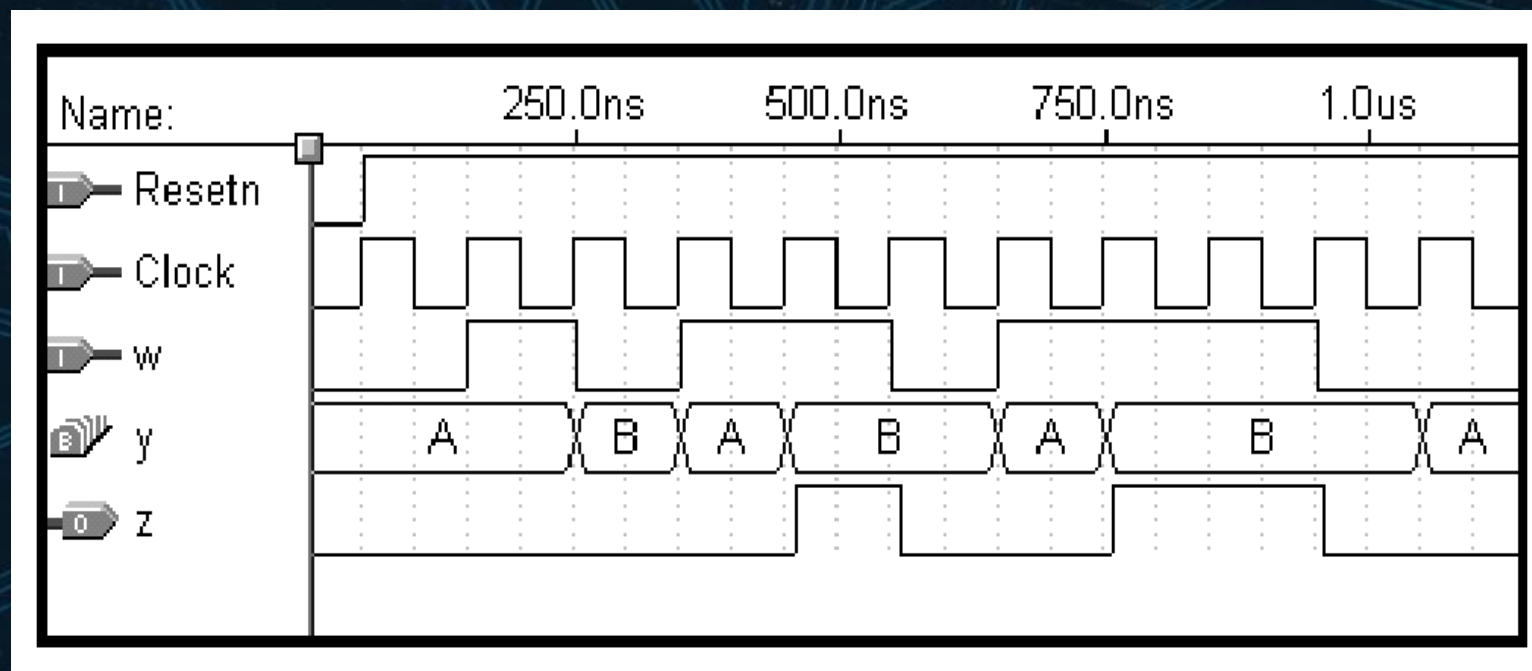


圖69 Mealy機器的模擬結果。



十九、串列加法器範例 Serial Adder Example

- 快速加法器更為複雜，因此也更為昂貴。若速度不是很重要，則成本有效率的選擇是串列加法器 (serial adder)，一個時間相加一對位元。



二十、串列加法器的Mealy類型FSM Mealy-type FSM For Serial Adder

- 對應的狀態表如圖72所示。需要單一正反器來表示兩個狀態。狀態指定可以如圖73所示。這指定產生下列下個狀態和輸出方程式

$$Y = ab + ay + by$$
$$s = a \oplus b \oplus y$$

- 將這些式子和5.2節的全加器相比，明顯發現 y 為進位輸入， Y 為進位輸出， s 為全加器的總和。因此，圖70加法器FSM方塊是由圖74的電路組成。正反器可在加法運算開始時用Reset訊號清除。

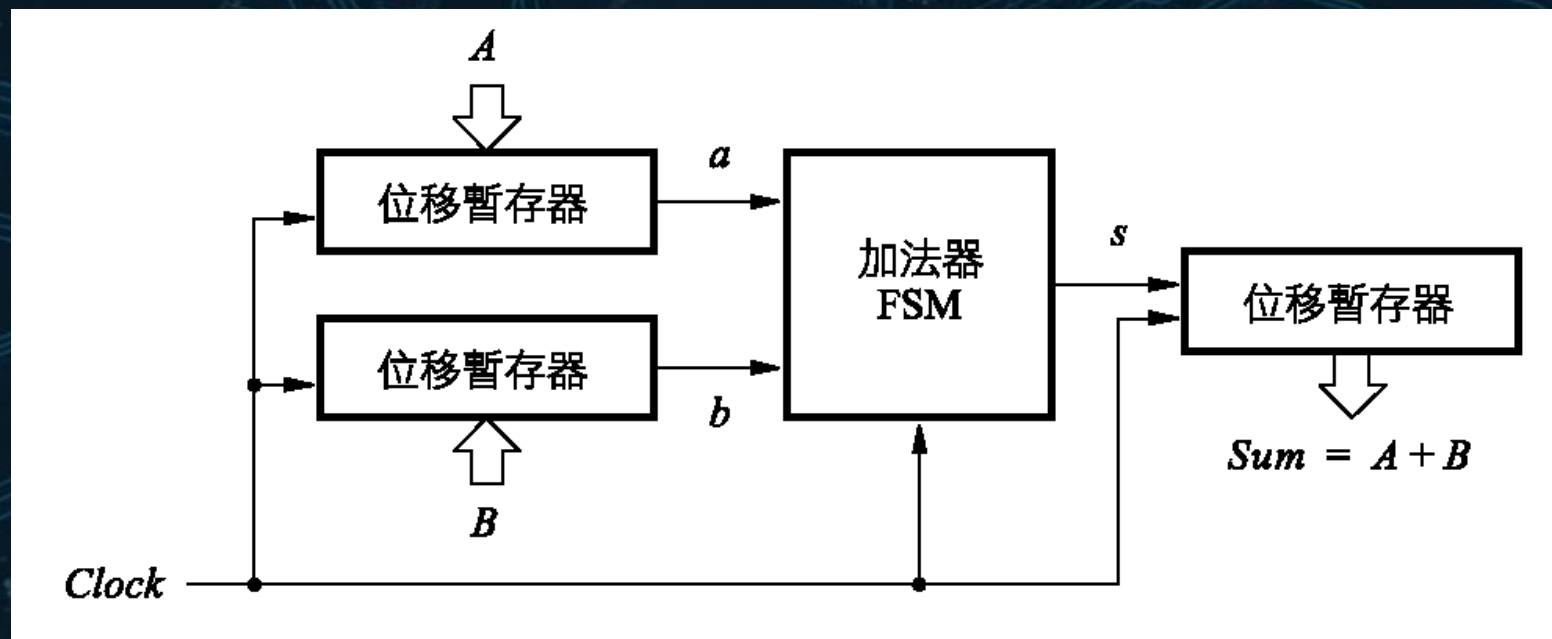


圖70 串列加法器的方塊圖。

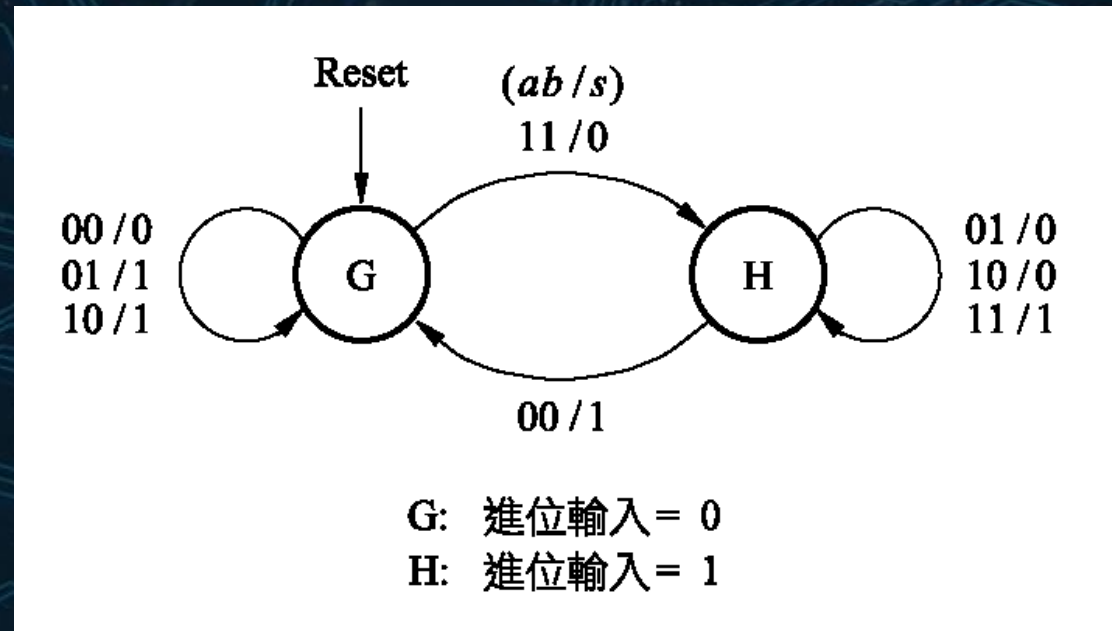


圖71 串列加法器FSM的狀態圖。



現在 狀態	下個狀態				輸出 s			
	$ab = 00$	01	10	11	00	01	10	11
G	G	G	G	H	0	1	1	0
H	G	H	H	H	1	0	0	1

圖72 串列加法器FSM的狀態表。



現在 狀態 y	下個狀態				輸出			
	$ab = 00$	01	10	11	00	01	10	11
	Y				s			
0	0	0	0	1	0	1	1	0
1	0	1	1	1	1	0	0	1

圖73 圖72的狀態指定表。

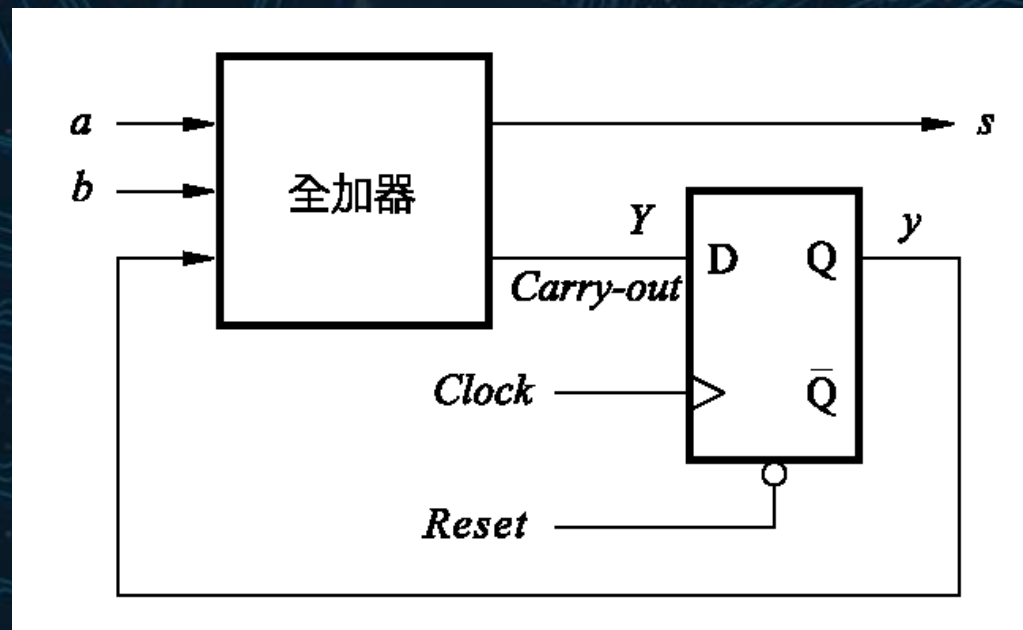


圖74 圖70的加法器FSM電路。



二十一、串列加法器的Moore類型FSM

Moore-Type FSM for Serial Adder

- 對應的狀態表如圖76所示，狀態指定表如圖77所示。下個狀態和

輸出表示式為

$$\begin{aligned}Y_1 &= a \oplus b \oplus y_2 \\Y_2 &= ab + ay_2 + by_2 \\s &= y_1\end{aligned}$$

- Y_1 和 Y_2 的表示式對應到全加器電路的總和和進位輸出表示式。

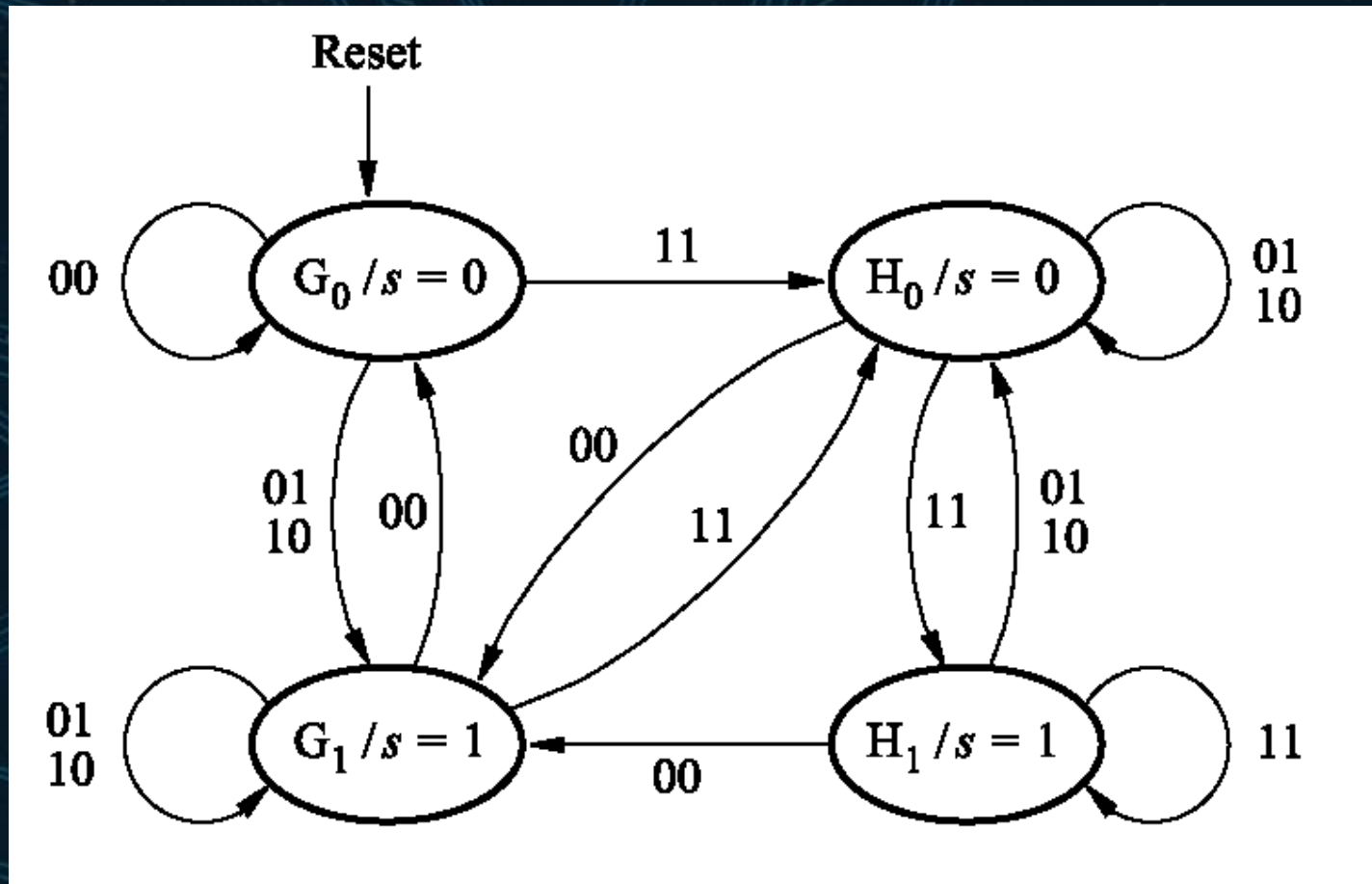


圖75 Moore類型串列加法器FSM的狀態圖。



現在 狀態	下個狀態				輸出 s
	$ab = 00$	01	10	11	
G_0	G_0	G_1	G_1	H_0	0
G_1	G_0	G_1	G_1	H_0	1
H_0	G_1	H_0	H_0	H_1	0
H_1	G_1	H_0	H_0	H_1	1

圖76 Moore類型串列加法器FSM的狀態表。



現在 狀態 y_2y_1	下個狀態				輸出 s
	$ab = 00$	01	10	11	
	Y_2Y_1				
0 0	0 0	0 1	0 1	1 0	0
0 1	0 0	0 1	0 1	1 0	1
1 0	0 1	1 0	1 0	1 1	0
1 1	0 1	1 0	1 0	1 1	1

圖77 圖76的狀態指定表。

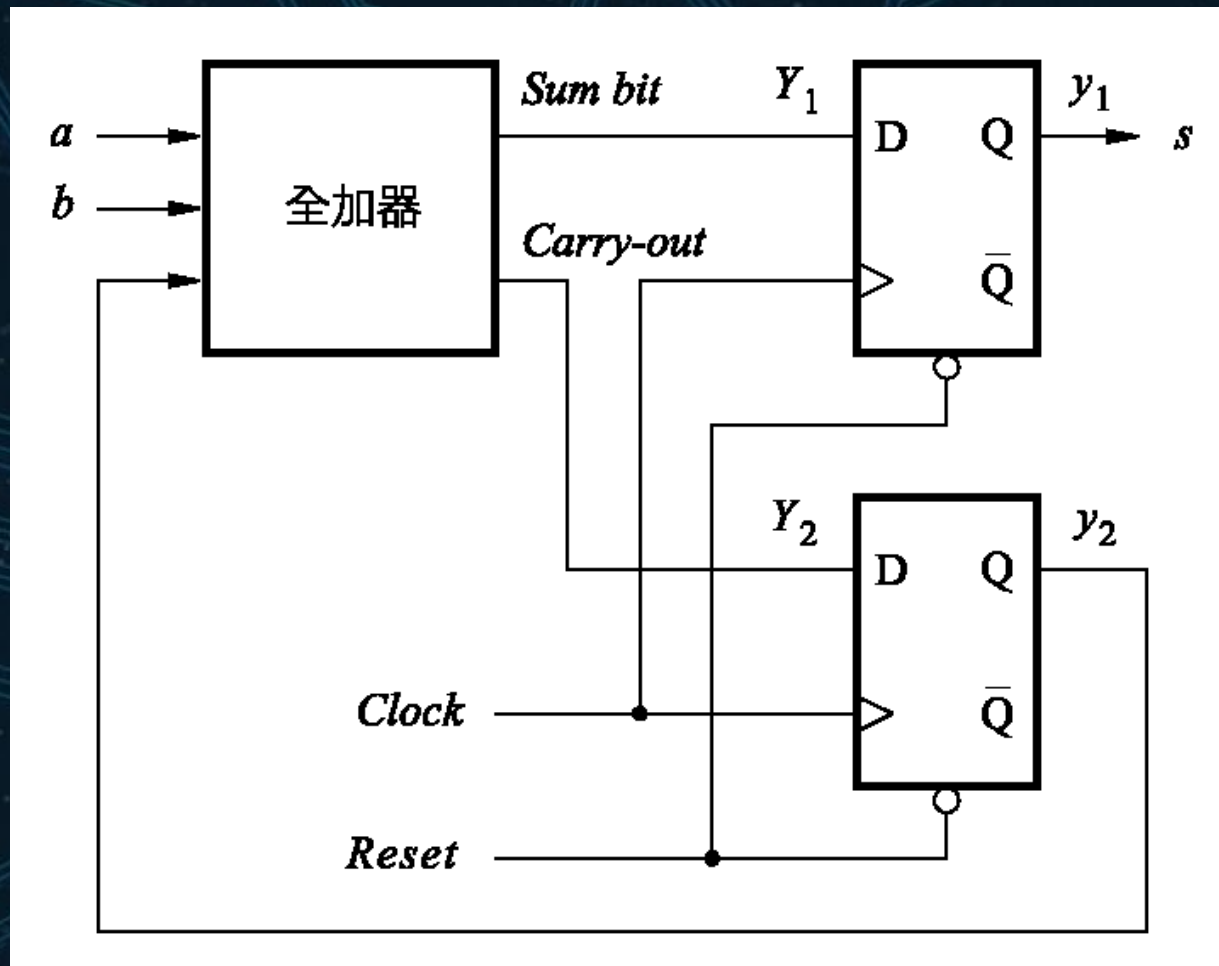


圖78 Moore類型串列加法器FSM的電路。



二十二、串列加法器的VHDL程式碼

- 串列加法器可以撰寫VHDL程式碼描述，由位移暫存器和加法器FSM組成。我們首先設計位移暫存器，然後將其作為串列加法器的子電路。



位移暫存子電路 Shift Register Subcircuit

- 圖80此串列加法器的好處是可以避免當有效時脈邊緣發生時位移暫存器的內容改變。圖79為位移暫存器的程式碼`shiftrne`，致能輸入為E。當 $E = 1$ ，位移暫存器的行為和圖80相同。設 $E = 0$ 避免位移暫存器的內容改變。**E輸入通常稱為致能(enable)輸入。**



```
LIBRARY ieee ;
USE ieee.std_logic_1164.all ;
-- left-to-right shift register with parallel load and enable
ENTITY shiftrne IS
    GENERIC ( N : INTEGER := 4 ) ;
    PORT ( R      : IN          STD_LOGIC_VECTOR(N-1 DOWNT0 0) ;
           L, E, w : IN          STD_LOGIC ;
           Clock   : IN          STD_LOGIC ;
           Q       : BUFFER      STD_LOGIC_VECTOR(N-1 DOWNT0 0) ) ;
END shiftrne ;
ARCHITECTURE Behavior OF shiftrne IS
BEGIN

PROCESS
    BEGIN
        WAIT UNTIL Clock'EVENT AND Clock = '1' ;
        IF E = '1' THEN
            IF L = '1' THEN
                Q <= R ;
            ELSE
                Genbits: FOR i IN 0 TO N-2 LOOP
                    Q(i) <= Q(i+1) ;
                END LOOP ;
                Q(N-1) <= w ;
            END IF ;
        END IF ;
    END PROCESS ;
END Behavior ;
```

圖 79 具有致能輸入的左往右位移暫存器程式碼。



完整程式碼 Complete Code

- 串列加法器的程式碼如圖81所示，實體化三個位移暫存器，輸入為A和B，輸出為Sum。當電路重置時位移暫存器平行載入資料。
- 圖81的程式碼實作八位元數值的串列加法器。

VHDL程式碼的合成和模擬

Synthesis And Simulation Of The VHDL Code

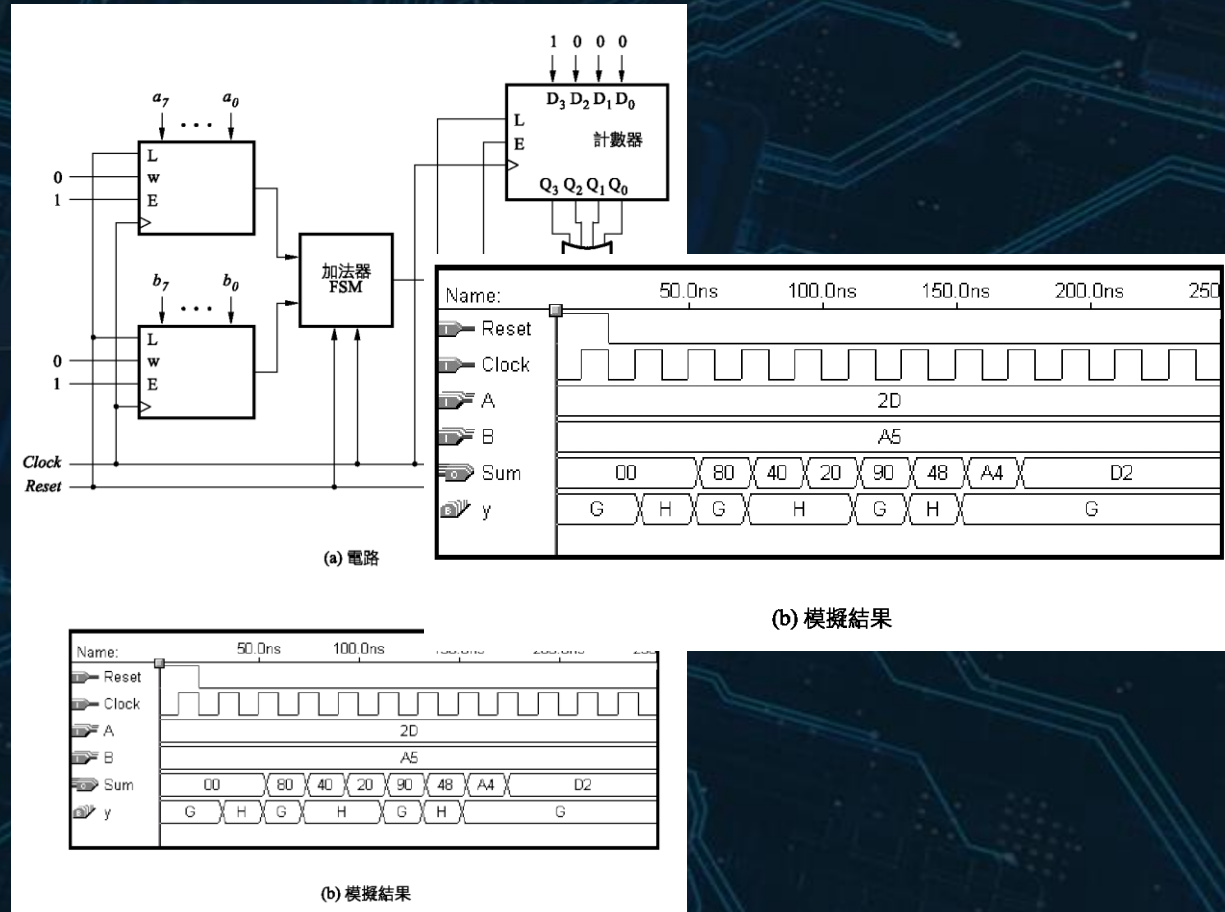


圖80 合成的串列加法器。



```
LIBRARY ieee ;
USE ieee.std_logic_1164.all ;
ENTITY serial IS
    GENERIC ( length : INTEGER := 8 ) ;
    PORT (   Clock      : IN          STD_LOGIC ;
            Reset       : IN          STD_LOGIC ;
            A, B        : IN          STD_LOGIC_VECTOR(length-1 DOWNT0 0) ;
            Sum         : BUFFER      STD_LOGIC_VECTOR(length-1 DOWNT0 0) );
END serial ;
ARCHITECTURE Behavior OF serial IS
    COMPONENT shiftreg
        GENERIC ( N : INTEGER := 4 ) ;
        PORT (   R          : IN          STD_LOGIC_VECTOR(N-1 DOWNT0 0) ;
                L, E, w     : IN          STD_LOGIC ;
                Clock       : IN          STD_LOGIC ;
                Q           : BUFFER      STD_LOGIC_VECTOR(N-1 DOWNT0 0) );
    END COMPONENT ;
    SIGNAL QA, QB, Null_in : STD_LOGIC_VECTOR(length-1 DOWNT0 0) ;
    SIGNAL s, Low, High, Run : STD_LOGIC ;
    SIGNAL Count : INTEGER RANGE 0 TO length ;
    TYPE State_type IS (G, H) ;
    SIGNAL y : State_type ;
... con't
```

圖81 串列加法器的VHDL程式碼。(a部分)



```
BEGIN
  Low <= '0' ; High <= '1' ;
  ShiftA: shiftrne GENERIC MAP (N => length)
    PORT MAP ( A, Reset, High, Low, Clock, QA ) ;
  ShiftB: shiftrne GENERIC MAP (N => length)
    PORT MAP ( B, Reset, High, Low, Clock, QB )
  AdderFSM: PROCESS ( Reset, Clock )
  BEGIN
    IF Reset = '1' THEN
      y <= G ;
    ELSIF Clock'EVENT AND Clock = '1' THEN
      CASE y IS
        WHEN G =>
          IF QA(0) = '1' AND QB(0) = '1' THEN y <= H ;
          ELSE y <= G ;
          END IF ;
        WHEN H =>
          IF QA(0) = '0' AND QB(0) = '0' THEN y <= G ;
          ELSE y <= H ;
          END IF ;
      END CASE ;
    END IF ;
  END PROCESS AdderFSM ;
... con't
```

圖81 串列加法器的VHDL程式碼。(b部分)



```
--Mealy機輸出訊號由目前狀態與輸入訊號決定
WITH y SELECT
    s <= QA(0) XOR QB(0) WHEN G,
        NOT ( QA(0) XOR QB(0) ) WHEN H ;
Null_in <= (OTHERS
ShiftSum: shiftrne GENERIC MAP ( N => length )
    PORT MAP ( Null_in, Reset, Run, s, Clock, Sum ) ;
Stop: PROCESS    -- Counter計算是否所有bit皆已計算過
BEGIN
    WAIT UNTIL (Clock'EVENT AND Clock = '1');
    IF Reset = '1' THEN
        Count <= length ;
    ELSIF Run = '1' THEN
        Count <= Count -1 ;
    END IF ;
END PROCESS ;
Run <= '0' WHEN Count = 0 ELSE '1' ; -- stops counter and ShiftSum
END Behavior ;
```

圖81 串列加法器的VHDL程式碼。(c 部分)



二十三、作為仲裁器電路的FSM FSM As An Arbiter Circuit

- 每個元件提供一個輸入到FSM，稱為**請求 (request)**，而FSM對每個元件產生個別輸出，稱為**授與 (grant)**。
- 以Moore類型FSM設計的狀態圖如圖82所示。
- 狀態圖重複於圖83。
- 圖84為此FSM的VHDL程式碼。

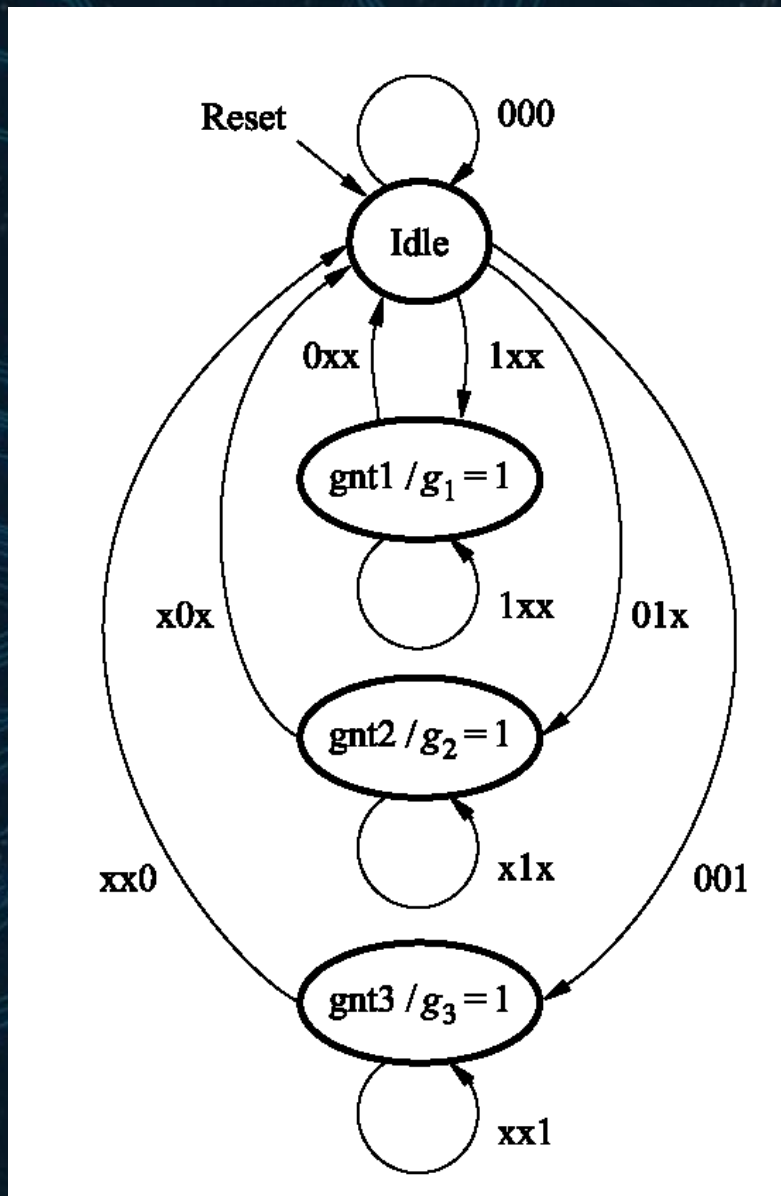


圖82 仲裁器的狀態圖。

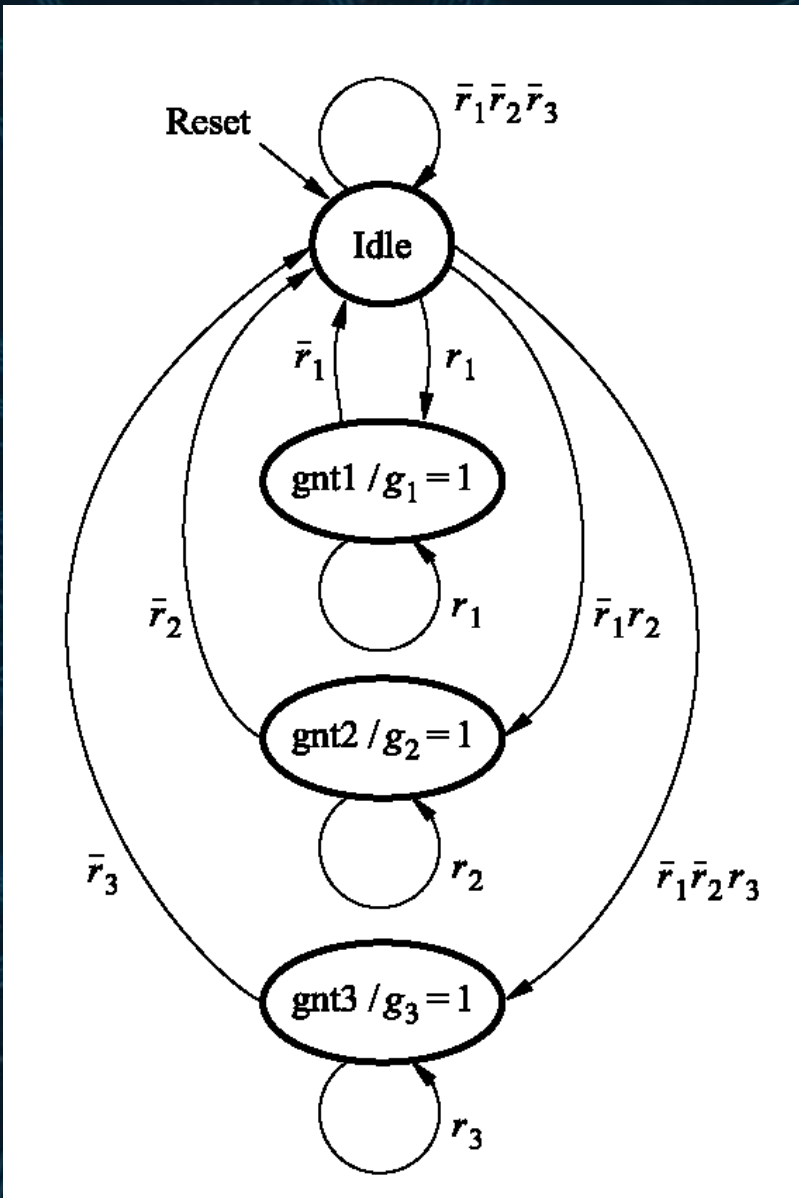


圖83 仲裁器狀態圖的另一種風格。



```
LIBRARY ieee;
USE ieee.std_logic_1164.all;

ENTITY arbiter IS
    PORT ( Clock, Resetn    : IN        STD_LOGIC ;
           r                : IN        STD_LOGIC_VECTOR(1 TO 3) ;
           g                : OUT       STD_LOGIC_VECTOR(1 TO 3) );
END arbiter ;

ARCHITECTURE Behavior OF arbiter IS
    TYPE State_type IS (Idle, gnt1, gnt2, gnt3) ;
    SIGNAL y : State_type ;
BEGIN
    PROCESS ( Resetn, Clock )
    BEGIN
        IF Resetn = '0' THEN y <= Idle ;
        ELSIF (Clock'EVENT AND Clock = '1') THEN
            CASE y IS
                WHEN Idle =>
                    IF r(1) = '1' THEN y <= gnt1 ;
                    ELSIF r(2) = '1' THEN y <= gnt2 ;
                    ELSIF r(3) = '1' THEN y <= gnt3 ;
                    ELSE y <= Idle ;
                    END IF ;
            ... con't
```

圖 84(a) 仲裁器的VHDL程式碼



```
        WHEN gnt1 =>
            IF r(1) = '1' THEN y <= gnt1 ;
            ELSE y <= Idle ;
            END IF ;
        WHEN gnt2 =>
            IF r(2) = '1' THEN y <= gnt2 ;
            ELSE y <= Idle ;
            END IF ;
        WHEN gnt3 =>
            IF r(3) = '1' THEN y <= gnt3 ;
            ELSE y <= Idle ;
            END IF ;
    END CASE ;
END IF ;
END PROCESS ;
g(1) <= '1' WHEN y = gnt1 ELSE '0' ;
g(2) <= '1' WHEN y = gnt2 ELSE '0' ;
g(3) <= '1' WHEN y = gnt3 ELSE '0' ;
END Behavior ;
```

圖 84(b) 仲裁器的VHDL程式碼

```
.
.
.
PROCESS( y )
BEGIN
    IF y = gnt1 THEN g(1) <= '1' ;
    ELSIF y = gnt2 THEN g(2) <= '1' ;
    ELSIF y = gnt3 THEN g(3) <= '1' ;
    END IF ;
END PROCESS ;
END Behavior ;
```

圖 85 授與訊號的錯誤 VHDL 程式碼



```
.  
.   
.   
PROCESS( y )  
BEGIN  
    g(1) <= '0' ;  
    g(2) <= '0' ;  
    g(3) <= '0' ;  
    IF y = gnt1 THEN g(1) <= '1' ;  
    ELSIF y = gnt2 THEN g(2) <= '1' ;  
    ELSIF y = gnt3 THEN g(3) <= '1' ;  
    END IF ;  
END PROCESS ;  
END Behavior ;
```

圖 86 授與訊號的正確 VHDL 程式碼

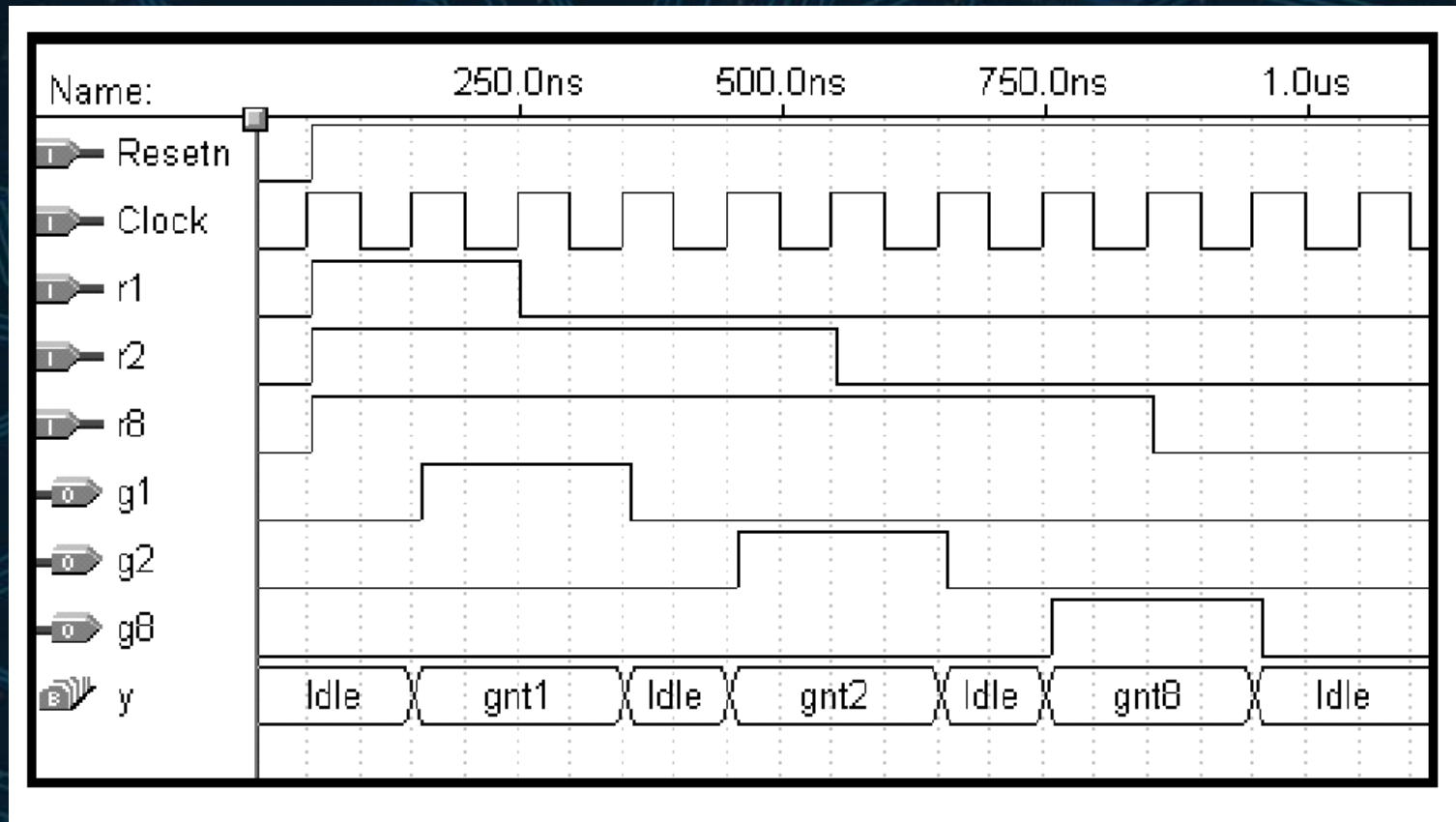


圖87 仲裁器電路的模擬結果。



實作於FPGA Implementation In An FPGA

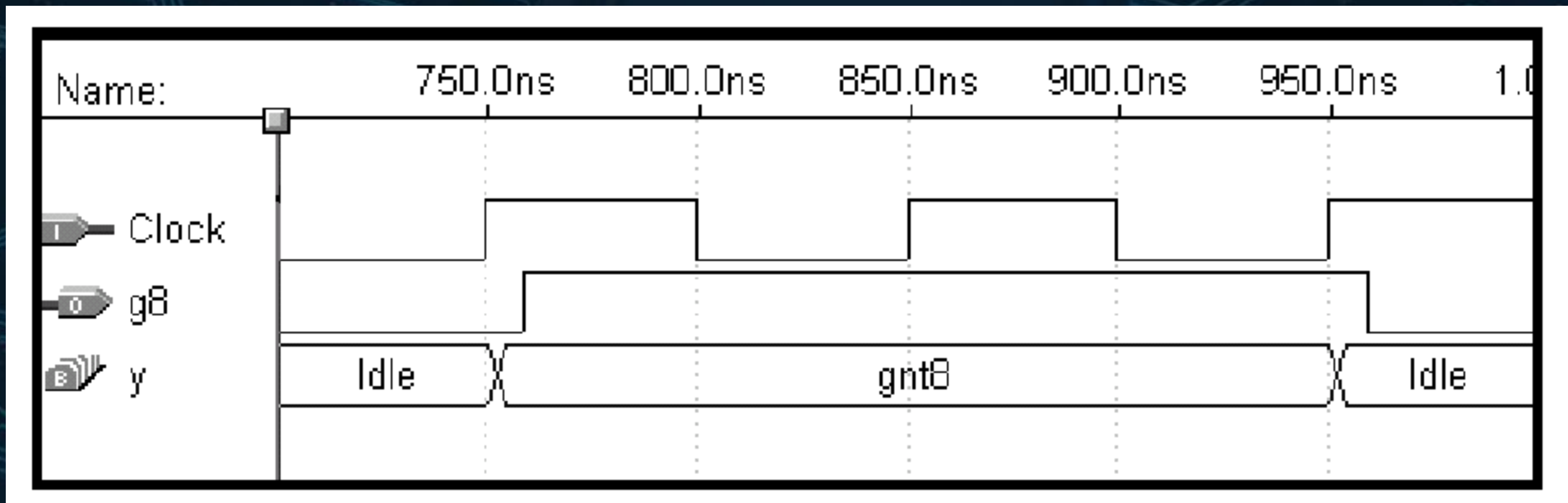


圖88 仲裁器電路的輸出延遲。



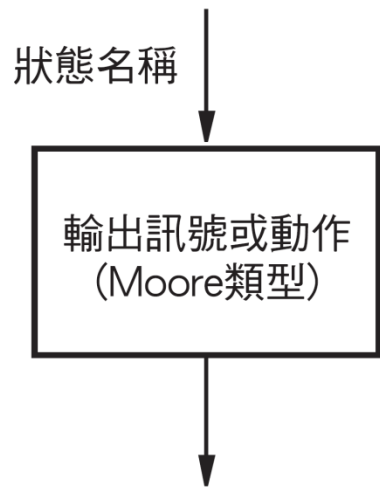
二十四、演算法狀態機器圖 Algorithmic State Machine (ASM) Charts

- 本章所用的狀態圖和表可以很方便地描述只有少數輸入和輸出的FSM行為。
- 對較大的FSM，設計者通常用不同的表現形式，稱為**演算法狀態機器圖** (algorithmic state machine chart, ASM chart)。
- ASM圖是一種流程圖，可以用來表示FSM的狀態轉換和產生的輸出。ASM圖的三種單元如圖89所示。

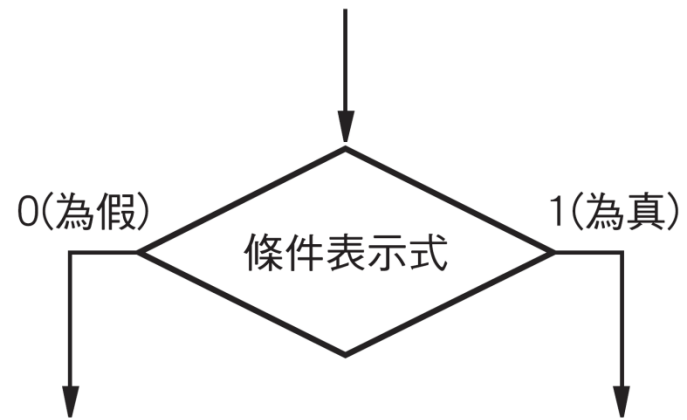


二十四、演算法狀態機器圖 Algorithmic State Machine (ASM) Charts

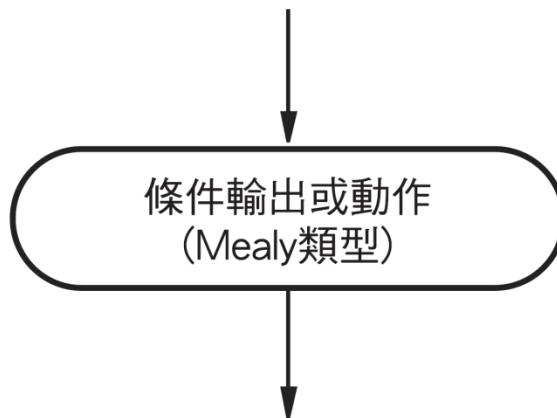
- **狀態方塊**——表示FSM狀態的矩形。
 - 輸出只和定義狀態的狀態變數值有關，稱為**Moore輸出(Moore output)**。
- **決定方塊**——菱形表示欲測試的狀態條件表示式，根據結果決定離開的路徑。
- **條件輸出方塊**——橢圓形代表輸出訊號為**Mealy類型**。
 - 輸出和狀態變數值與FSM輸入有關，稱為**Mealy輸出(Mealy output)**。
- **ASM區塊(ASM block)**，指單一狀態方塊和任何可能連接的決定和條件輸出方塊。



(a) 狀態方塊



(b) 決定方塊



(c) 條件輸出方塊

圖 89 用於ASM圖的單元

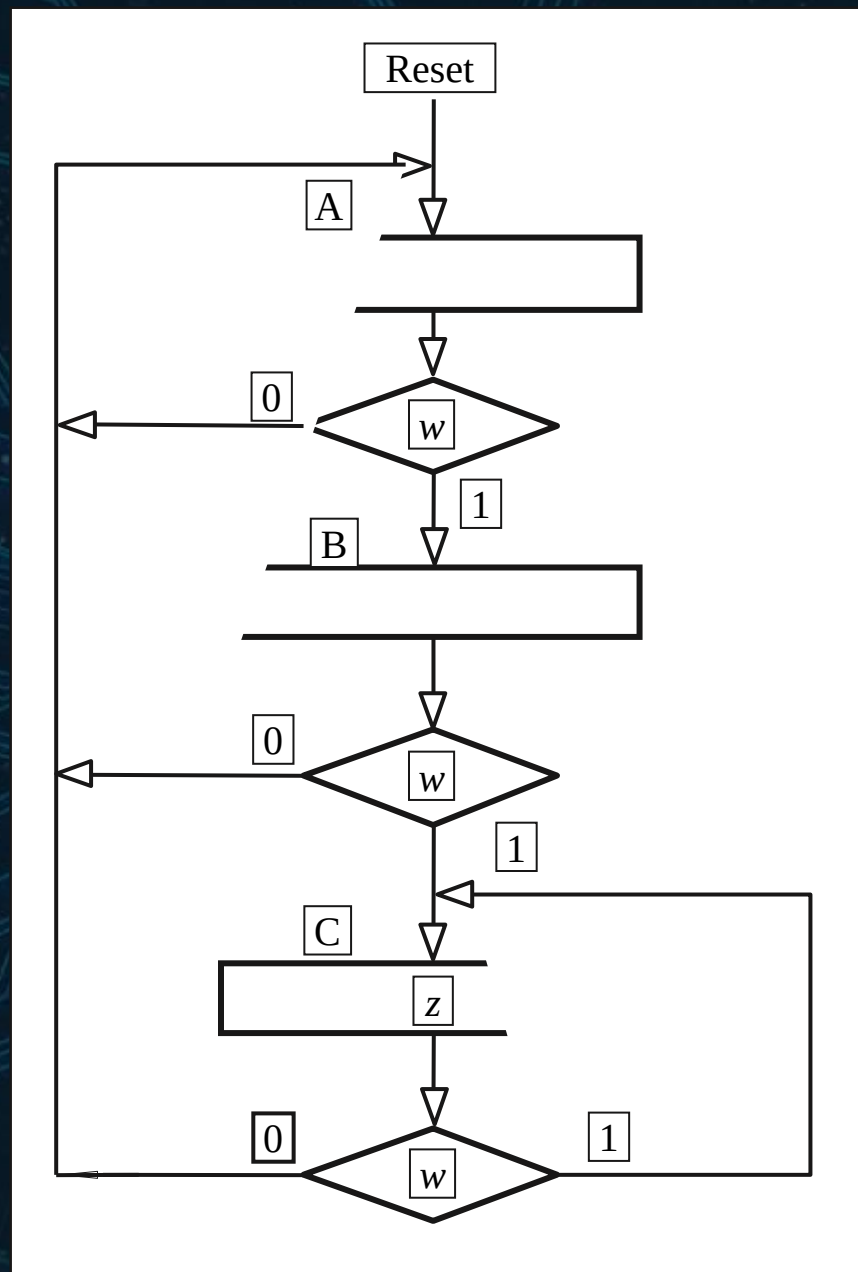


圖 90 圖3的FSM的ASM圖

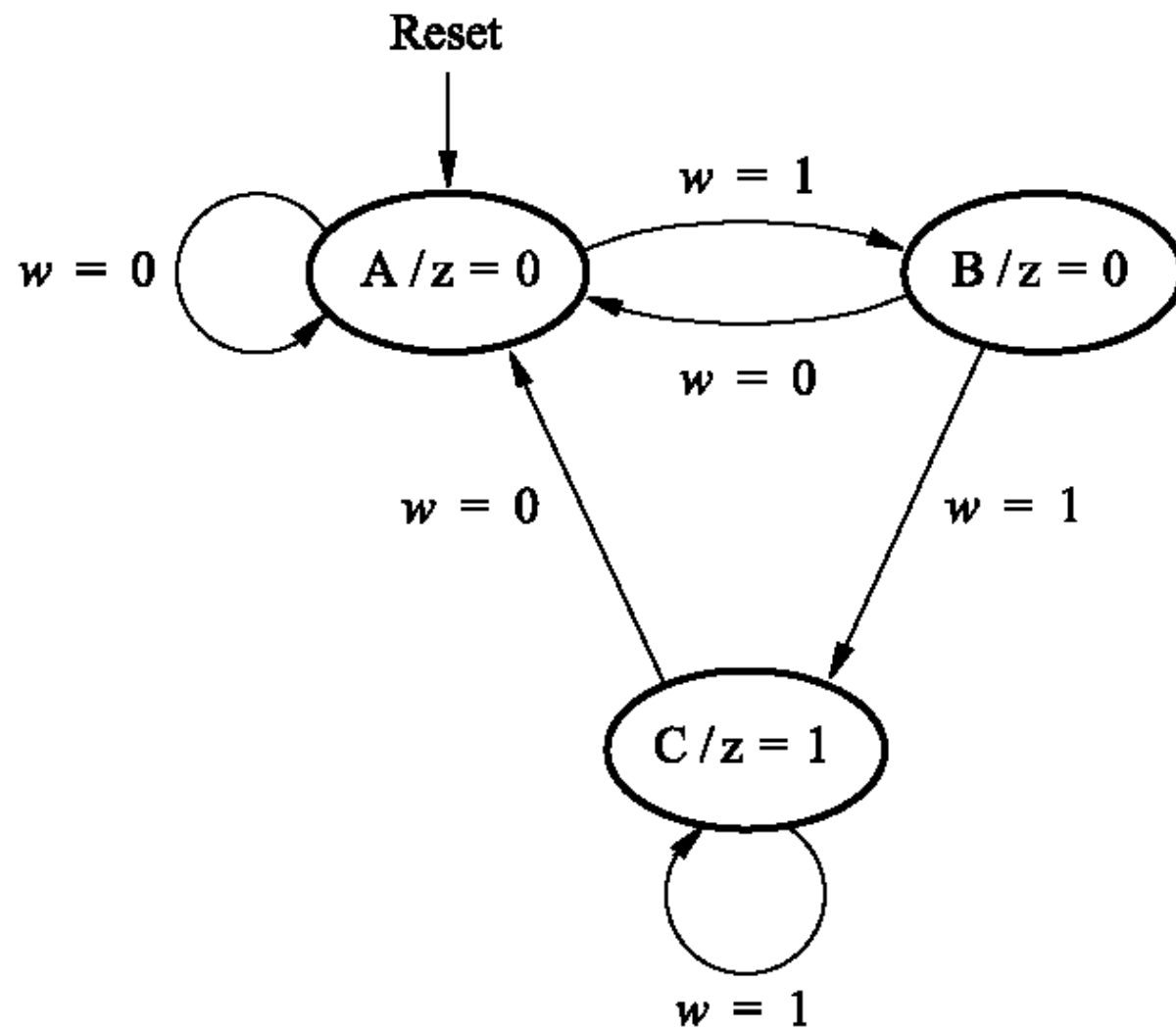


圖3 簡單順序電路的狀態圖。

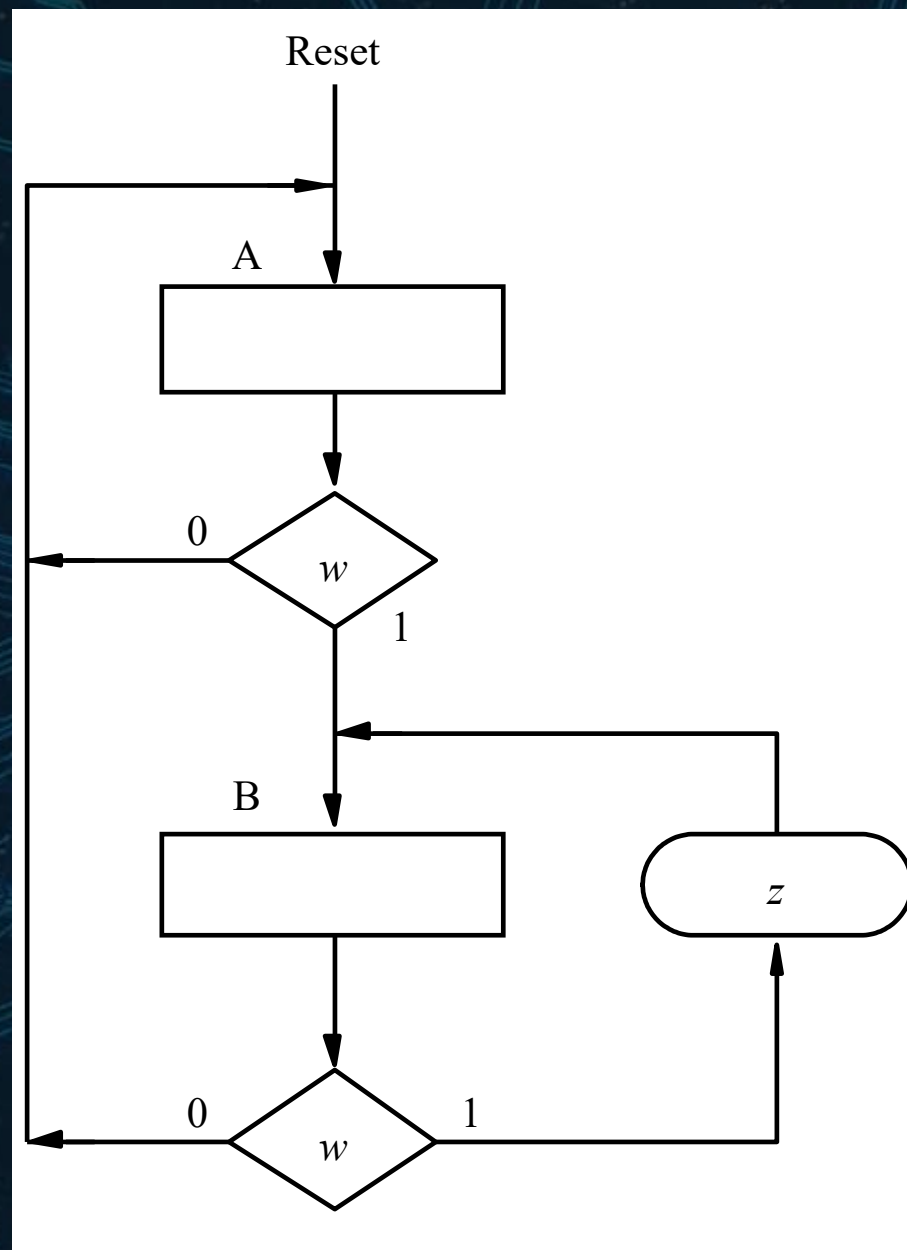


圖91 圖52的FSM的ASM圖

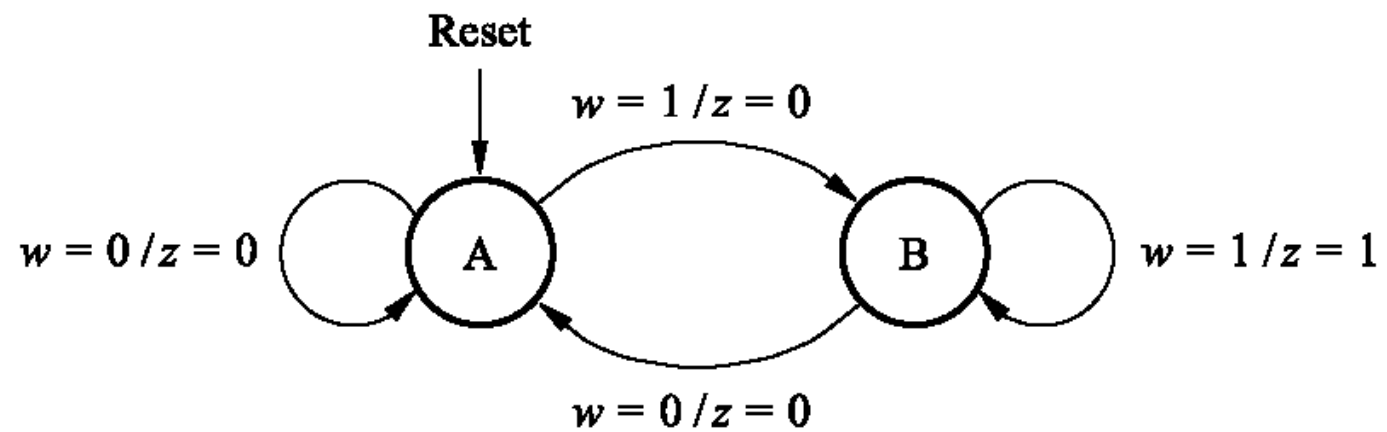


圖52 實現圖44工作的FSM的狀態圖。

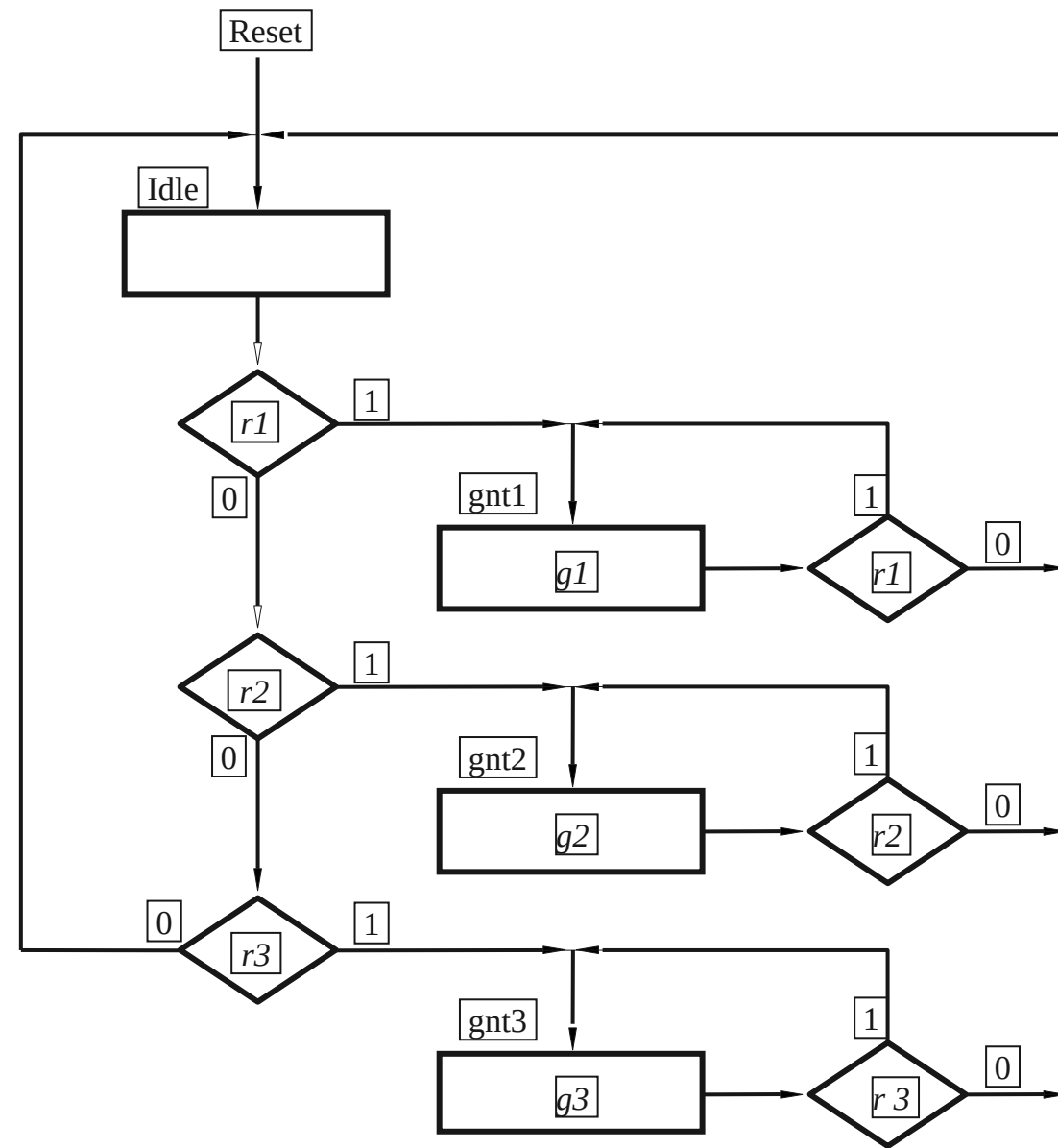


圖92 圖83的FSM的ASM圖